



University of
Stavanger

FACULTY OF SCIENCE AND TECHNOLOGY

BACHELOR'S THESIS

Study programme/specialisation:

Data - bachelorstudium i ingeniørfag

Spring/ ~~Autumn~~ semester, 2018

~~Open~~ / **Confidential**

Author:

**Markus Fjellheim
Nils Magne Fossåen**

.....
(signature of author)

.....
(signature of author)

Programme coordinator: **Tom Ryen**

Supervisor(s): **Terje Per Kårstad**

Title of bachelor's thesis:

Simulating Classical Mechanics on a Web Platform

Credits: **20 points**

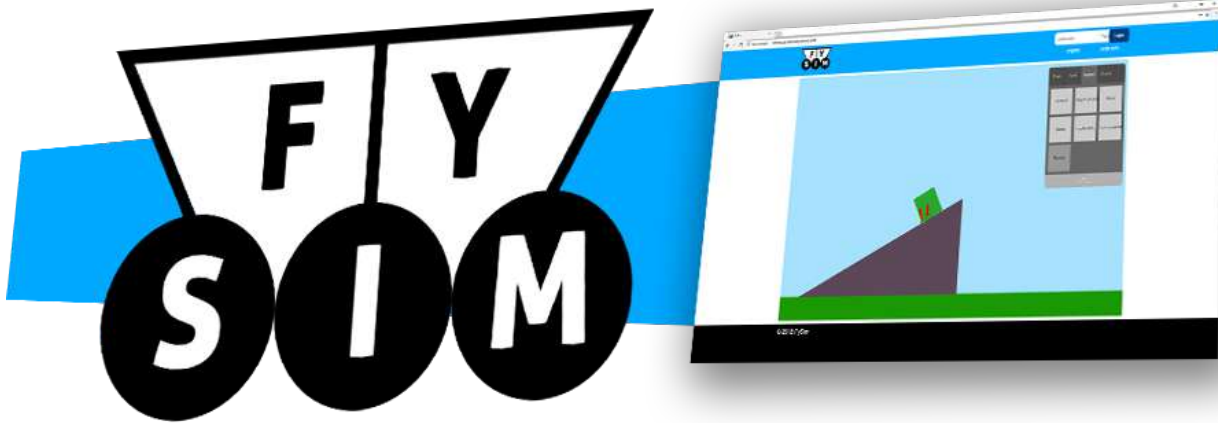
Keywords:

**physics, simulation, single page app, collision
detection, collision resolution, javascript,
FYS100**

Number of pages: **113**

+ supplemental material/other: **codebase.7z**

Stavanger, **14. mai 2018**
date/year



Simulating Classical Mechanics on a Web Platform

FYS100 has historically had a high failure percentage and many students have complained that it is a curriculum that is hard to grasp. We believe there is a need for a tool that can develop both understanding and intuition and provide and share specialized information about applicable problems and good visualizations of these.

Developed by

Markus Fjellheim and Nils Magne Fossåen,
University of Stavanger 2018.

Demo:

<http://nmfoss.pythonanywhere.com/>

Vision

A basic course in physics will teach the students to break down what happens in the physical world into basic physical components. We learn about forces, masses, momentum and energy and how these simple components interact in larger systems.

Since everyone learns differently, some students are fine with only theory while others learn better with visual tools like drawings and animation, different tools are required for each learning need. At the various physics courses we are given access to various learning tools. The problem with the currently existing tools, is that they are often very specialized. An example of such a tool is a simulation of a pendulum (1). Using this tool, we can see various properties of the system, like energy conservation. We learn about pendulums, but pendulums only. Some tools exist where you can learn about pulleys, and other tools exist where you can learn about collisions. Separating the scenarios like this has the advantage that we learn about the different components in an isolated environment, but prevents us from seeing how more complex systems interact with each other. Referring to the pendulum simulation, if we for example want to see how adding a second pendulum at the end of the first pendulum would look like, we would have to search for an entirely new simulation, one that may be missing a feature such as being able to adjust gravity.

There is a need for a more general simulation that can show how objects interact in more complex systems where the user can manipulate the system in any way he/she would like. The problem with the more general simulations that exist is that they are designed for animation making or video games. Since a general simulation is a chaotic system, simplifications will inevitably be made. The simplifications might lead to breaking the principle of energy conservation or translational and angular momentum conservation (the latter is very common). A simulation that can both simulate general mechanical systems and simulate them accurately enough to be compatible with the theory taught in class would give students a better intuition of the concepts in question. A simulation like this is what we have developed.

To make the simulation available for the students, it will be distributed via a web application. The teacher would also through this application be able to publish scenarios and problems for the students to solve, and the students can then hand in solutions through the application. However, the main purpose for the application is to function as a supplement to the curriculum, a way for the student to acquire a more intuitive understanding of mechanical physics. The curriculum we develop the tool for is the FYS 100 curriculum course at UiS. Most of the exercises in this course is simplified to two dimensions so a two-dimensional simulation is sufficient.

Table of Contents

Simulating Classical Mechanics on a Web Platform.....	2
Vision.....	3
1 Reading map.....	7
Chapter 2, Introduction.....	7
Chapter 3, Front-end.....	7
Chapter 4, Back-end.....	7
Chapter 5, Database.....	7
Chapter 6, Theory.....	7
Chapter 7, Process.....	7
Chapter 8, Conclusion.....	7
Chapter 9, References.....	7
Chapter 10, Word list.....	7
2 Introduction.....	8
2.1 The web application.....	8
2.2 Web development.....	8
2.3 Python back-end development.....	9
2.4 MySQL database.....	10
2.5 Front-end development.....	11
3 Front-end.....	12
3.1 The single page application architecture.....	12
3.2 Initialization of assets.....	13
3.3 Routing.....	15
3.4 Graphical User Interface (GUI).....	15
3.5 Front.....	18
3.5.1 Simulator functionality.....	19
Draw.....	20
Tools.....	21
Inspect.....	22
Scene.....	23
Hotkeys.....	24
3.5.2 Simulator technology.....	24
3.6 Messages.....	25
3.7 Scenes.....	26
3.8 Groups.....	27
3.9 Forum.....	27
3.10 General GUI elements and utilities.....	28
4 Back-end.....	30
4.1 Architecture.....	30
5 Database.....	31
5.1 Architecture.....	31
6 Theory.....	32
Focus.....	32
The first axiom has to do with symmetry.....	32
The second axiom has to do with conservation of momentum.....	32
The third axiom is conservation of energy.....	33
Code architecture.....	33
6.1 Interface.....	33
6.2 Graphics.....	33

6.2.1	Triangulate polygon.....	37
6.2.2	Vertex buffer.....	38
6.2.3	Element buffer.....	38
6.2.4	Shader program.....	38
6.2.5	Mesh handler.....	39
6.2.6	GLSL.....	41
6.2.7	Debug renderer.....	42
6.3	Kinematics.....	44
6.3.1	What is a polygon?.....	44
6.3.2	The animation.....	45
6.4	Collision detection.....	50
6.4.1	Why collision detection?.....	50
6.4.2	Overlap based collision detection.....	50
6.4.3	Motion based collision detection.....	50
6.4.4	Continuous collision detection.....	51
6.4.5	Broad collision detection.....	53
6.4.6	Narrow collision detection phase.....	57
6.4.7	Update shadow.....	58
6.4.8	Shadow optimization.....	60
6.4.9	Collision box optimization.....	66
6.5	Collision resolution.....	67
6.5.1	Properties of polygons.....	67
6.5.2	Normal collisions.....	67
6.5.3	Friction.....	69
6.5.4	Compensators.....	70
6.5.5	Displacement resolution.....	74
6.6	Tools.....	74
6.6.1	Default.....	74
6.6.2	Move tool.....	75
6.6.3	Custom shape spawner.....	75
6.6.4	Polygon drawing.....	76
6.6.5	Static polygons.....	76
6.6.6	Show forces.....	76
6.6.7	Hinges.....	76
6.6.8	Ropes.....	77
6.6.9	Pulleys.....	78
6.6.10	Undo redo.....	80
6.7	Definitions and proofs.....	80
6.7.1	Corner.....	80
6.7.2	Polygon.....	80
6.7.3	Mesh.....	81
6.7.4	Theorem 1.....	82
6.7.5	Corner segment are colliding.....	83
6.7.6	The separating axis theorem.....	88
6.7.7	The tunneling problem.....	90
6.7.8	Concave corners and holes.....	92
6.7.9	You can always split a polygon.....	93
6.7.10	Collision formulae.....	95
6.7.11	Area of a polygon.....	100
6.7.12	Center of mass of polygon.....	101

6.7.13	Moment of inertia of polygon.....	105
6.7.14	Random seed generator.....	108
6.7.15	Variants of applying impulses and displacements.....	108
	Apply impulse.....	108
	Displace by impulse.....	108
	Set relative velocity.....	109
7	Process.....	111
8	Conclusion.....	113
9	References.....	113
10	Word list.....	115

1 *Reading map*

Chapter 2, Introduction

A short introduction to the general process of developing a web based application going through the different levels of the stack: Front-end, Back-end and Database.

Chapter 3, Front-end

The process of developing a Single Page Application as the front-end model of a web based app and a tour of the functionality of the app.

Chapter 4, Back-end

Structure and architecture of back-end technology. How the Python Flask app is constructed modularly using Blueprints and the app and a modular underlying file structure.

Chapter 5, Database

About the database structure and architecture and the reasoning behind.

Chapter 6, Theory

Diving into the simulator theory and modular structure of the simulator. Graphics, kinematics, collision detection and resolution and certain definitions needed within this framework.

Chapter 7, Process

About the process of this project. Topics include debugging, testing, version control, etc.

Chapter 8, Conclusion

Did we accomplish what we set out to do?

Chapter 9, References

A reference list.

Chapter 10, Word list

Index of acronyms and word definitions used in this report.

2 Introduction

2.1 The web application

While making this tool we had the choice of making a web application or a mobile application. We quickly decided to go for a web application because of its universality vs. the more complicated and less universal way of developing a mobile application. Mobile application developing at best divides into two major platforms, Android and iOS. Then there are the different versions of these major systems and so on. Also developing a web application builds on knowledge and experience we already have achieved during the course of our education so the choice was easy. But we also think it is the best choice overall. It meets the most users and the drawbacks of not making a mobile application (push-messages and device specific hardware access) are not big considering what kind of application we are making.

What kind of web application (app from now on) do we see fit for the project? First of all the audience we are targeting will know about this app through the channels at school so we are not developing something that needs to be advertised and found through search engines. This makes us free to disregard focusing on search engine optimization oriented web designs. We are developing content relevant only within the app itself.

Another criteria is to keep the app as "server-light" as possible. Doing this will keep hosting prices low and also be beneficial for the end user experience as it does not rely on fast connection speeds or server load. So the app will be "client-heavy" meaning most of the logic will reside on the client.

A good architecture that meets our needs is the single-page application architecture. It makes the application more "server-light" as it reduces the content needed to be transferred to the client to the minimum. It also gives a better user experience as it behaves more like a desktop application in the sense that it does not need page refreshing for every feature clicked. The single-page app is not very search engine friendly, but as stated above this is not a criteria for our needs.

2.2 Web development

Web development is sometimes referred to as a stack because it is often a stack of technologies that communicate with each other in order to give the end user experience. This stack often consists of three major levels: front-end, back-end and database. A front-end developer works in the user experience layer with technologies like HTML, CSS and Javascript (JS). The back-end is where the business logic occurs and handling of the data input and output from the database level. Some major

technologies for back-end developing is PHP and .NET, but here there are a lot more choices to be made and mostly rely on what is supported by your hosting service and server architecture. The database layer is where the data is stored and depending on the type of app developed this can be transactions, page content, messages and so on. Popular technologies here are MySQL, Oracle and MongoDB among others. The most popular model for a database is a relational database management system, but as with MongoDB document store models are also gaining popularity.

For this project we have decided to go with Python as our back-end technology and MySQL relational database management system. This is mostly due to our use of these technologies prior in our education and is therefore where we can start to develop as quickly as possible. Also both of these technologies meet our requirements for our app.

2.3 Python back-end development

The standard interface for Python web server applications is to use the WSGI (Web Server Gateway Interface). Here we could start building from scratch, but we choose to use a framework, namely Flask (22). We were already familiar with this framework and its design makes it applicable for quick prototyping and easy development. It also supports modular architecture which is beneficial.

2.4 MySQL database

For the database structure we used MySQL Workbench as a tool to create our model of the database. Here we also tried to follow the principle of modular design using a naming scheme for tables related to the different modules. The database is a standard normalized relational database.

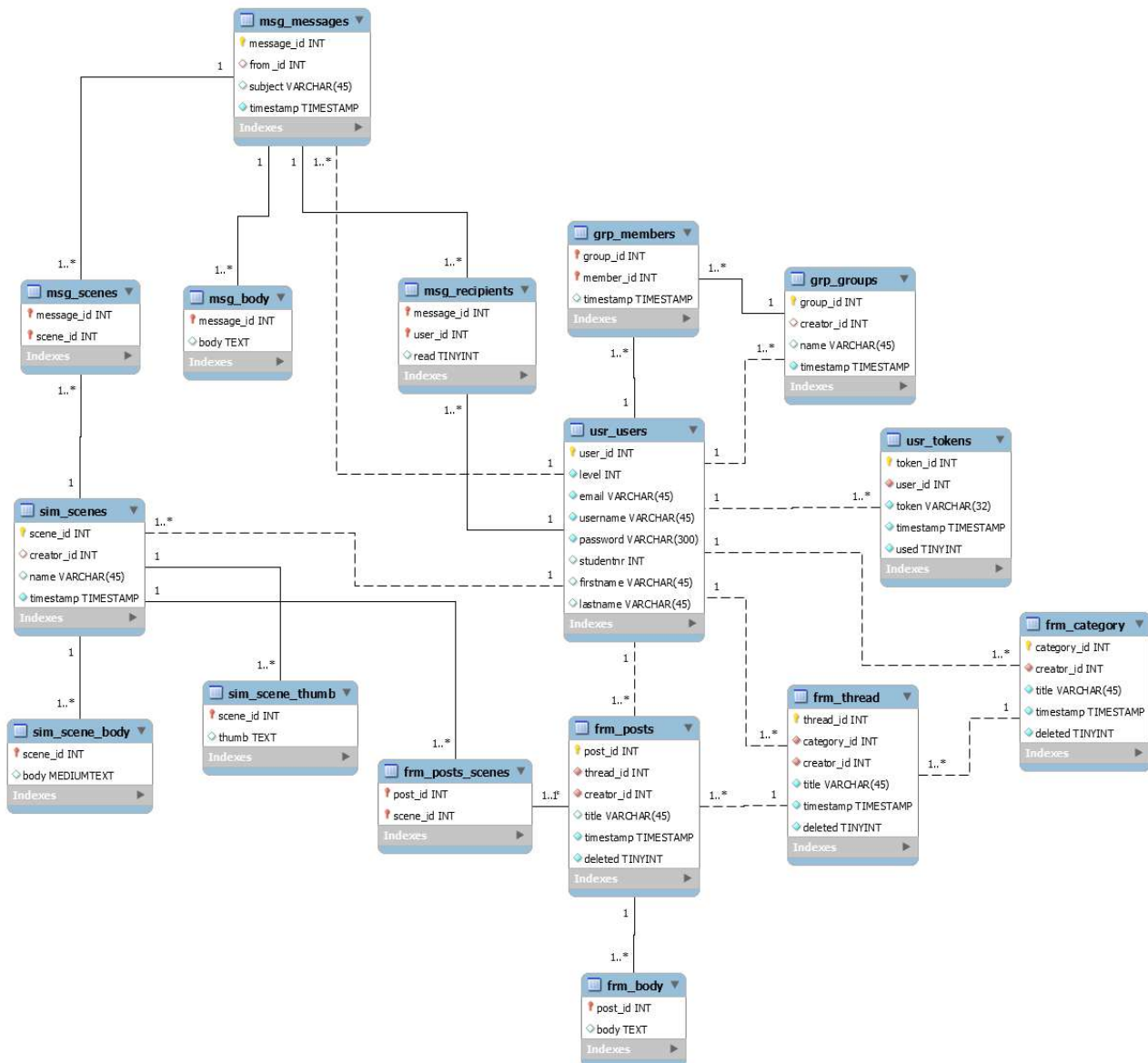


Illustration 1: Database model

2.5 Front-end development

The major part of this project was the front-end technologies. Using mainly JavaScript as programming language. The development pipeline was from front-end through back-end to database. A data need was developed at the front-end and from there the necessary structure to ensure that data need was met was developed through back-end to database. We have used JavaScript for virtually all the front-end development and generation of HTML. This is consciously done so, and is in conflict with the common approach to front-end development where separation of concerns between logic and layout has been the norm. More about the reasoning behind this choice later.

3 Front-end

Unless otherwise stated all reference to programming in this chapter is in JavaScript.

3.1 The single page application architecture

A single page application (SPA from now on) differs greatly from traditional web applications in the way it delivers the end user experience. Whereas traditional web development serves a server-side rendered web page at a specific URL endpoint the SPA relies on client-side rendered content. The client has to handle the state of the application and fetch the necessary content to serve the current state. In a SPA this is referred to as routing. A URL specifies a desired state, the router takes this URL and decompose it to variables and parameters and triggers the desired state for the SPA.

In our SPA all URL routes deliver the exact same template (discounting the specified AJAX URL endpoints that deliver the JSON data for the app), an index.html (`app\modules\_static\index.html`) document containing only header information, JS assets for loading and handling JS modules and CSS style sheets.

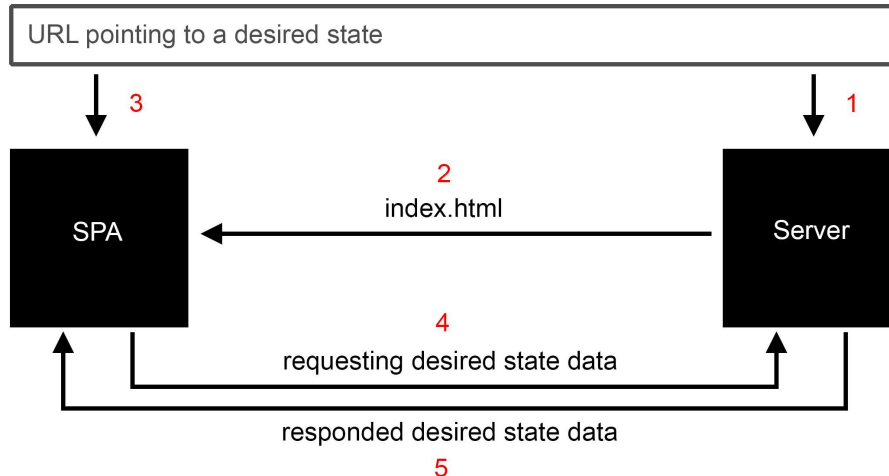


Illustration 2: Simple model of data flow in SPA

Following the flow in the above illustration:

- 1) the client starts with a HTTP request to the server
- 2) the server serves the index.html (`app\modules\_static\index.html`) page (no matter what the request is, front-page, forum, etc.)

- 3) the SPA routing module decompose the URL present in the client browser and calls the callback function tied to that particular URL (or calls a default function if no match)
- 4) the callback function calls a chain of functions that requests needed data from the server via AJAX JSON protocol
- 5) the server responds to each request with the desired data via JSON protocol

3.2 Initialization of assets

When index.html (`app\modules\_static\index.html`) is loaded in the browser, metadata about the site is delivered and a small list of utility functions and 'classes' (JavaScript does not have native classes, but we define functions with the similar properties as classes as such) needed for the SPA to initialize its modules. The modules of the app is:

- frm (forum, `app\modules\frm\_static\init.js`)
- grp (groups, `app\modules\grp\_static\init.js`)
- gui (GUI related functions, `app\modules\gui\_static\init.js`)
- msg (messages, `app\modules\msg\_static\init.js`)
- sim (simulator, `app\modules\sim\_static\init.js`)
- usr (user, `app\modules\usr\_static\init.js`)
- router (routing, `app\modules\_static\Routing.js`)

In addition to these there is a parent module named app (`app\modules\_static\init.js`) which umbrellas these modules. app is given as attribute to the Assets load function (`app\modules\_static\js\Assets.js`) together with a callback function. app (and the other modules) has four attributes important to this process:

- model
- assets

- name
- element

model is an instance of the Model class (`app\modules\_static\js\Model.js`) which gives the functionality to register HTML elements to listen for events triggered by the model such as updated values. Model has three methods available:

- get
- register
- set

Further on in Assets the app model is accessed and a 'init' event is registered on the element and a callback function for this event is defined to set this module as loaded. This is done after all the assets in the model.assets is set to true. Each asset is loaded in as a SCRIPT element in the header and contains at the bottom of the script file a function call to its modules model to set its asset name to True. When this is done it triggers a callback function registered on the model to check if all assets are set to True. This is done because the loading of script files is async and also the onload event of the SCRIPT element appended to the header does not trigger when the script is actually loaded but rather when the tag is appended to DOM.

When all assets are set to True the module is set as loaded in the model.init set to True. This then triggers the callback function sent into Assets together with the module and the module is loaded.

Since the app module is an umbrella module and all the assets are subsequently all the child modules init.js files this functionality is then recursively called for each modules assets. When this recursion is done and all modules assets are loaded and initialized the SPA is ready and a final callback function in the app module is called and starts the router in the Routing module (`app\modules\_static\js\Routing.js`).

3.3 Routing

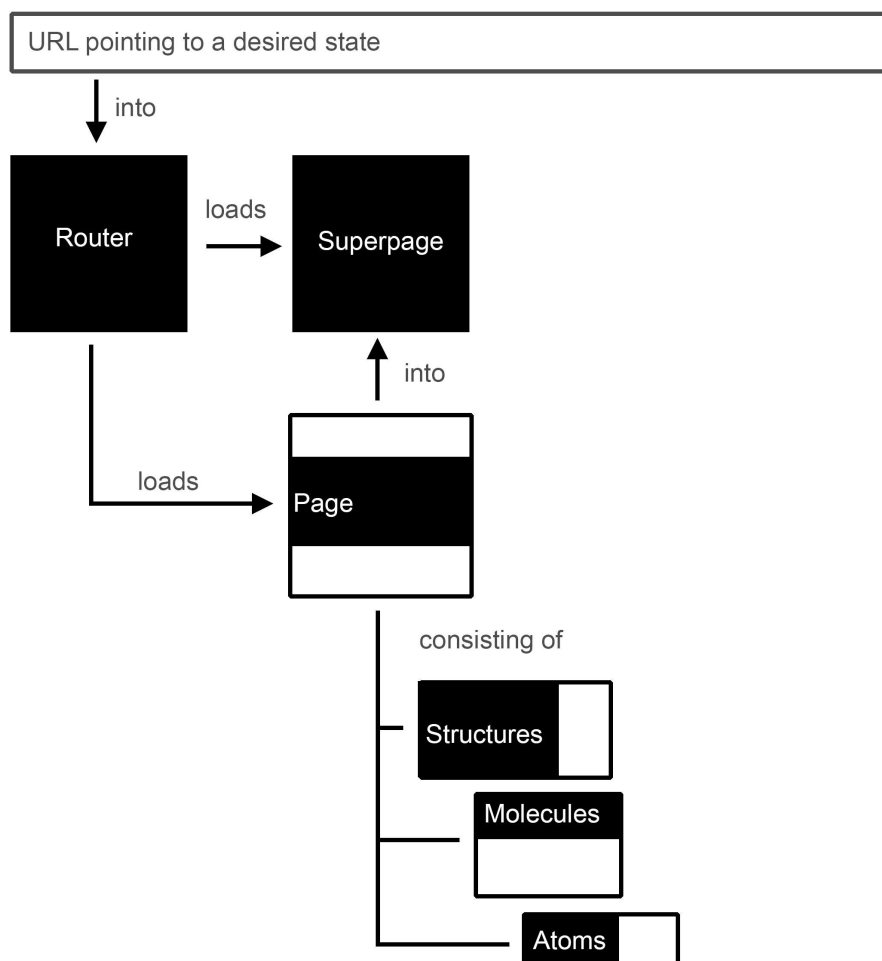


Illustration 3: Simple model of SPA routing in this project

For this project we decided to use a third party module for handling the routing and pushState history management. Navigo is a lightweight and no frills router module that does only one thing, routing, so it fit perfectly as we did not want any extra functionality. It works by specifying callback handlers for desired URLs. In these handlers URL parameters (e.g. ?sceneid=1) are passed on to the callback functions. There is also support for dynamic parameters (e.g. forum/:category_id/:thread_id). The module works by checking the browsers location.href (URL in browser) at an interval and detecting changes. For more specific information about this module we reference to the GitHub repository for Navigo by Krasimir Tsonev (2).

3.4 Graphical User Interface (GUI)

The user experience is developed with the Atomic Design pattern in mind (3). This pattern breaks the design into its constituents and creates building blocks. Combined with the idea of generating

all the HTML with JS we went a bit away from the traditional MVC (Model View Controller) structure. Instead of fragmenting the architecture into HTML templates and JS functions needing a lot of hooks and placeholders and complexity with tunneling data through several layers of templates we decided to try to combine it all in pure JS function calls that returns HTML elements. Using the Atomic Design pattern we developed a structure we named jML (javascript Markup Language).

jML is at core two functions (`app\modules\gui\_static\jML.js`), `tag` and `single` which takes in three and two attributes. For a tag it takes in the tag-name, the content and the attributes. For a single it takes in the tag-name and the attributes. A tag is typically a DIV and a single is for instance an INPUT. The syntax is then as follows:

```
var example = jML.atom.tag('DIV', "Some content...", {'class': 'some_class'});
```

Example contains a HTML element of the DIV kind with "Some content" as a text and with the class 'some_class'. One can also use a list of such HTML elements as content, syntax is then as such:

```
var example2 = jML.atom.tag('DIV', [example], {'class': 'some_class'});
```

Example2 contains a HTML element of the DIV kind containing the DIV element from example.

In addition to these two atom elements, there is a special canvas atom element that is described later.

The different elements of the graphical user interface (GUI) is built using these structural names:

- atom
- molecule
- structure
- page
- superpage

For example the HEADER element of the GUI (`app\modules\gui\_static\jML.js`) consists of molecules and atoms and returns a structure which again is included in a superpage.

All the functionality is contained in these functions, so for instance the login molecule (`usr.jML.molecule.login()`, `app\modules\usr\_static\jML.js`) contains atoms with the input fields and login button and also the click event for the button calling a function that does a simple form check (checking if username and password fields are not empty) and submits the form using `Request ( app\modules\_static\Request.js )` which is a utility function for making AJAX calls. `Request` also takes a callback function which is called when the request is finished. In the callback a response attribute is available and the structure of the response is generalized to look as follows:

```
{
  'header': 'login_result',
  'notifications': [],
  'success': true,
  'data': {
    'username': 'nils',
    'level': 0
  },
  'formFailed': []
}
```

- "header" gives an indication to what this is a reply to.
- "notifications" is a list of notifications triggered on the server, errors and such will be in here, but this functionality we have not implemented yet.
- "success" is a True/False boolean check to see whether the request was successful or not.
- "data" is where any data replied back from the server is contained.
- "formFailed" is a list of form elements that failed when checked on the server.

So if a response is successful as in the above example with the login then we set three variables in the user model using the set functions;

- username
- loggedin
- level

What happens then is that elements in our app that has registered to listen to updates of these variables in the user model gets called and their respective callback functions are triggered. For instance the superpage element (`gui.jML.super.default()`, `app\modules\gui\_static\jML.js`) containing the HEADER, MAIN and FOOTER elements for the page has a listener on the user loggedin variable switching between two different header elements depending on whether loggedin is True or False. So when a user logs in the header switches automatically. Same for other elements listening to these variables so the app becomes event driven and ideally everything is driven through changes in the different modules models.

As per now the app is not fully driven this way, but in future updates more of the data throughput will be through each modules model. This pattern resembles the SAM (State-Action-Model) pattern architecture (4), but as mentioned we have not fully followed this pattern in the app so far.

3.5 Front

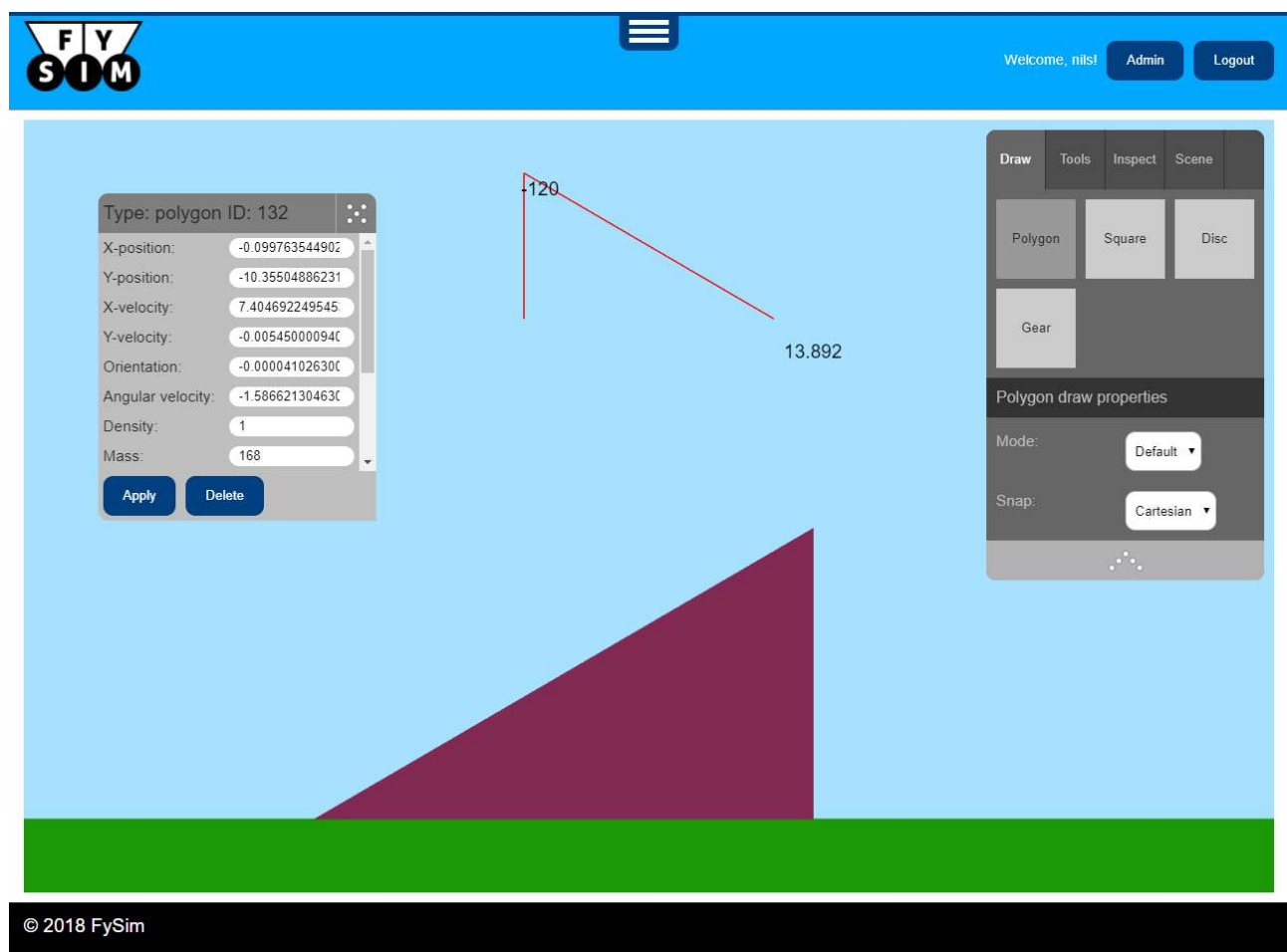


Illustration 4: Screenshot of front-page

The front page consists of a small top header with login and a main section with the "scene" with the physics simulator and a footer section.

Focusing on the simulator you can find the public panel in `app\modules\sim\_static\jML.js`, `sim.jML.structure.public_panel()`. It consists of mainly a canvas element where the simulation is drawn and a toolbar with buttons for the different states the simulator can be set in, it be drawing, inspecting or if logged in, saving and loading of scenes.

The canvas element is a special atom with a series of callback functionality attached to it handling the different tools and inspection states the simulator is set to.

3.5.1 Simulator functionality

In the top right corner of the simulator canvas element there is a panel visible with clickable buttons for changing the states of the simulator. The main categories of these states is:

- Draw
- Tools
- Inspect
- Scene

The simulator initializes in the *"Inspect"* category with *"Default"* as the default state. This state enables the user to click and drag polygons in the simulation showing a force vector for the force which is used to manipulate the object.

Clicking on the category "tabs" on top will show the states in the panel below with big buttons.

Clicking a button activates the state and also enables the user to manipulate any properties linked to the active state in a panel shown below the state buttons.

Draw

Under "Draw" we find the states that create new polygons.

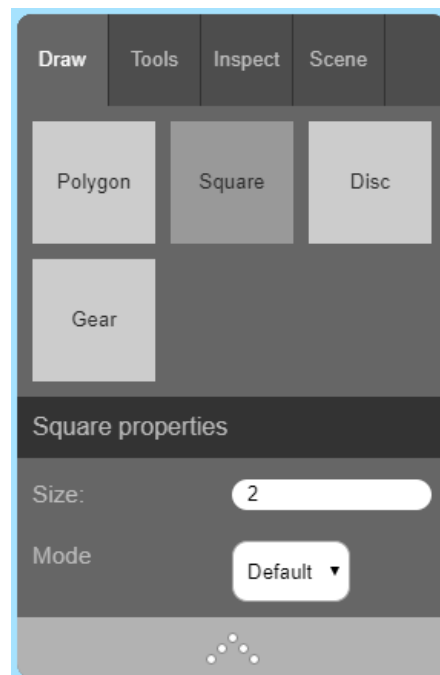


Illustration 5: Simulator Draw states panel

- "Square", "Disc" and "Gear" are predefined polygon shapes with properties that can be changed, like size, resolution and mode. The shapes can be made with three different modes;
 - "Default" which means the polygon is a free body in the simulation.
 - "Hinged" which means the polygon is hinged to an already existing polygon in the simulation.
 - "Fixed" which means the polygon is hinged at a fixed location and immovable in the simulation.
- "Polygon" enables the user to draw custom polygons. Properties here is mode, same as above and snap. Snap enables the user to create polygon sides that follows a grid. It is currently possible to follow a Cartesian or polar coordinate grid. To start the drawing of a new polygon, left click an open space in the scene. Left click to place subsequent corners forming the outline of the polygon. Right click to undo the creation of the corners. Press enter to finish a circumference. If a circumference is finished. To create holes inside a

polygon, draw circumferences inside the already existing outline. Finish these circumferences as before by pressing enter. Press enter again to complete the creation of the polygon. If the shape is overlapping already existing polygons, the polygon cannot be completed.

Tools

Here we find states that enables functionality to be put into the simulation.

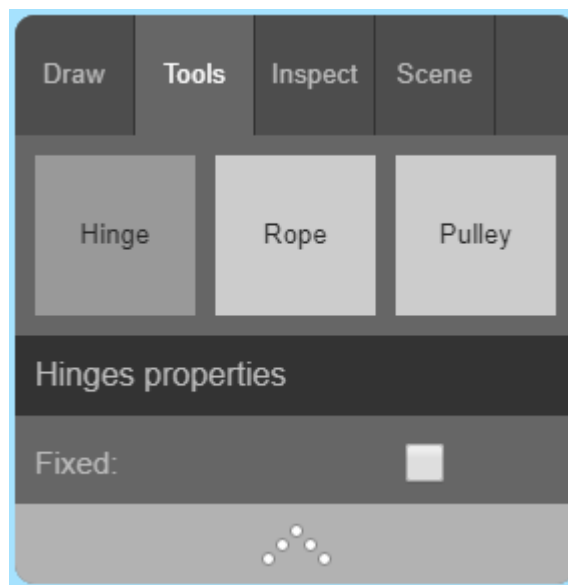


Illustration 6: Simulator Tools states panel

- *"Hinge"* enables the user to hinge polygon together or to a fixed location using the fixed property. Hinges have friction which can be edited using the *"Tool properties"* state explained later.
- *"Rope"* enables the user to tie two polygons together with a fixed perimeter that can also be edited using the *"Tool properties"* state.
- *"Pulley"* is used to link two rope endpoints together making a pulley effect letting the rope slide back and forth over the pulley connection.

Inspect

States that enables inspection and manipulation of the objects in the simulation.

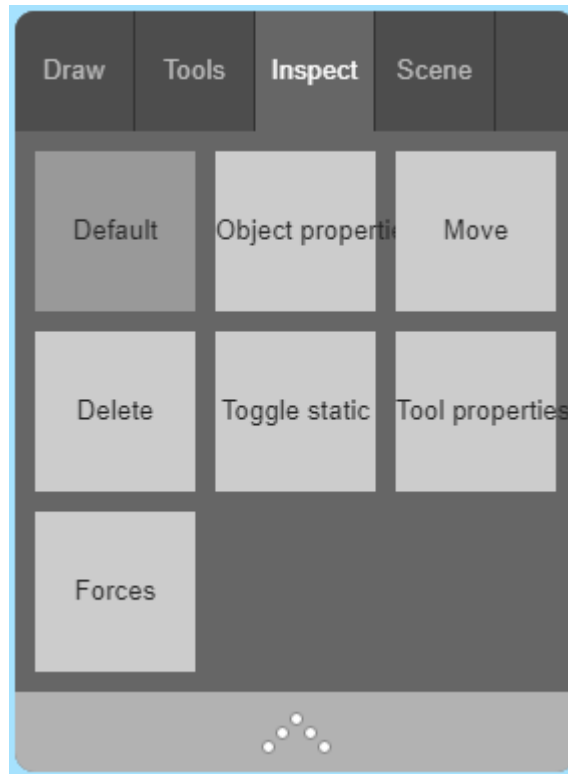


Illustration 7: Simulator Inspect states panel

- *"Object properties"* is a state that enables the user to click on a polygon in the simulation and get a contextual menu with list of properties about that polygon. These properties are also editable. Example properties is position, orientation and velocity. One can also delete the polygon from this contextual menu.
- *"Move"* state differs from *"Default"* state in that the movement follows the cursor explicitly and one can also rotate the polygon using the middle scroll-wheel on the mouse.
- *"Delete"* deletes the polygon you click in the simulation.
- *"Toggle static"* enables the user to stick the polygon you click to a static location in the simulation. Is also possible to edit through the *"Object properties"*.
- *"Tool properties"* is similar to *"Object properties"* but it applies to tools such as hinges or ropes.

- *"Forces"* enables a visualization of force vectors that affect the clicked polygon. Vector and decomposed vectors are shown.

Scene

States and operations affecting the simulation or scene itself.

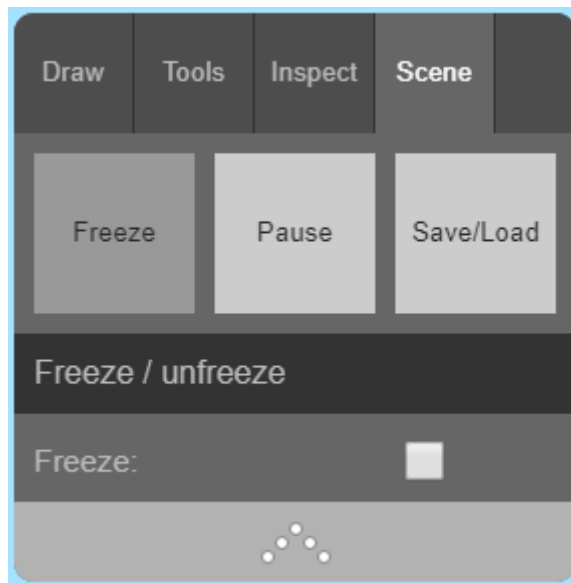


Illustration 8: Simulator Scene states panel

- *"Freeze"* halts all simulation. The user can apply forces and do manipulations in this state, but the effects will not take place until the simulation is "unfrozen". This is a deterministic way of pausing the simulation.
- *"Pause"* pauses the simulation, but enables the user to apply changes to polygons like movement and such in real time. This state is not deterministic, meaning that a simulation will run with slightly different values each time it is unpaused due to the ability to move and manipulate objects.
- *"Save/load"* enables the user to save and or load a simulation. This operation requires the user to register with a profile.

Hotkeys

- w, a, s, d or up, left, down, right Translate the camera position.
- q, e Rotate camera.
- r, Reset orientation of the camera
- comma, numpad plus Zoom in
- period, numpad minus Zoom out
- ctrl+z Undo
- ctrl+shift+z, ctrl+y Redo

3.5.2 *Simulator technology*

Since the physics simulation is distributed via the web, the engine is written in JavaScript and run on the clients' machines. JavaScript is a very fast language and therefore suitable for simulating mechanical systems as it requires a lot of performance to do calculations in real time. For graphics, we chose to use the HTML canvas object and WebGL. WebGL, which is based on OpenGL.

OpenGL is a graphics card API (application program interface), that allows relatively low level control over the graphics calculations which in turn improves performance. The alternative to using WebGL would be to use the higher level API, like the Canvas API. Even though it is easier to use, it gives less control and slower performance.

GLSL is the language used to control the rendering done on the graphics card. The language is used by WebGL and sent to the graphics card.

3.6 Messages

READ	SUBJECT	FROM	DATE	ACTION
☒	Deterministisk pausemodus	markus	2018-04-20 23:41:09	<button>Read</button>
☒	Deterministisk	markus	2018-04-20 01:09:16	<button>Read</button>
☒	Trebuchet_2	markus	2018-04-18 12:35:25	<button>Read</button>
☒	Se på denne.	markus	2018-04-17 21:41:19	<button>Read</button>
☒	fdsa	markus	2018-04-17 08:18:28	<button>Read</button>
☒	Her er en scene	markus	2018-04-17 08:18:00	<button>Read</button>
☒	Demonstrasjon	nils	2018-04-17 08:17:56	<button>Read</button>
☒	Polar and cartesian coordinates bug	markus	2018-04-13 12:55:17	<button>Read</button>
☒	Slette scener?	markus	2018-04-12 19:29:13	<button>Read</button>
☒	Message system	markus	2018-04-10 20:23:17	<button>Read</button>
☒	Hello test group	markus	2018-04-10 20:19:40	<button>Read</button>

© 2018 FySim

Illustration 9: Screenshot of messages page

We have a simple messaging system in our app where you can read and send messages to all the systems users and attach scenes from My scenes.

All GUI functions regarding messages can be found in `app\modules\msg\_static\jML.js`.

The "New message" button in the My messages structure (`msg.jML.structure.my_messages()`, `app\modules\msg\_static\jML.js`) has the following event listener for click:

```
new_button.addEventListener('click', function(){
  var new_message = msg.jML.structure.new_message(),
  modal = utils.modal("New message...", new_message);
  modal.addEventListener('formSuccess', function(event){
    modal.remove();
  });
});
```

First, `new_message` function is called, it returns the `new_message` HTML and its functionality, then a utility function is called which is a window wrapper where we can put the `new_message` and using a event driven system we remove the modal with the `new_message` if there is a `formSuccess`

event bubbled up.

The messages inbox is a simple listing of all users messages with a indicator if the message is read or not and a button to open and read the message.

Future improvements here is clicking on From username will automatically open a message with user as recipient. Switch to view messages per user as a chat dialogue. Pagination of the messages listing.

3.7 Scenes

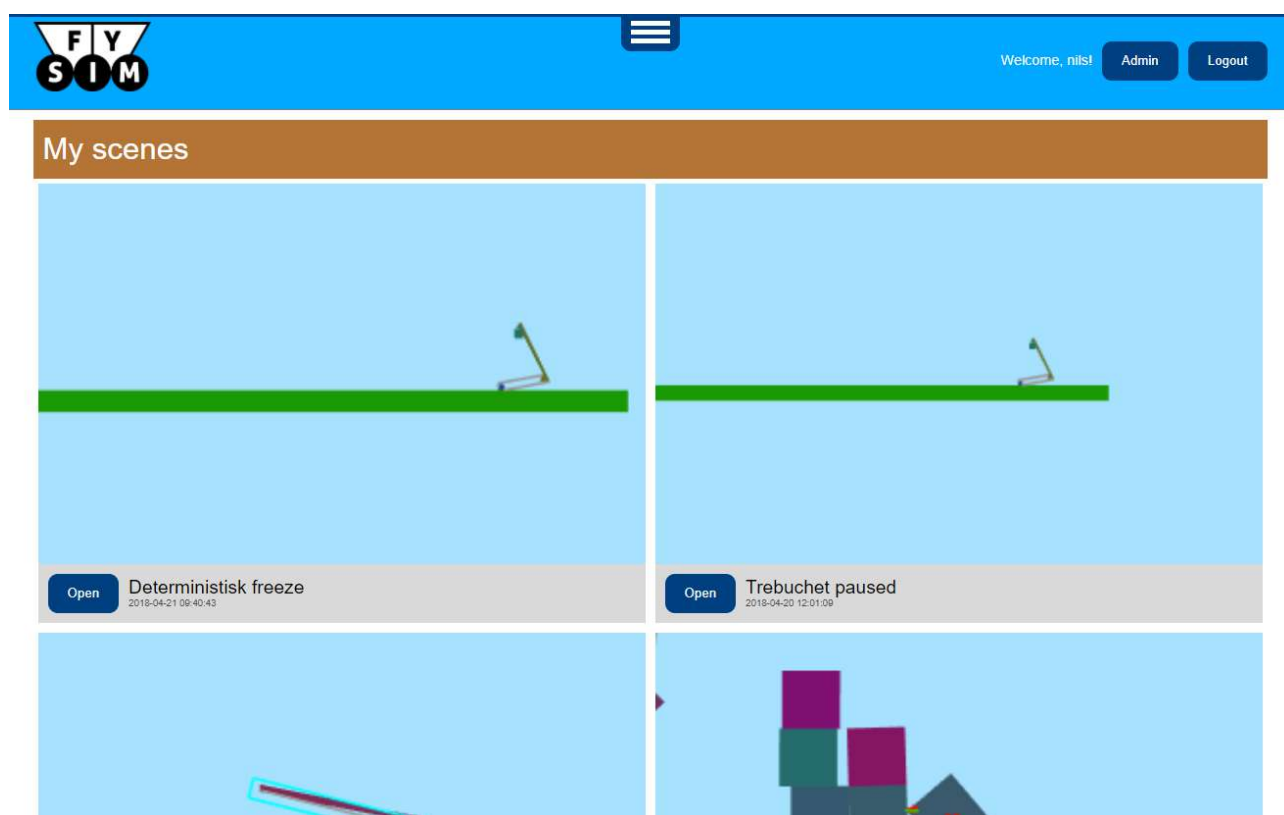


Illustration 10: Screenshot of my scenes page

All GUI functions regarding scenes can be found in `app\modules\sim\_static\jML.js`.

A listing of users scenes with button linking to the scene URL (`sim.jML.structure.my_scenes()`). A Request is sent for a listing of users scenes, in the callback the response is then looped and a scene element (`sim.jML.molecule.open_list_element()`) is added to the listing.

Future improvements includes adding pagination.

3.8 Groups

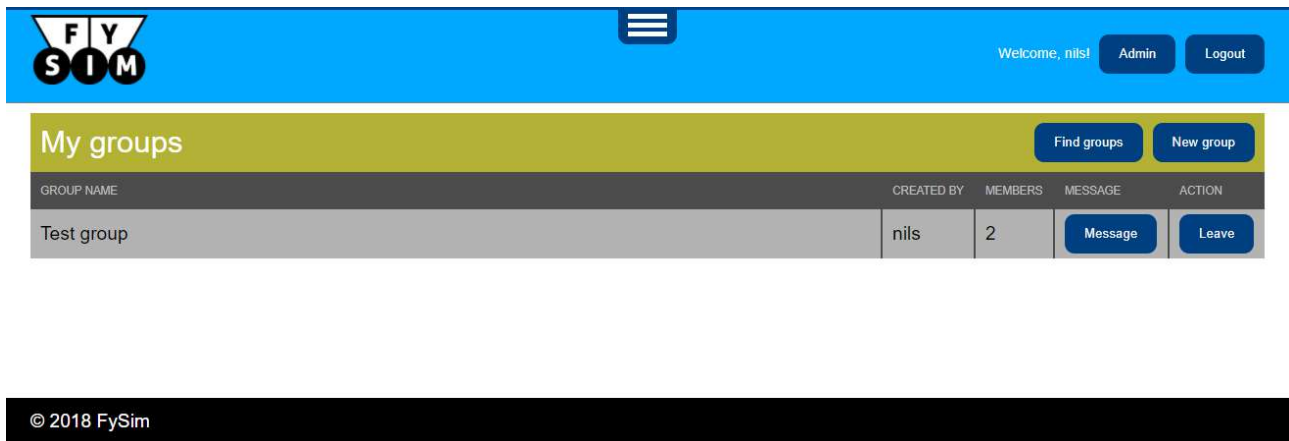


Illustration 11: Screenshot of groups page

All GUI functions regarding groups can be found in `app\modules\grp\_static\jML.js`.

Listing of users joined groups with buttons to send message to group members or to leave the group. Also in the top there is buttons to open a modal window for searching for and joining groups. Depending on user level a button for making a new group is also available ($\text{level} < 2$).

Future improvements includes possibility to see individual members of groups and message them directly.

3.9 Forum

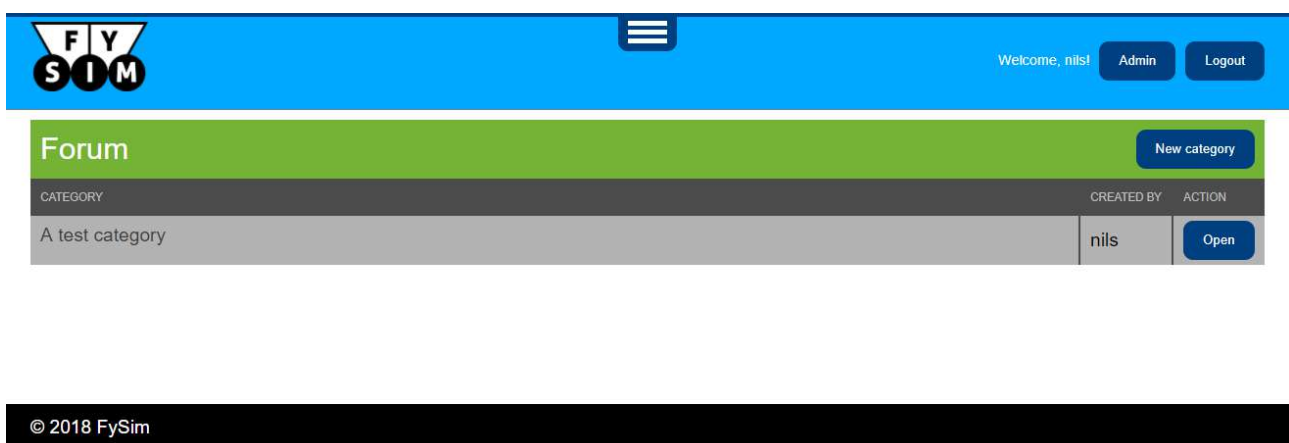


Illustration 12: Screenshot of forum page

All GUI functions regarding groups can be found in `app\modules\frm\_static\jML.js`.

A simple forum with categories, threads and posts. Making new categories and threads is restricted

by user level and the idea is that teacher at level 1 can create categories and student assistants at level 2 can create threads and all users at level 3 can create posts.

Posts can have scenes embedded to them in the same way messages does.

Categories, threads and posts can all be linked to directly by URL.

Future improvements include moderation like deleting or moving posts/threads/categories and search.

3.10 General GUI elements and utilities

Throughout the app there are some general elements that is reused like a modal window and a listing element.

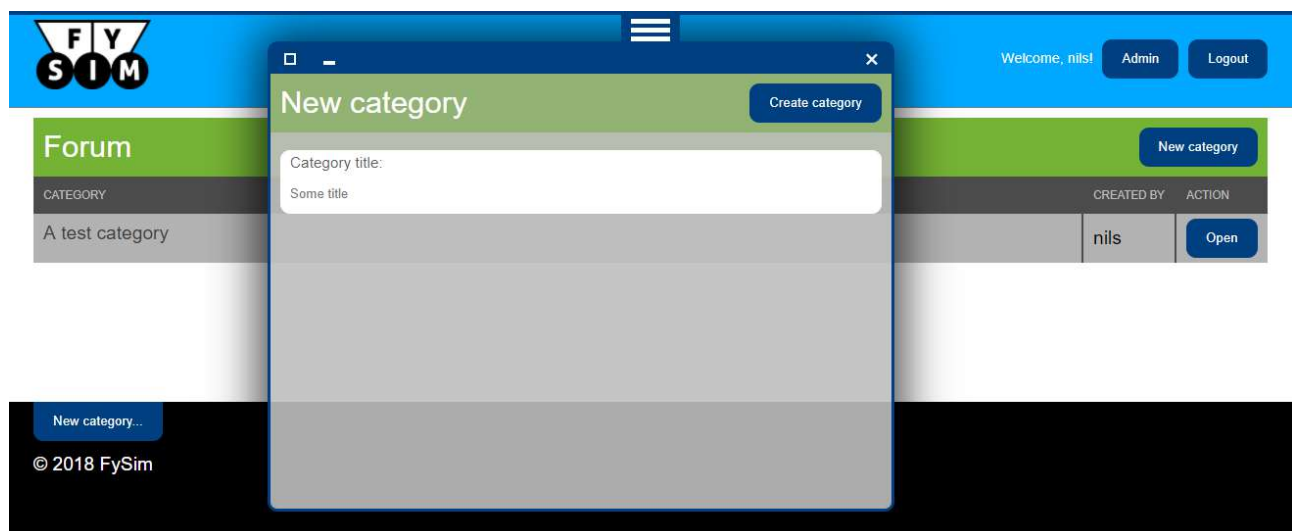


Illustration 13: Screenshot displaying modal window with docking

The modal window is made with a OS window functionality in mind. It has a docking element in the FOOTER section of the site, all modal windows get a button here and it is possible to minimize to the docking and reopen windows, also a maximize button on the modal expands it to cover the main content area.

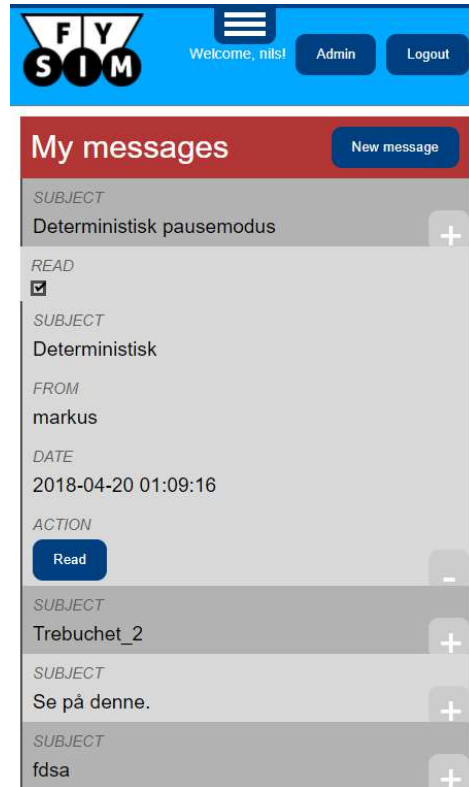


Illustration 14: Screenshot displaying compressed table view

The listing/table element is made with responsive design in mind and has a collapsed view making viewing a table listing with many columns easier and quicker. With simple classes one can decide what columns to collapse in the minimized view. This is done 100% with CSS. (the app is not currently styled and developed for mobile view in general, this will come in future developments).

4 Back-end

Unless otherwise stated all reference to programming in this chapter is in Python.

4.1 Architecture

Using Flask and Blueprints we went for a modular architecture dividing the app functionality into modules following the same structure as in the front-end.

Each module is divided into three "stages", first there is a routing stage with routes set up for the different requests to the server, this is divided into GET requests and POST requests. These requests are then sent to the next stage determined by an action URL parameter deciding which method call is required. This method call then handles data verification and security and if needed does calls to the last stage, the database stage where all the SQL and database interaction is done.

```

app/ config.py (1)
    run.py
    modules/ __init__.py (2)
              common.py (3)
              sendmail.py (4)
              _static/   index.html
                      js/
                      css/
                      images/
    *module*/ __init__.py (5)
              io.py (6)
              db.py (7)
              _static/  *assets*

```

- 1) Variables for Flask configurations
- 2) Initialization of Flask app, registering of blueprints, global routes returning index.html and app teardown context closing MySQL db connection
- 3) Utility functions and classes used throughout the app
- 4) A MailSender class for sending mail courtesy of <https://github.com/dimaba/sendmail>
- 5) Module/Blueprint initialization and module route handling
- 6) Handling of module action requests pushing data in and out of the app
- 7) Module database queries

Illustration 15: Displaying structure of back-end modular architecture

This way of separating the code makes it more maintainable and easy to work with.

5 Database

Unless otherwise stated all reference to programming in this chapter is in MySQL.

5.1 Architecture

The database was developed in two phases, first phase was making a rough scaffold of tables that would satisfy what we imagined would be necessary and then in second phase adapting and adding tables as we developed the different modules functionality in the pipeline mentioned earlier; first making a data need on the front-end, develop the functionality delivering the data in the back-end and last making the tables in the database.

User data like password is stored in the database as a hashed key with a salt, making it more secure in case of a data breach.

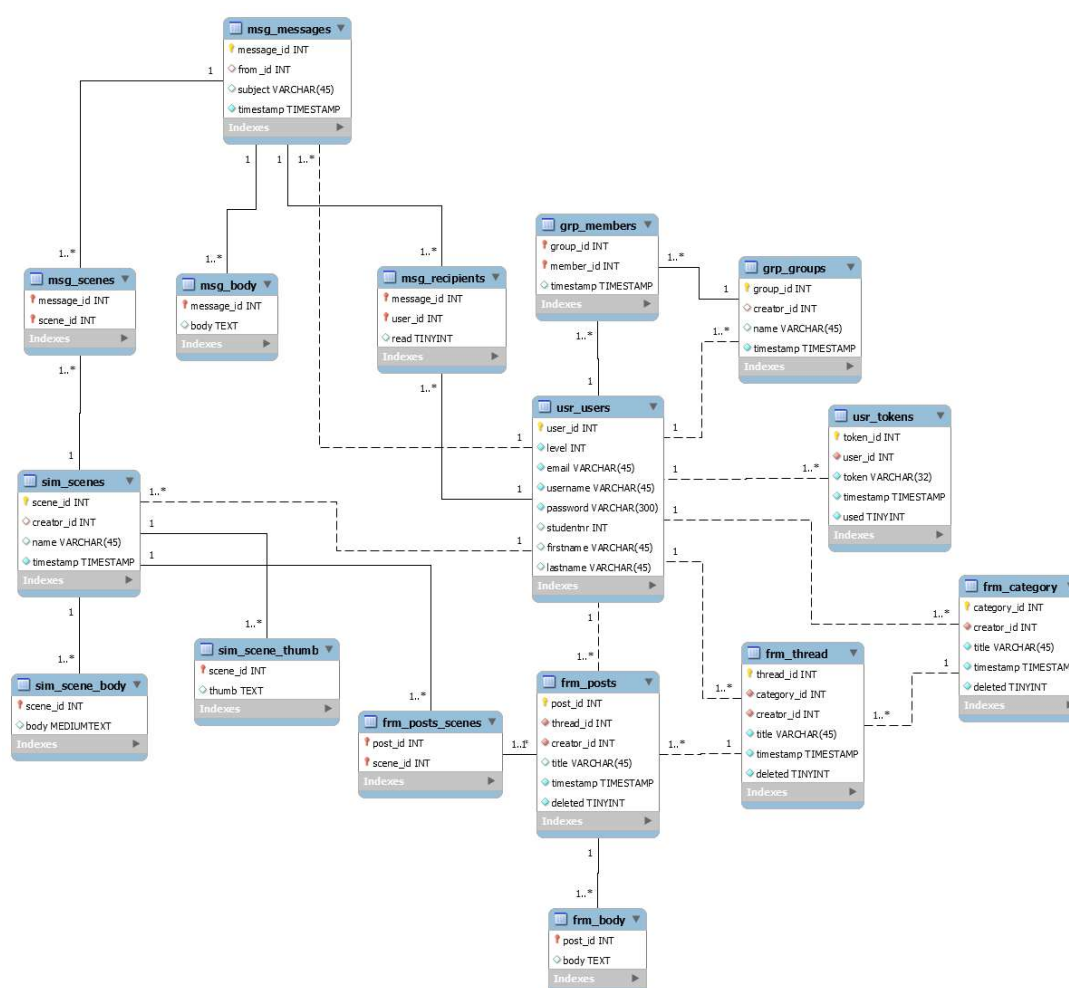


Illustration 16: Database architecture

6 Theory

Focus

A chaotic system cannot be simulated with perfect precision without an infinite access to processing power. The simplifications required will be a compromise between accuracy and performance. To make sure the simplifications does not break laws of physics taught in PHYS100, some axioms will be defined that should not be broken. These were taken into account through the development of the engine.

The first axiom has to do with symmetry

The choice of translational coordinate system must not change the outcome of the physical simulator nor should it change the performance requirement.

If the entire system is moving at 100m/s to the right, and the view point is moving with the same velocity, it should not be possible to, just by observing the physics, notice that the system is moving, compared to if it was still. The symmetry is only limited to translational velocities and positioning since it is unlikely to have objects that rotate very fast. It is also difficult to calculate accurate collisions between fast moving objects since the movement of the points on rigid bodies will be moving in spirals rather than straight lines. Due to floating point precision, this symmetry will only apply as long as the speed is not too large as that would move the rigid bodies very far away from the origin.

The second axiom has to do with conservation of momentum.

All displacement in position of mass, either due to a shift in position or due to a change of momentum over time, must conserve momentum both the translational and angular directions.

In real life, changes in positioning of mass would only occur due to changes in momentum. However, due to inaccuracies in the simulation it will sometimes be necessary to change the positions of rigid bodies without adding momentum. It is then important to change the positioning of the masses in such a way that if momentum of the system was measured as changes in positioning of mass over time, momentum would be conserved. It is easy to make mistakes when it comes to conservation of momentum that could lead to weird behavior and exploits. Just a tiny inaccuracy in conservation of angular momentum would cause a system to quickly spin out of control, as can be seen in various engines like VRAGE used in the game space engineers.

The third axiom is conservation of energy.

Due to errors in the way kinematics are calculated, some mechanical energy might disappear. In many physics engines hinges and other constraints are simulated as tiny springs. This easily cause energy to be added to the system. We use a different approach as explained.

We make sure whenever masses are moved, or momentum is changed, any mechanical energy changed in the system, potential or kinetic, is conserved or removed (turned into thermal energy).

Code architecture

Due to the object oriented nature of the structure of the physics simulation algorithm, the code structure will also follow such a structure. Even though Javascript is based on prototypes, the typical object oriented terminology will be used when referring to code.

6.1 Interface

The interface between the physics simulator and the web part of the application is defined by the physics simulator module. The interface specification can be found in a file called `how_to.js` located at `app/modules/sim/_static/physics_engine/how_to_use.txt`. The interface is a set of functions and listeners. The web application part is the one to initially establish contact.

6.2 Graphics

The graphics related files can be found in `app/modules/sim/_static/graphics/`. We will now go through the process of rendering. We will go through the process from when a new mesh is created, till it ends up as pixels on the screen. The graphics are done using WebGL which is build on OpenGL. How WebGL works or the graphics card's render pipeline will not be explained, but that can be read about here (6). An abstraction layer is made on top on WebGL. It is the abstraction layer that is explained as the underlying functionality is as referred to in (6).

First a HTML canvas element must be chosen for rendering. The `RenderContext` class will handle the communication with the canvas.

```
var renderContext = new RenderContext();
renderContext.bindCanvas(canvas);
```

A camera object will be made to handle the viewpoint position, orientation and zoom.

```

var camera = new Camera();
camera.viewportWidth = renderContext.canvas.width;
camera.viewportHeight = renderContext.canvas.height;
camera.width = 60;
camera.position = new Vec2(0,0);
camera.calculateProjection();
camera.calculateView();

```

The camera is using matrix and vector math. The libraries for this can be found under [app/modules/sim/_static/utilities/matrix_3d.js](#) and [app/modules/sim/_static/utilities/vector.js](#). The functions in the matrix library `Matrix3D.getOrthogonalProjection()` was constructed from this 4 dimensional version (7). Since the engine we are making is 2 dimensional, 3 dimensional instead of 4 dimensional matrices is used for transforming the vertices.

What is a mesh 6.7.3?

The mesh handler is initiated as following:

```

var meshHandler = new MeshHandler();
meshHandler.bindGL(renderContext.gl);

```

Now a new mesh can be created.

```

var geometry = [
  [
    new Vec2(-0.9,-0.9),
    new Vec2(-0.9, 0.9),
    new Vec2( 0.9, 0.9),
    new Vec2( 0.9,-0.9),
  ],
  [
    new Vec2(-1,-1),
    new Vec2( 1,-1),
    new Vec2( 1, 1),
    new Vec2(-1, 1),
  ],
];
var newMesh = meshHandler.createMesh();
newMesh.loadOutlineToTriangles(geometry);

```

"newMesh" is now an object of Mesh. The meshes consists of triangles, but the polygons used in the physics engine is of a different format. The physics engine describe polygons as a 2 dimensional list of vertex positions. The outer list is a list of closed circumferences (the last position connects

back to the first) that make up the polygon. If a circumference is oriented in the counter clockwise direction, it is describing the outer boundary of the polygon. If the circumference is oriented clockwise, it is describing a hole in the polygon. `newMesh.loadOutlineToTriangles(geometry)` will triangulate this description of the polygon and turn it into triangles so that the mesh can render the polygon. "loadOutlineToTriangles" uses `Tool.triangulatePolygon(circumferences)` to do the triangularization. This function takes in the 2 dimensional list of vectors and gives out a list of references to the same vector objects from the argument list. 3 subsequent vectors make up one triangle. The geometry in the code example above is describing a hollow box.

Here is an example of the triangularization of a more complex shape.

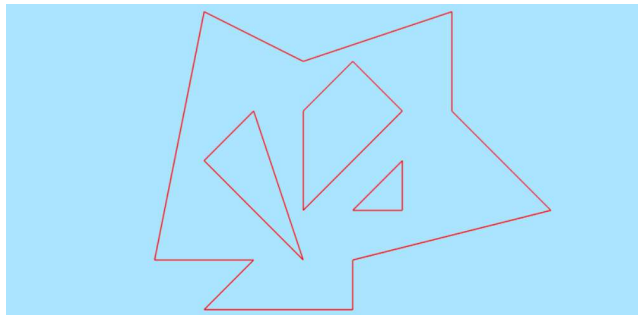


Illustration 17: Complex shape

This shape has 3 holes and therefore 4 circumferences. The vector data looks like this:

```
var geometry = [
  [ // Outer circumference.
    new Vec2(0,-2),
    new Vec2(4,-1),
    new Vec2(2,1),
    new Vec2(2,3),
    new Vec2(-1,2),
    new Vec2(-3,3),
    new Vec2(-4,-2),
    new Vec2(-2,-2),
    new Vec2(-3,-3),
    new Vec2(0,-3),
  ],
  [ // Inner hole 1.
    new Vec2(-1,1),
    new Vec2(0,2),
    new Vec2(1,1),
    new Vec2(-1,-1),
  ],
  [ // Inner hole 2.
    new Vec2(0,-1),
```

```

    new Vec2(1,0),
    new Vec2(1,-1),
  ],
  [ // Inner hole 3.
    new Vec2(-3,0),
    new Vec2(-2,1),
    new Vec2(-1,-2),
  ],
];

```

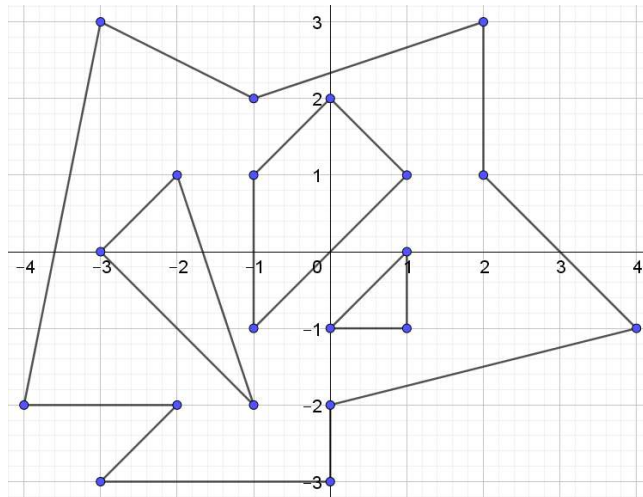


Illustration 18: Coordinate reference

The filled polygon looks like this:

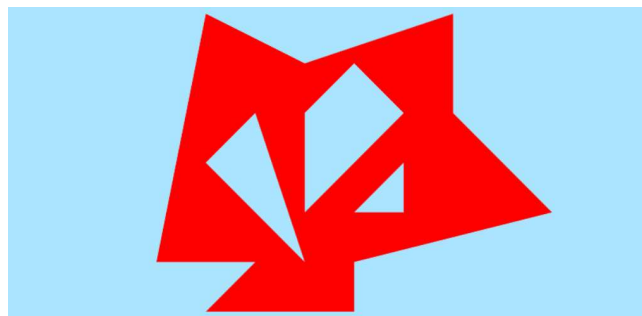


Illustration 19: Filled polygon

This is the triangularized version:

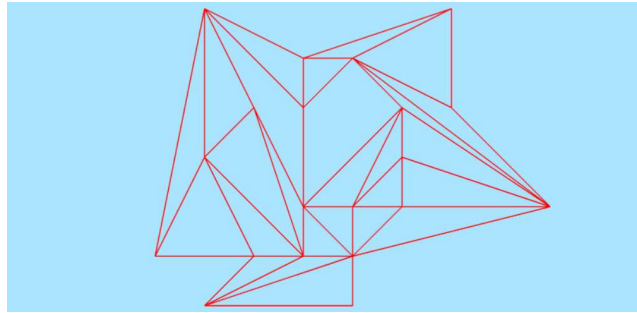


Illustration 20: Triangularized complex shape

6.2.1 Triangulate polygon

We will now look at the inner workings of the `Tool.triangulatePolygon(circumferences)` function.

The idea behind the algorithm is that it will always be possible to draw a straight line segment from one corner of a polygon to another corner of the same polygon without intersecting any circumferences except for the beginning corner and the end corner of the segment. A proof for why this is possible can be found here 6.7.9. If you split a polygon using this segment, you end up with something simpler than what you started with. Simpler means one of two things: if the split happens in between different circumferences, the polygon will have one less hole in it. Here is an example of that.

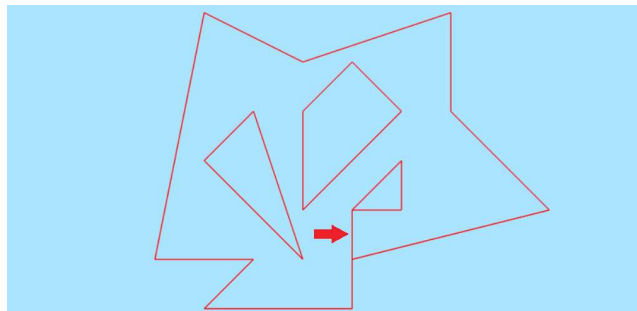


Illustration 21: Example of splitting during triangularization

Alternatively, the split is within the same circumference. Then the shape is split into two less complex areas (fewer corners). Since the two areas will each have fewer corners than the original shape, they are simpler by our definition. If you call the function recursively on the two new shapes, it will be further simplified until all that is left is triangles. This sums up the algorithm "Tool.triangulatePolygon()".

6.2.2 Vertex buffer

Now as all the vertices are in triangle format, the next step is to load the vertex data to a buffer in the graphics card. The triangles are split into 3 positions and each position consists of 2 coordinates. Each coordinate is a 32 bit floating point that is stored inside a buffer called a vertex buffer inside the graphics card. The format is as follows:

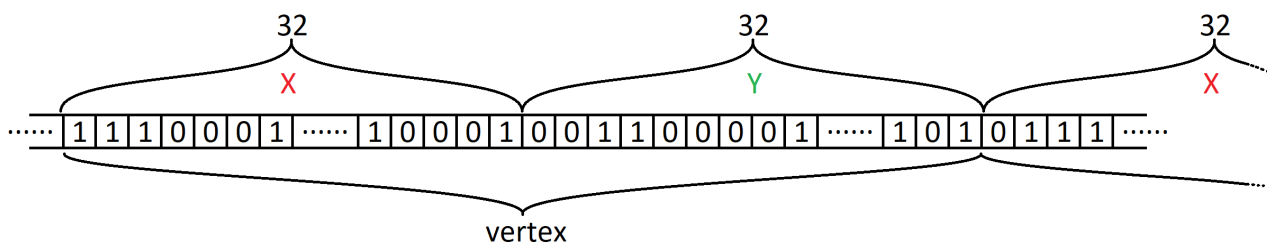


Illustration 22: Vertex buffer

The `vertexBuffer` class is responsible for handling the loading of the position data to the graphics card and is responsible for remembering the format. It can be found at [app/modules/sim/_static/graphics/graphics_engine/vertex_buffer.js](#). To handle how the positions are connected to triangles, an element buffer is needed.

6.2.3 Element buffer

There is also an element buffer of 16 bit integers that bind together 3 vertices to triangles. The `ElementBuffer` class can be found at [app/modules/sim/_static/graphics/graphics_engine/element_buffer.js](#). This class handles loading and binding of the buffer containing the 16 bit indices. Each index points to one vertex position. The format of the data is tightly packed 16 bit unsigned integers, where 3 subsequent indices describe one triangle, and the next 3 indexes describe the next triangle. Index 0 refer to the first vertex in the vertex buffer.

6.2.4 Shader program

The `ShaderProgram` class handles the creation of the shader program that is compiled and sent to the graphics card which is then used to interpret the data that is loaded from the vertex and element buffers. The shader program contain a vertex shader which is used to calculate perspective. The triangle data in the vertex shader is coordinates local to the polygons. When this data is sent through the vertex shader, it will be scaled, translated and rotated into the right positions. The graphics card will then, after rasterizing the triangles, use the fragment shaders to set the color of the triangles. More details about this comes later.

6.2.5 Mesh handler

The Mesh class use the shader program and the vertex and element buffers. Each mesh has a separate vertex and element buffer, but only one shader program is needed. The MeshHandler class contain the shader program and handles all the meshes and their rendering. The mesh handler uses a projection matrix and a view matrix to render the meshes in the right position in the right orientation with respect to the viewpoint of the camera. Each mesh calculates a model matrix based on its position and orientation that is used to translate its vertex coordinates to the world space coordinate system.

After all the data is uploaded, the next step is to render the meshes. First the cameras position and orientation must be set, and its new view and projection matrix must be calculated.

```
camera.position = new Vec2(1,1);
camera.setUp(new Vec2(0,1));
camera.calculateView();
camera.calculateProjection();
meshHandler.setView(this.camera.view);
meshHandler.setProjection(this.camera.projection);
```

Next, the canvas is cleared with a background color...

```
renderContext.clear();
```

... the meshes positions and orientation is set...

```
mesh.position = new Vec2(1,1);
mesh.orientation = 0;
```

... and the meshes are rendered.

```
meshHandler.renderAll();
```

Lets take a look at the inner workings of the "renderAll()" function. The mesh handler will loop through all the meshes and handle them the view and projection matrix for rendering. For each mesh, the shader program and buffers will be bound. "this." refer to the mesh itself.

```
this.shaderProgram.use();
this.elementBuffer.bind();
this.vertexBuffer.bind();
```

After that, the model matrix will be created and sent to the shader program.

```
var modelMatrix = Matrix3D.getScaleXY(this.scale.x, this.scale.y);
modelMatrix = Matrix3D.rotate(modelMatrix, this.orientation);
modelMatrix = Matrix3D.translate(modelMatrix, this.position);
this.shaderProgram.setUniformMat3("model", modelMatrix);
```

Since the coordinates of the meshes that are loaded to the vertex buffer is in local coordinate space, the model matrix is used to transform the coordinates to global space. The local coordinate space is one that each mesh has. The mesh is not transformed inside this space. The global coordinate space is a shared space between the meshes. This is how the meshes look like after their transformation, before rendering. As we see in the code, a scaling matrix will be created first. The next step is rotation, and so translation. The matrix is transformed through matrix multiplication. The model matrix is then sent to the shader program.

A similar process is used to send the rest of the mesh data (like color and depth), the view matrix and the projection matrix to the shader program. The view and projection matrices take 2d vectors from global space and into clip space. The clip space is a coordinate system where x, y and z coordinates are limited between -1 and 1. x and y go from the left to right and bottom to top of the canvas, while z has to do with depth (what meshes are rendered on top of what).

Lastly, a render call causes the color buffer to be updated with the new triangles.

```
this.gl.drawElements(this.gl.TRIANGLES, this.nrOfIndices, this.gl.UNSIGNED_SHORT, 0);
```

The drawing calls are the bottle neck of the rendering part of the simulation. When profiling the simulation we see that the GPU does almost no work compared to its potential processing power.

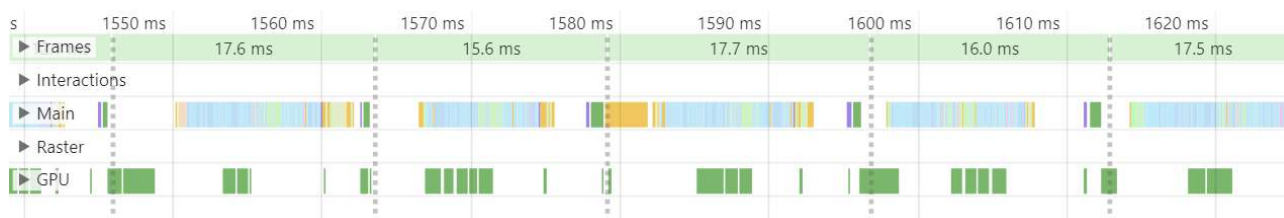


Illustration 23: Performance profiling

The chrome performance profiler was used to measure the CPU and GPU performance and find bottle necks. The blue bars at the "Main" row, is the physics calculation. This is the work of the

CPU. This is the physics calculations and communications through WebGL. The green bars are the work of the GPU. As we can see, the GPU is not the bottle neck, but the CPU is. Lets take a closer look at the work of the CPU.

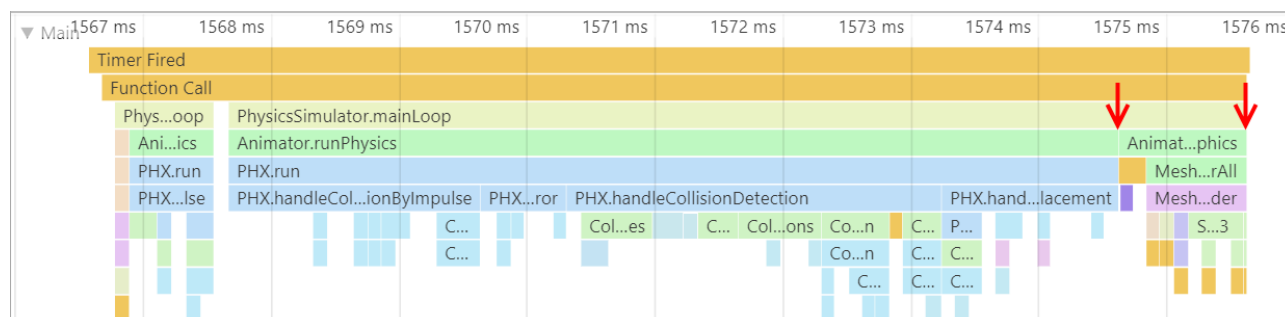


Illustration 24: WebGL CPU profiling

The entire screen shot is one frame. Between the red arrows, we can see the work the CPU does sending data to the graphics card. This includes render calls and sending the matrices describing the positions and orientations of the meshes, and other properties like color and depth to the GPU.

Originally the plan was to optimize this CPU to GPU communication by doing the animation in the shaders run on the GPU. That would mean the model matrix would only need to be updated when an object changed velocity, and then the GPU would calculate the movement based on a time variable received from the CPU. The problem with this approach, is that it would require functionality which is standard in OpenGL, but experimental in WebGL, namely uniform buffer objects. Alternatively the CPU to GPU communication could run in separate threads with the use of web workers, but this is only supported in Firefox at the moment. The attempt to use this type of optimization was eventually abandoned. According to the profiling the CPU GPU communication part of the work of the CPU, this is insignificant due to the low relative performance. This portion would be more significant if the number of meshes were bigger, but physics calculations were smaller. An example of such a situation would be if the gravity was turned off, and multiple objects were floating freely.

6.2.6 GLSL

When the vertices are loaded to the graphics card, and the transformation matrices are calculated and sent and the color and depth data are sent, the shaders will be used to combine this data correctly. The shader program uses the language GLSL (OpenGL Shader Language) (8). Its syntax is similar to that of C. The two shaders customizable using WebGL is the vertex shader and the fragment shader. The vertex data is stored in a local coordinate system so that the center of mass of the described polygon is in the origin. To move these positions into the correct location, the

positions will be transformed one by one in the vertex shader.

```
vec3 worldPos = view * model * vec3(position, 1.);
worldPos.z = depth; // Set depth.
gl_Position = vec4(projection * worldPos, 1.);
```

This code is from

`app/modules/sim/_static/graphics/shaders/mesh_shaders/mesh_vertex_shader.vert`. Here the position, is the local position of the vertex to the center of mass with an orientation of 0. This coordinate will be transformed by being multiplied with the model, view and then the projection matrix. "lg_Position" is a 4 dimensional vector where the 4th variable is used to divide the other components, namely the x, y and z component. This is used in various projections to be able to represent a projection as a matrix. Since this physics engine is only operating in 2 dimensions, and that only require an orthogonal projection, such a projection can be represented as a matrix, and the division component is not used, so it is left to be just 1. The z-component is used for depth calculation, what pixels will be rendered on top of which. -1 will be on top of everything, and 1 will be at the back.

The fragment shader of the mesh is very simple. All it does is pass the color onward.

```
gl_FragColor = vec4(color, 1.);
```

6.2.7 Debug renderer

For debugging purposes the class DebugRender is used. This class is simple to use, but is not well optimized regarding performance.

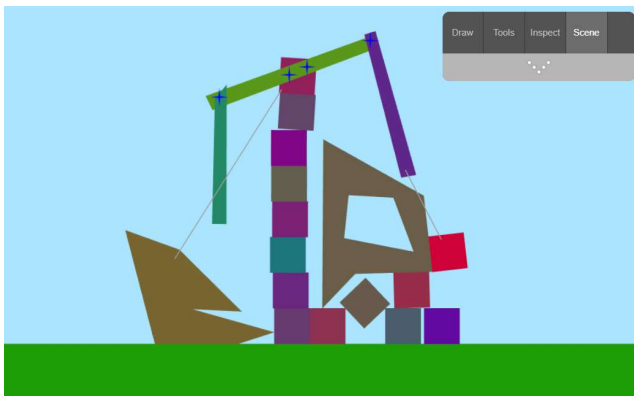


Illustration 25: Standard view

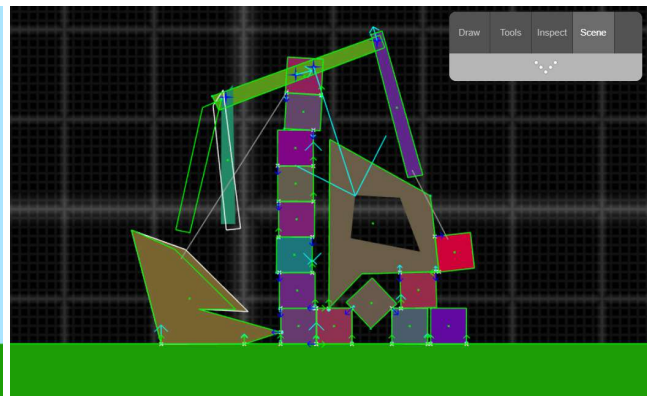


Illustration 26: Debug view

The left picture is a scene without debug mode activated and the right picture depicts when debug mode is activated. While debugging collision detection it is useful to know where in the scene the

coordinates relate to. Just looking at numbers and trying to interpret their positions, or exporting the debugging information to separate software, is time consuming. Bugs are found faster if the information is visualized in the form of colors and shapes. For example turquoise arrows showing force vectors, bright green circumferences showing where polygons are moved to and from, blue crosses showing the center of mass of moving polygons, and a grid in the background showing the global coordinate system.

Making the black and white grid (called ruler in the DebugRender class) in the background is a bit more tricky than placing graphical elements on the screen. When placing a blue cross at some coordinates in global coordinates, the local 2d coordinates of the cross will first be multiplied with a model matrix, then by a view matrix and then by a projection matrix. This takes the coordinates from from local space to clip-space coordinates (9). When rendering the grid, an opposite transformation is done. The clip space coordinates are taken to the global space system.

```
var inverseView = Matrix3D.inverse(this.view);
var inverseProjection = Matrix3D.inverse(this.projection);

rulerShaderProgram.setUniformMat3("inverseView", inverseView);
rulerShaderProgram.setUniformMat3("inverseProjection", inverseProjection);

gl.drawArrays(this.gl.TRIANGLES, 0, 6);
```

This is the Javascript code showing the calculation and sending of the inverse view matrix and inverse projection matrix and the render call that renders the grid. The grid is rendered on a square mesh that is filling the entire screen. A square consists of two triangles, each consisting of three corners, thereby 6 vertices to render. The code can be found in

[app/modules/sim/_static/graphics/graphics_engine/debug_render.js](#) under the function `DebugRender.renderRuler()`.

```
worldCoordinate = vec2(inverseView * inverseProjection * vec3(position, 1.));
```

This GLSL code is from the

[app/modules/sim/_static/graphics/shaders/debug_render_shaders/ruler_vertex_shader.vert](#).

"position" is the clip space coordinate. The worldCoordinate is sent to the fragment shader.

```
float r = max(
    cos(worldCoordinate.x * 2. * PI / spacing) * sharpness - sharpness + 1.,
    cos(worldCoordinate.y * 2. * PI / spacing) * sharpness - sharpness + 1.
);
```

This GLSL code is from

[app/modules/sim/_static/graphics/shaders/debug_render_shaders/ruler_fragment_shader.frag](#). "r" is the result used by the fragment shader to calculate the brightness of a pixel in the background grid. There is more calculations in this shader that is not shown here, but this shows the most important part. The horizontal and vertical grid lines are created using cosine curves. They are adjusted, using the sharpness variable. The larger the variable, the taller the cosine curves will be, but they will also be moved downwards, so only the top is visible. This results in a wave pattern, where most of the pixels end up being black, while some is bright white, which is where the rulers are. The spacing variable is used to space the grid. It might be that we want to see a new ruler every 0.1 distance unit, or maybe every 100 distance units and so on. This creates the grid pattern as seen in the screenshot shown above.

6.3 Kinematics

The kinematics calculations can be found under [app/modules/sim/_static/physics_engine/phx.js](#).

The simulation is of polygons consisting of concave and convex corners. It can also be holes inside the polygons. The polygons move around in the simulated world as rigid bodies. They have a center of mass that they rotate around as the center of mass moves under influence of gravity. The kinematics part of the algorithm is about how these polygons are described, and how they move.

6.3.1 What is a polygon?

When referring to polygons talking about the physics simulation, we refer to objects of the Polygon class. This class can be found at [app/modules/sim/_static/physics_engine/polygon/polygon.js](#). A polygon has various physical properties like mass, a position, velocity, and much more. The geometry of the polygon follows a mathematical definition that can be read here [6.7.2](#).

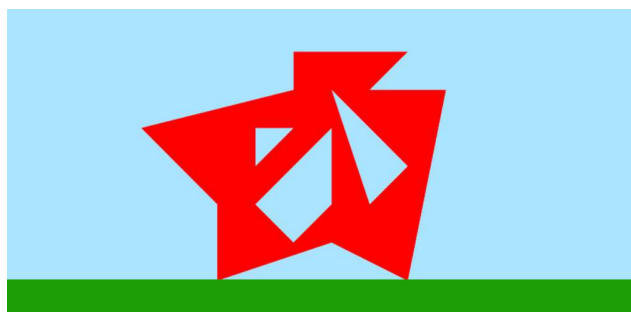


Illustration 27: Filled polygon with holes

The red area is the inside of the polygon, since those pixels are on the inside of the outer boundary, and on the outside of the circumferences describing the holes.

All the vertices of all the circumferences of the polygon is stored as local coordinates relative to the polygons center of mass with an orientation of 0. The center of mass is also the position of the polygon and is described as a 2d vector. The orientation is a number describing the offset orientation from a defined identity orientation in radians. The orientation does not reset every $2 * \pi$ rounds, but just keep increasing, or decreasing as the polygon rotates. This means you might find a polygon with an orientation of any positive, negative or zero orientation. The coordinate system that the position of the polygon is defined within, is the "world coordinate system" or "global coordinate system".

6.3.2 The animation

The word animation is used here as "making something move". This movement does not necessarily have to be visualized.

Even though the physics does not have anything directly to do with the visualization of the scene, the physics simulation must still follow a general architecture that can be visualized in the end. The movement of the polygons takes use of the same illusion used in animated movies we see on TV. The pipeline is as follows.

1. Move the polygons a tiny bit according to their velocities and positions.
2. Render the polygons at their current position and orientation.
3. Wait a tiny moment.
4. Repeat.

This gives the illusion of motion. Usually on TV, the rate of the pipeline is 30 hertz (usually called frames per second (fps)). This frame-rate is usually good enough for TV since the object in focus followed by the camera, and therefore does not move too fast over the screen. Since this is a simulation, we don't know where the user is looking at the screen, so a higher frame-rate is required. Since 60 hertz is the standard refresh rate of a computer screen, this is chosen to be the animation rate.

The visualized animation could have a different rate than the simulated animation. This would allow for a higher frame-rate for monitors with a higher refresh rate and is something that could be implemented in the future. For simplicity, currently, the rendered frame rate is locked at simulation frame rate.

When a real rigid body is moving under influence of gravity, there is several variables that are continuously influencing each other. As time passes, the velocity of the bodies will be changed based on the gravitation acceleration. At the same time as this is happening, the position will be changed by the velocity. Here is an example of such a motion.

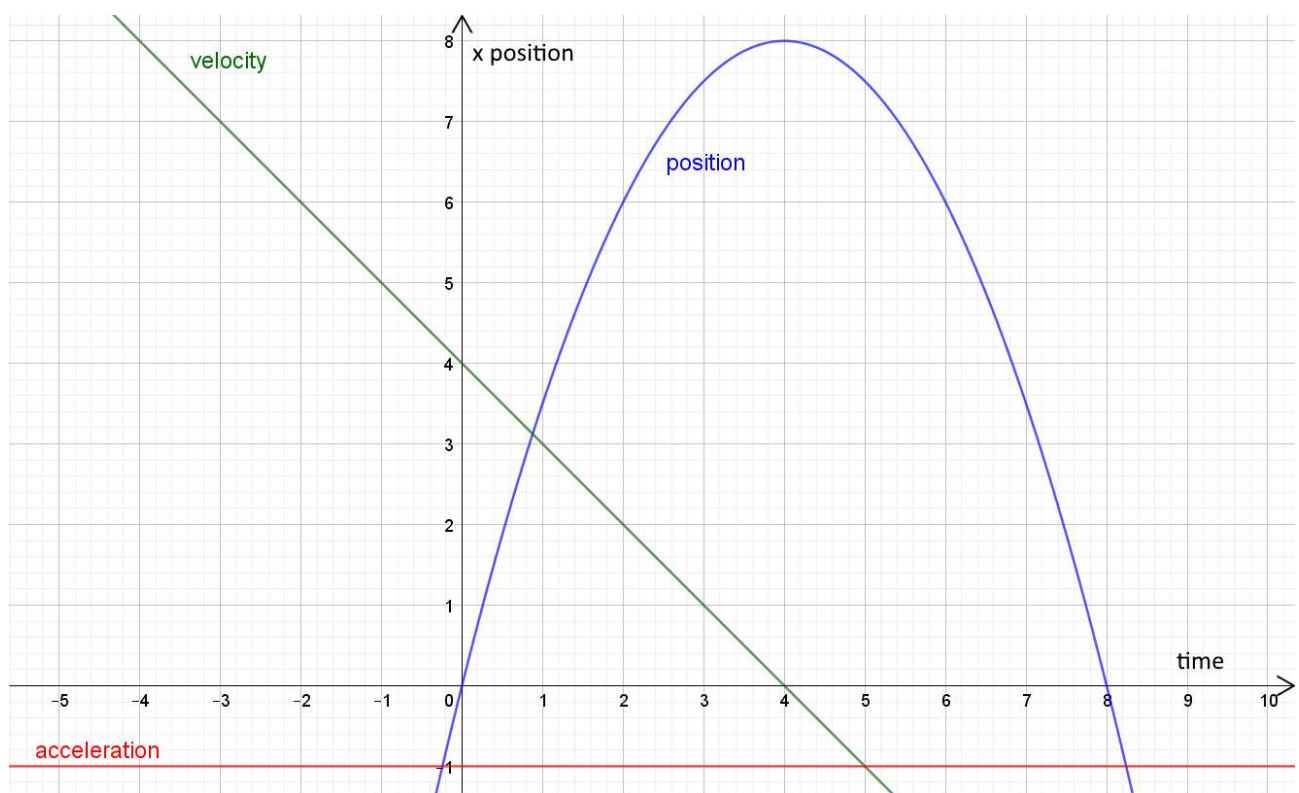


Illustration 28: Motion graphs

Lets say the units are meters and seconds. This describes a polygons movement in the vertical direction. The gravity is in the negative y direction of magnitude 1 m/s^2 . The initial position is 0 and the initial velocity is 4 m/s . The horizontal axis is time.

In the physics simulation, instead of animating the objects analytically, numerical integration will be used instead. Here is how that movement looks like.

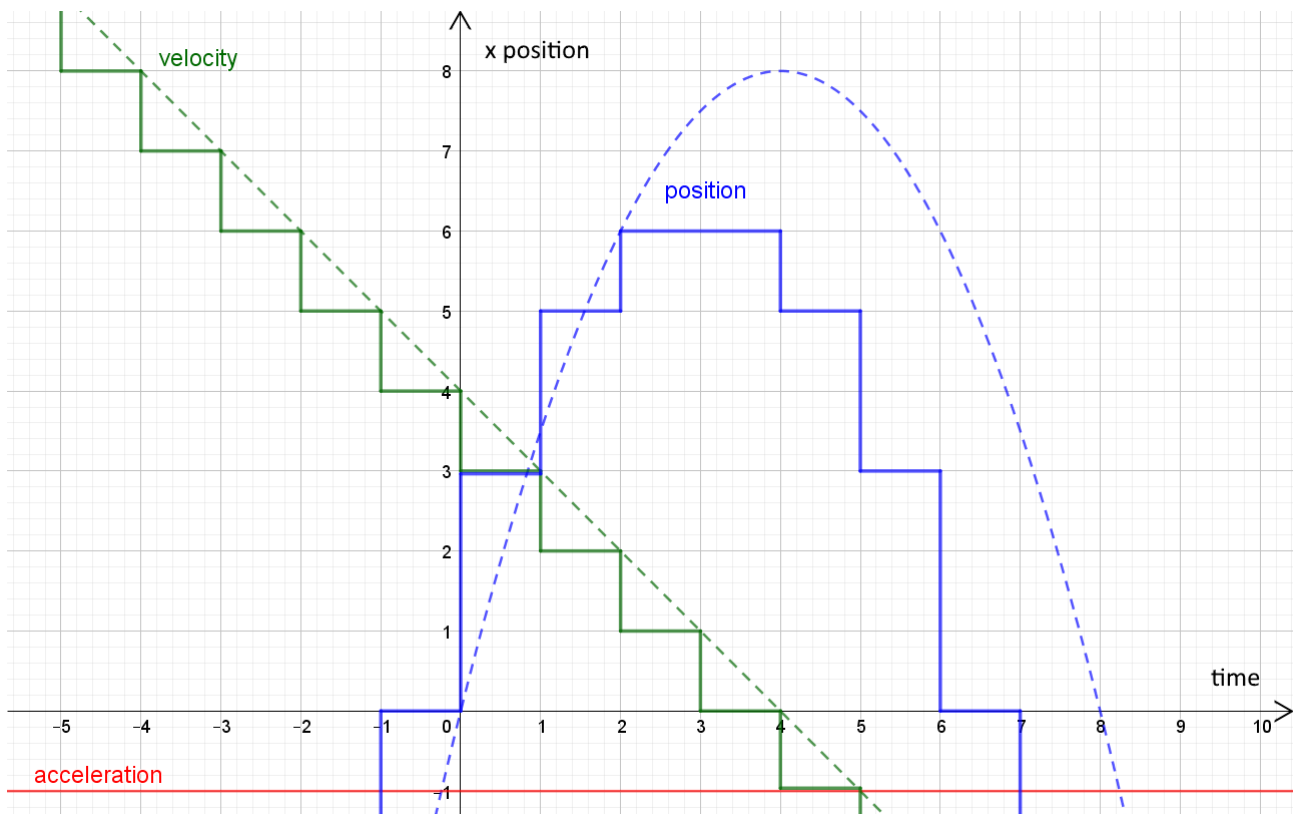


Illustration 29: Motion graph, numerical integration

Here all acceleration for the first second is added to velocity. Then the new velocity will be added to the position. Since the velocity starts as 4 and the acceleration is -1, this leads to a new velocity of 3. The position starts as 0 and is then added to the velocity. The new position is then 3. After this, there is a small delay of 1 second, and then the process is repeated. As we can see, the further the simulation runs, the less accurate it is. If we measure the position just after the delay, we see that the position is 0.5 meter further down than where it should have been for each second. This causes some problems regarding conservation of energy. If the velocity is correct after each delay, but the position is 0.5 meters further down than where it should have been, kinetic energy will be calculated correctly, but we lose potential energy. If we swap the order of changing velocity and changing position, we will get the opposite situation, we gain a bit of energy for each iteration. To make the energy gain or loss less, we could decrease the time step. The smaller the time step, the closer the numerical simulation will be to the analytical approach.

$$\frac{x(t) - x(t - \Delta t)}{\Delta t} = v(t)$$

$$x(t) = x(t - \Delta t) + v(t) * \Delta t$$

$$\frac{v(t) - v(t - \Delta t)}{\Delta t} = a(t)$$

$$v(t) = v(t - \Delta t) + a(t) * \Delta t$$

Illustration 30: Numerical simulation formula

$x(t)$ is the position at time t . The first and third equation is the definition of the derivative. The second and third equation is the ones used in the simulation. Here is how the movement looks like in code.

```
for (var i = 0; i < this.polygons.length; i++) {
  var p = this.polygons[i];
  p.position.add(p.velocity);
  p.orientation += p.angularVelocity;
}
```

This code is from [app/modules/sim/_static/physics_engine/phx.js](#), the function is "handleMovement()". Here the positions of each polygon increased by their velocity vector and their orientation by their angular velocity. The time step used in the simulation is 1/60 seconds. As explained above, the order of which acceleration and motion is applied, determine whether energy is constantly being added to the system or removed. For most physics engines used for example in video games, the energy loss version would be chosen and the loss of energy would be blamed on friction. Since this simulation is made for learning purposes, it is important that it tackles idealized situations, like for example if the polygons are completely friction-less and all collisions are perfectly elastic.

If we look at the kinematic formulas (10), we see that under influence of a constant acceleration, the average velocity can be used to achieve the correct displacement in position.

$$\Delta x = \left(\frac{v + v_0}{2} \right) t$$

Illustration 31: Displacement

Since the movement is not accurate after each simulation cycle is accurate, energy is also conserved

correctly. The new motion looks like this.

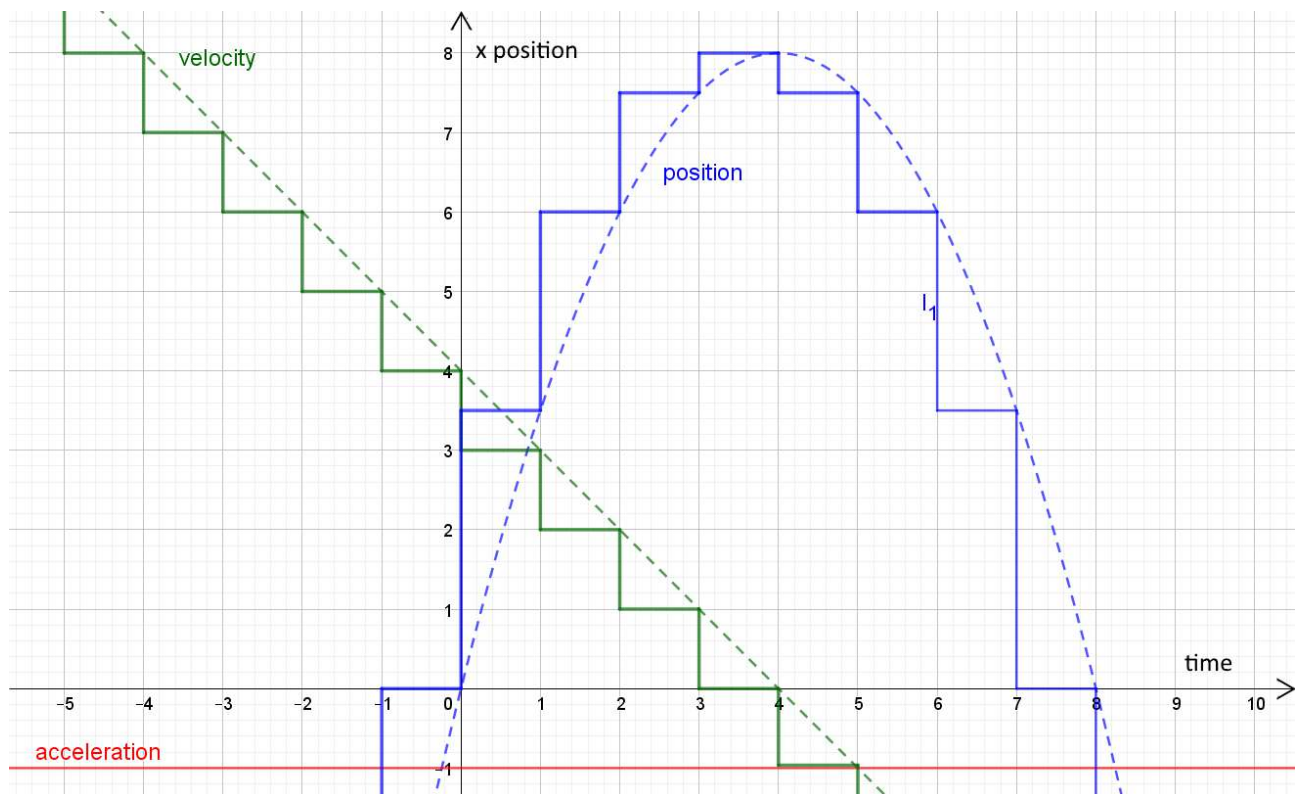


Illustration 32: Displacement graph

We see that at the rightmost corners (of the blue line (the motion of the particle)), the motion perfectly follows the analytical motion. This means that as long as the friction less polygons are moving within a linear gravity field, mechanical energy will be conserved.

The way average velocity, with respect to gravity, is implemented is shown below.

```
// Accelerate half.
this.handleForces();
// ...
this.handleMovement();
// ...
// Accelerate half.
this.handleForces();
```

This code can be found in `physics_engine/phx.js`, the function is `"run()"`.

6.4 Collision detection

The code for collision detection can be found at [app/modules/sim/_static/physics_engine/phx.js](#) "handleCollisionDetection()".

6.4.1 Why collision detection?

When real physical bodies get close to each other, electromagnetic forces will push them apart. This will occur just when the surfaces touch each other. We want to detect when the surfaces touch, so the appropriate impulses can be applied between the bodies. Since this simulation discretizes the movement of the bodies, the moment the surfaces hit, might be missed. Due to this problem, an approximation is made to detect if the polygons collide at some point between the two discrete moments in time.

The way collision detection is done in this simulation is quite different from how it is calculated in most engines. Being aware of many of the typical problems one usually runs into making an engine, we took a new approach that attempts to solve these problems right from the beginning. To show why this approach is useful, let us start with how most engines do collision detection.

6.4.2 Overlap based collision detection

When rigid bodies move, you want to detect when the surfaces intersect. Due to the discrete way motion is calculated, two surfaces would most likely first intersect in between two time steps. Overlap based collision detection is based on the idea that if two surfaces touch and the bodies are allowed to move freely for the rest of the time step, the polygons will end up in an state of overlapping. Testing for overlapping is the next step. For this, the separating axis theorem is used. The theorem is explained here [6.7.6](#).

6.4.3 Motion based collision detection

The method used in this engine is often referred to as continuous collision detection. Continuous collision detection will detect if polygons' motion is overlapping, but after the polygons are in an overlapping and potentially non moving state, collision will not be detected. A second method is needed to be able to detect the second and onward collisions. Let us begin with introducing the physics pipeline.

1. Collision resolution

2. Kinematics
3. Collision detection ← we are here
 - A. Update shadow
 - B. Continuous collision detection
 - C. Optimize shadow
4. Render
5. Wait
6. Repeat

The overall structure of the physics engine is a rendering loop as explained earlier. 1-6 are the main steps performing the various tasks happening within one tick. The steps are further split into sub steps marked A-C. We are currently explaining step 3 collision detection.

6.4.4 Continuous collision detection

We will ignore step A for now, and begin with explaining step B. Let us begin with a theorem:

For two polygons' surfaces to intersect, at some point, a corner of one of the polygons will pass through a segment of the other polygon, or the other way around.

The proof can be found under 6.7.4.

According to the theorem, in order to test for collision, it is sufficient to test for intersection between all corners of one polygon versus all segments of the other polygon and the other way around.

To simplify the algorithm testing for collision between the moving segments and the corners, the motion of the polygons between each tick is assumed to be linear.

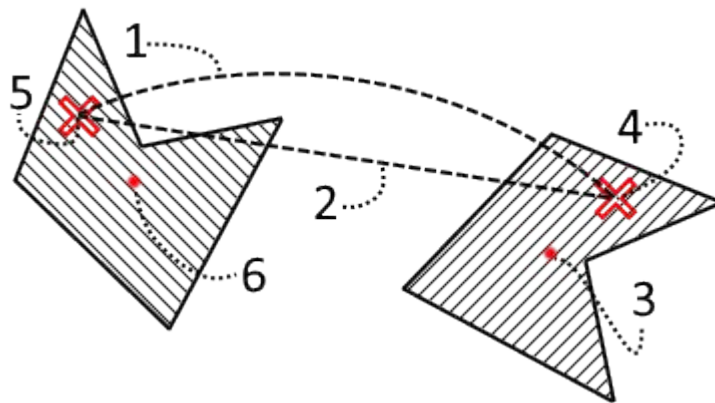


Illustration 33: Rotating polygon colliding

The illustration shows how motion between ticks is simplified. Here we see a polygon moving from one position to another. The center of mass of the polygon is moving from 3 to 6. If we assume that the center of mass is moving linearly, the arbitrary point 4 and 5 on the polygon would follow the curve 1 as it is translating along with the center of mass while rotating. To simplify the math and increase performance by reducing the calculation needed during collision detection, the motion is simplified to curve 2. Here the motion is simply linear.

This simplification will cause inaccuracies when the objects are rotating quickly as it will cause the objects to shrink between ticks.

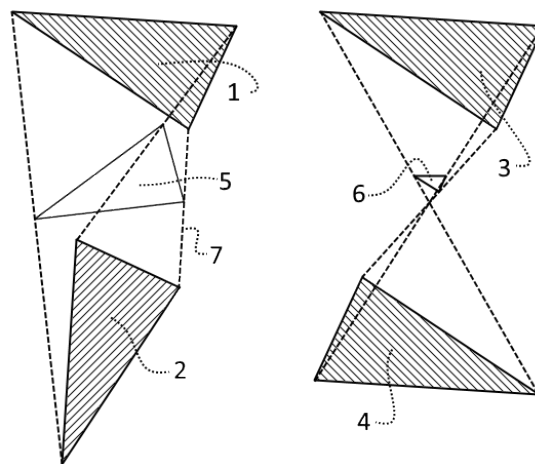


Illustration 34: Projecting collision

This is an example of two triangular polygons moving and rotating. The left triangle is moving from 1 to 2. The angle of rotation is 90 degrees. The striped lines, for instance 7, is the simplified motion of the corners. 5 is the position of the triangle some time between the beginning and the end of the tick. As we can see, the triangle is smaller here. In the example to the right, the triangle is moving

from 3 to 4. The angle of rotation is 180 degrees. Here the triangle is shrinking to a size of 0.6 is the position and the size of the triangle at some point before it reaches 0. The minimum size of rotating polygons will be $\cos(a/2)$ times the original size, where a is the angular change in between each tick. The error due to the simplification is usually acceptable as long as the angle is less than 90 degrees. Since the simulation tick rate is usually 60 hertz, this means an angular velocity of 90 degrees/tick * 60 ticks/second = 5400 degrees/second or 15 rounds per second. As this angular velocity is relatively high, this inaccuracy will probably not cause problems. The simplification does not break axiom 1 (page 32) since it is only limited to translation. Since the inaccuracy only increases with higher angular velocities, the error will hardly be noticeable to the human eye.

6.4.5 Broad collision detection

Since the number of polygons might be large, and the number of corners and segments might be large as well, comparing all corners to all segments would take a lot of performance, even with a cheap comparison test.

Say, for example, 100 squares are being simulated. This would be $100 * 4 = 400$ corners and 400 segments. The number of comparisons would be $400^2 - 4 * 4 * 100 = 158\,400$. (400^2 is the number of comparisons of all corners to segments and $- 4 * 4 * 100$ is the number of comparisons a square's corners does not have to be compared to its own segments). This means 158 400 comparisons would have to be done. This would repeat 60 times a second. As the complexity and or the number of the polygons would increase, the performance would go up by the big O notation $O(n^2)$, where n is the number of corners. To reduce the performance, axis aligned collision boxes (bounding boxes) are used (11).

Each polygon consists of segments. Each segment first updates its bounding box.

```
vertex.right = Math.max(vertex.worldPosition.x, nextVertex.worldPosition.x);
vertex.left = Math.min(vertex.worldPosition.x, nextVertex.worldPosition.x);
vertex.top = Math.max(vertex.worldPosition.y, nextVertex.worldPosition.y);
vertex.bottom = Math.min(vertex.worldPosition.y, nextVertex.worldPosition.y);
```

The full function can be found under [app/modules/sim/_static/physics_engine/polygon/polygon.js](#) Polygon.calculateCollisionBoxes(). Each vertex object store the collision box information of the segment starting at its own position, and ending at the next vertex position. After this, the polygons axis aligned collision boxes will be created around the collision boxes of the segment collision boxes.

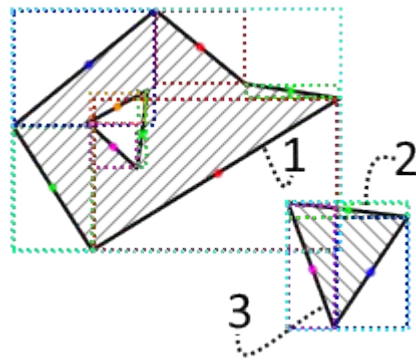


Illustration 35: Axis aligned collision boxes

This figure shows the collision boxes of the segments and the collision boxes of two polygons. The boxes are marked in different colors to make it easier to see the difference between them. The segments are also marked with a colored dot to make it easier to see what collision box it belongs to. The collision boxes of the polygons are turquoise. Since the polygons' collision boxes (turquoise) overlap, the segments' collision boxes will be compared. The red collision box of segment 1, is overlapping with the green collision box of 2 and the pink collision box of 3. When two polygon collision boxes overlap a `collisionGroup` object will be created. This class of this object can be found under `physics_engine/collision_group.js`. This class holds information about what objects collided and other information related to how they collided. Some of this information is structured as a list of `contactPoints`. The class of these objects can be found under `app/modules/sim/_static/physics_engine/contact_point.js`. This object is created when the collision boxes of segments are overlapping. In the example of illustration 35, there will be created 1 `collisionGroup` containing two `contactPoints`. Each `contactPoint` is responsible for the narrow collision detection, more on narrow collision detection is explained under the next heading.

The way collision box overlap detection is usually done in most engines do not take into account the tunneling problem 6.7.7. The box comparison technique used, is the separating axis theorem 6.7.6. In order to take into account polygons and segments moving so fast that they might pass by each other during one tick, the collision detection must be done in a slightly more complicated way.

It is not sufficient to check if a separating axis between the axis aligned boxes can be found, as this could be the case both before and after two boxes pass by each other completely.

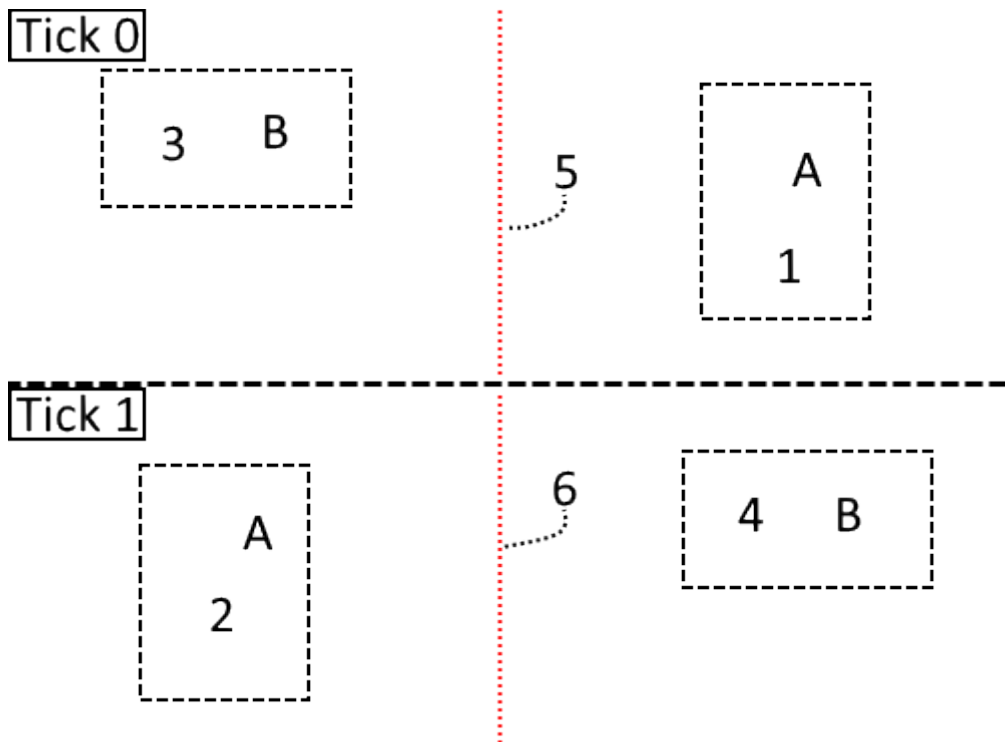


Illustration 36: Separating axis

The collision box A moves from 1 to 2 and B moves from 3 to 4 in just one tick. A separating axis can be found both at tick 0 and tick 1. Since we assume linear motion of the polygons, we assume linear motion of the collision boxes as well. In this case, the boxes clearly collide at some point between the ticks, but the overlap based collision detection misses it. One solution to this problem could be to expand the collision boxes by the movement of the boxes, but this would violate the first axiom as defined at page 32. If multiple collision boxes move together in parallel at a high constant speed, they might all overlap even though they are far apart. If the speed of the boxes is higher than the distance between them (in the direction of their velocity), an overlap would be detected.

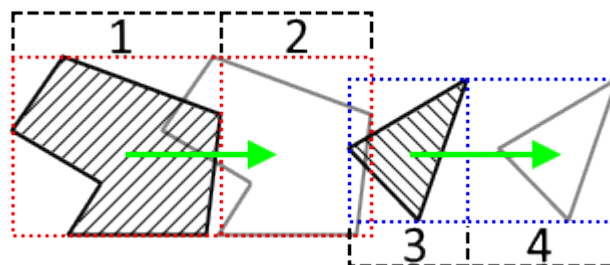


Illustration 37: Overlap detection

1 and 3 are the original sizes of the collision boxes. The green arrows are the velocities of the polygons. The velocities are the same. If we changed the coordinate system so that the velocities

would be zero, the collision boxes would not overlap, but since the collision boxes are expanded by 2 and 4, they overlap even though they don't change their relative positions over time.

The approach used in our engine is as follows. In illustration 36 the separating axis 5 and 6 are found. Axis 5 is to the right of the right edge of B and axis 6 is to the left of the left edge of A. An axis like 5 can only be found at tick 0, but not at tick 1. The same goes for axis 6, it can only be found on tick 1 and not on tick 0. No axis that is to the right of A and at the same time is to the left of B exists at tick 0. In order to be sure there is no collision between the boxes, a separating axis must be found separating the same edges over two frames in a row. The code is as follows.

```
if(
  // If both boxes were separated horizontally now and the previous tick.
  pA.right < pB.left && pA.previousRight < pB.previousLeft ||
  pA.left > pB.right && pA.previousLeft > pB.previousRight ||
  // If both boxes were separated vertically now and the previous tick.
  pA.top < pB.bottom && pA.previousTop < pB.previousBottom ||
  pA.bottom > pB.top && pA.previousBottom > pB.previousTop
){
  return false; // No collision happened.
}else{
  return true;
}
```

This code can be found under [app/modules/sim/_static/physics_engine/polygon/polygon.js](#)

Polygon.collisionBoxesAreColliding(). A similar function for segment collision box detection can be found under [app/modules/sim/_static/physics_engine/polygon/vertex.js](#)

Vertex.collisionBoxesAreColliding().

“pA” and “pB” is polygon A and polygon B collision boxes that are being compared.

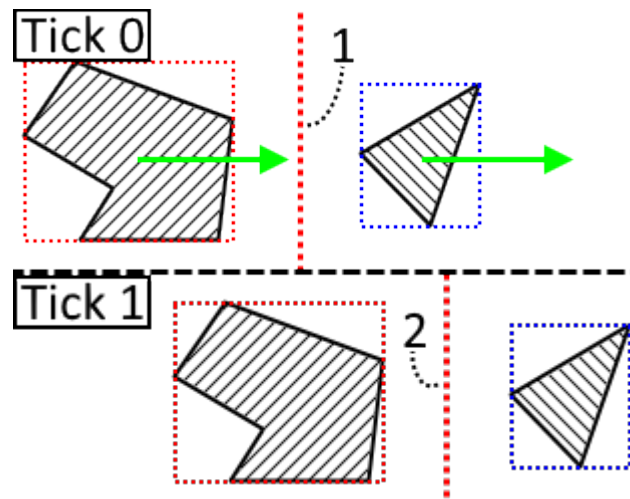


Illustration 38: Collision box comparison

The separating axis, axis 1 and axis 2, are separating the polygons the same way. Both are to the right of the right side of the left polygon, and to the left of the left side of the right polygon. Since there is a separation twice in a row, no collision is detected. This method is independent of the speed of the objects, so performance will not increase when the objects move faster. This way, axiom 1 (page 32) is not broken.

6.4.6 Narrow collision detection phase

Now as we know which segments are colliding with which other segments, we need to do the corner-segment collision detection. Each `ContactPoint` contains two segments that have overlapping collision boxes (or have overlapping motion of the boxes). A segment is part of a circumference. A circumference has an orientation (see definition of polygon 6.7.2). We define that a segment has an orientation in the same direction as the circumference it is part of. We let the vertex at the beginning of the segment define the segment. It is sufficient to do two corner-segment collision detection, one between the beginning corner of segment A, and segment B, and one detection between the beginning corner of segment B and segment A. The information on whether there is an intersection or not, is stored as Boolean values in the attribute `cornerAIsColliding` or `cornerBIsColliding` of the `ContactPoint` object.

```
this.collisionTimeA = ContactPoint.cornerSegmentAreColliding(
    this.vertexA.previousWorldPosition, this.vertexA.worldPosition,
    this.vertexB.previousWorldPosition, this.vertexB.worldPosition,
    this.nextVertexB.previousWorldPosition, this.nextVertexB.worldPosition,
    this.collisionGroup.collisionMargin,
);
this.cornerAIsColliding = this.collisionTimeA != -1;
```

This code can be found under `app/modules/sim/_static/physics_engine/contact_point.js`
`handleCollisionDetection()`.

Note: the if-structure in this function separates the collision detection into two scenarios. One where there is an initial collision, and one where there has been a collision before. We are currently explaining the initial collision. The latter is explained later.

CollisionTimeA is calculated in the function `cornerSegmentAreColliding()` to be the time between the ticks that the corner collides with the segment. The time will be between 0 and 1 if there is a collision and -1 if there is no collision. The signature of the function is as follows.

```

ContactPoint.cornerSegmentAreColliding = function (pointPrevious, pointNow,
segStartPrevious, segStartNow, segEndPrevious, segEndNow, margin = 0)

```

Due to floating point rounding errors, a margin can be used to increase the length of the segment that is compared with a corner. The corner is in this function referred to as a point.

See 6.7.5 for the mathematics behind the function.

6.4.7 Update shadow

Continuous collision detection in itself is not sufficient to detect collisions lasting for more than one tick. Consider the scenario:

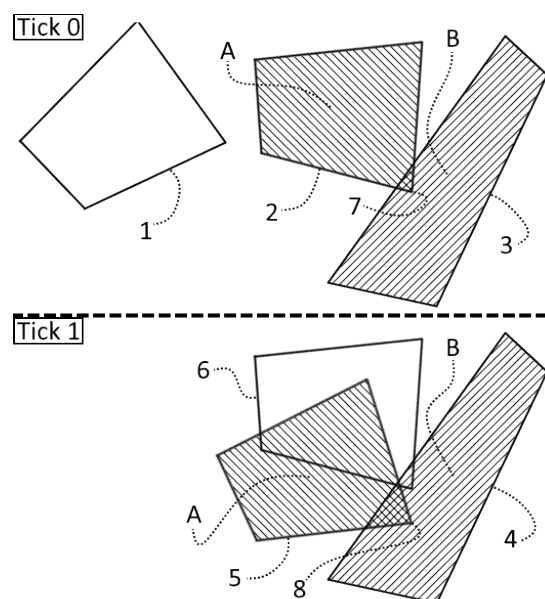


Illustration 39: Continuous collision problem

Let us introduce a new term “shadow”. When a polygon collides with a new polygon for the first time, a shadow will be generated on the previous position of the polygon. This information is stored in the CollisionGroup object, that is, the data structure used to store collision data of a pair of colliding polygons.

```
// Shadow related.  
this.shadowOrientationA = this.polygonA.previousOrientation; // What is the  
orientation of the shadow version of A.  
this.shadowOrientationB = this.polygonB.previousOrientation;  
  
this.shadowDisplacement = Vec2.sub(this.polygonB.previousPosition,  
this.polygonA.previousPosition); // What is the translational displacement between  
A and B. The displacement describes B's position relative to A's position.
```

This code can be found at `app/modules/sim/_static/physics_engine/collision_group.js` in the constructor. The code is run when the collision group is first created, that is, when there is an initial collision. In illustration 39, the initial collision is at tick 0. As can be seen in the illustration, when the polygons A and B are colliding, the shadow of polygon A is stored at position 1. Note that since the position is stored as a relative position, saying that the shadow is of polygon A, or polygon B, is indifferent. To make the explanations and illustrations simpler, the shadow will be thought of as polygons A's shadow.

At tick 1, polygon A moves to a new position, 5. In this example, polygon B does not move. In tick 0, a collision detection is detected, because the corner at 7 moved past B's segment. The corner moved from its shadow position, that is also its previous position, to its current position. If we at tick 1, update the shadow position to be A's previous position, there will be no collision detected, as corner 8 (which is corner 7 from previous frame) will move from inside B, to still be inside B (from position 7 to 8). This is why we won't update the shadow position, unless there is an initial collision. We could end up with polygons falling through each other.

Note: As collision resolution has not been explained yet, how polygon A changes direction from tick 0 to tick 1, is not explained yet. The reason polygon A is allowed to remain inside B is due to errors in the numerical approach to the kinematics of the engine, and that the collision resolution stage is not perfect. More on resolving collision will come later in chapter 6.5.

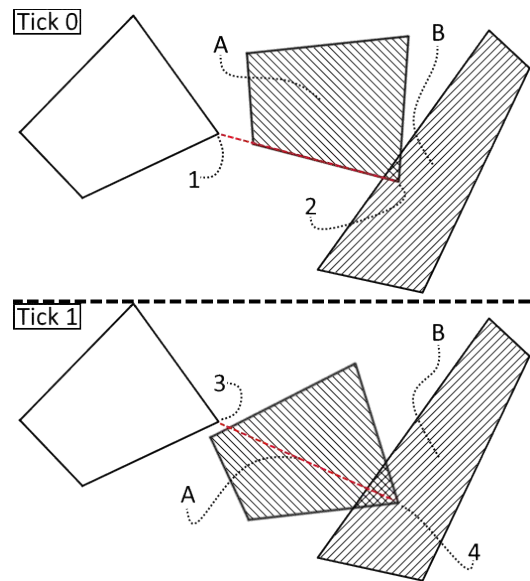


Illustration 40: Shadow update

In the illustration, we now see that the shadow on tick 1, is not updated, since the collision was not initial. The initial collision happened as the corner on polygon A moved from 1 to 2. The collision detection at tick 1 is in between 3 and 4. This last collision (3 to 4) was missed in illustration 39 when the shadow was updated from 1 to 6.

6.4.8 Shadow optimization

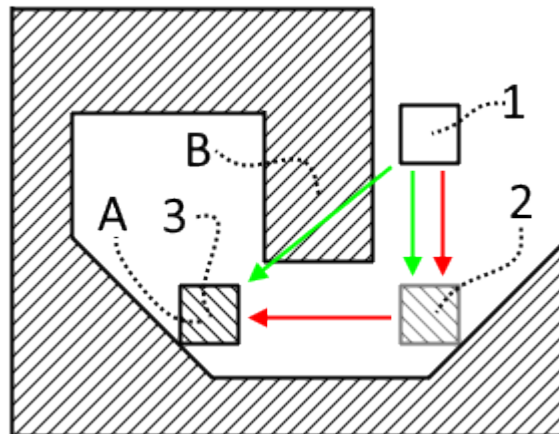


Illustration 41: Shadow problem

The illustration illustrates a problem that could occur when the shadow position is only updated when the collision is initial. If there is multiple collisions in a row, the shadow position and the actual position of the polygons will differ so much, that the collision detection will become inaccurate. In illustration 41 we see two polygons, A and B. B is not moving, but A starts at 1 with a velocity downwards. At tick 1, A is moving downwards, following the red arrow, to position 2. Since there has not been any collisions between A and B the previous tick, the shadow will be set to A's previous position 1. Collision detection is calculated from 1 to 2 (the vertical green arrow). A collision is detected.

A collides with B at the lower right corner and bounces to the left. At tick 2, polygon A moves from 2 to 3, following the red arrow. Realistically, A would now collide with B at its lower left corner and bounce upwards, but this is not what happens. Since the collision detection at tick 2 is not initial, the shadow of A is not updated and is still stuck at position 1. This means that this collision detection will follow the green arrow (position 1 to 3). Since the green arrow moves through the wall, the collision detection algorithm will interpret that A collides with the outer wall, instead of moving through the entrance from position 2 to 3. This problem creates a need to optimize the shadow position to be closer to the actual position of the polygon.

The optimization consists of two steps. The first step is to create a potential shadow. The goal of a potential shadow is to be at a position that is closer to the accurate position of the polygon than the shadow. The next step is to check if the potential shadows of the polygons collide. If no collision is detected, the potential shadow will become the new shadow. This process is illustrated below.



Illustration 42: Shadow optimization tick 0

Tick 0.

Here the rectangle to the left is moving to the right, it will soon collide with the square coming from the right. The blue outlines are the shadows of the polygons. The striped shapes are the current positions of the polygons. The snapshot is during collision detection, so movement has just been applied. Let us revisit the physics pipeline.

Physics pipeline.

1. Collision resolution
2. Kinematics
3. Collision detection
 - A. Update shadow
 - B. Continuous collision detection
 - C. Optimize shadow ← We are here.
4. Render
5. Wait
6. Repeat from 1.

We have already explained step A and step B. We will now explain how step A, step B, and step C works together.

We begin with step 3.1.

- A. There was no collision the previous tick, so the shadow is set to be the polygons previous positions.
- B. Collision detection is performed from the shadows to the current positions. No collision is detected.
- C. Since there was no collision, optimization was skipped.

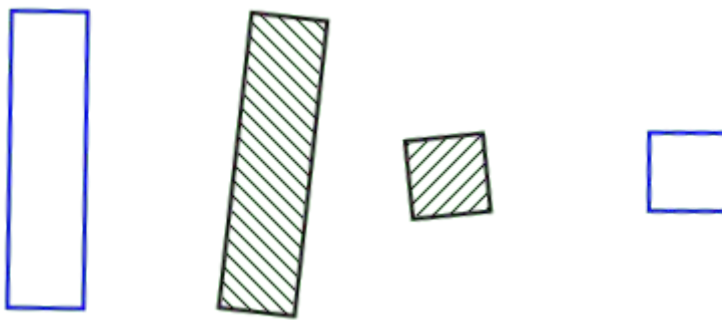


Illustration 43: Shadow optimization tick 1

Tick 1:

- A. There was no collision the previous tick, so the shadow is set to be the polygons previous positions.
- B. Collision detection is performed from the shadows to the current positions. No collision is detected.
- C. Since there was no collision, optimization was skipped.

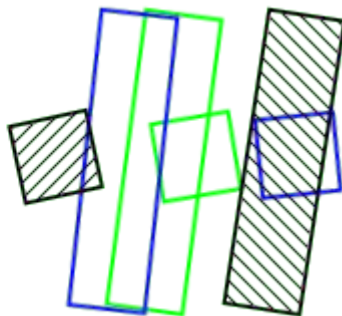


Illustration 44: Shadow optimization tick 2

Tick 2:

- A. There was no collision the previous tick, so the shadow is set to be the polygons previous positions. (blue)
- B. Collision detection is performed from the shadows to the current positions. Collision is detected. The rectangle and square collide. The collision information is stored for later use of the collision resolution step that comes at step 1 in the physics pipeline.
- C. Since there was collision, shadow optimization is attempted. The potential shadow position (green) is generated at a random position and orientation between the shadow at the current position. Collision detection is performed from the shadows (blue) to the potential shadow positions (green). Since there is a collision (the green shadows are overlapping), the optimization failed and the shadow is not updated.

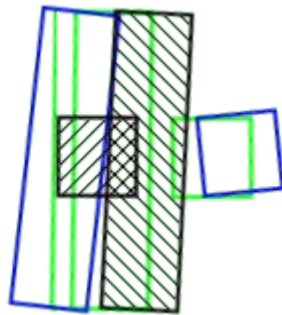


Illustration 45: Shadow optimization tick 3

Tick 3:

- A. There was collision the previous tick, so the shadow is not updated to the previous positions of the polygons.
- B. Collision detection is performed from the shadows to the current positions. Collision is detected. Since the shadow of the square is to the right of the shadow of the rectangle, the collision information sent to the collision resolution step is that the square is to the right of the rectangle, even though their actual positions are opposite. The reason why the collision resolution did not resolve the collision perfectly by making the polygons end up in the correct order, is because the collision resolution and kinematics step is flawed due to the numerical approach to calculating motion and forces.

- C. Since there was collision, shadow optimization is attempted. The potential shadow position (green) is again generated at a random position and orientation between the shadow at the current position, but this time at a different location since it is random. Collision detection is performed from the shadows (blue) to the potential shadow positions (green). Since there is no collision, the optimization succeeded. The shadow is updated and is set equal to the potential shadow.

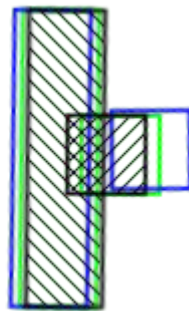


Illustration 46: Shadow optimization tick 4

Tick 4:

- A. There was collision the previous tick, so the shadow is not updated to the previous positions of the polygons.
- B. Collision detection is performed from the shadows to the current positions. Collision is detected.
- C. Since there is collision, shadow optimization is attempted. The potential shadow (green) position is again randomly generated. The green positions are overlapping so there will be a collision detected from the shadows (blue) to the potential shadows (green). Shadow optimization failed.

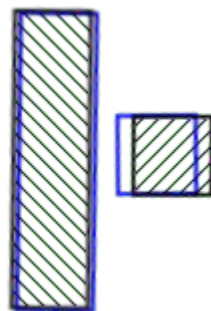


Illustration 47: Shadow optimization tick 5

Tick 5:

- A. There was collision the previous tick, so the shadow is not updated to the previous positions of the polygons.
- B. Collision detection is performed from the shadows to the current positions. Collision is not detected.
- C. Since there is no collision, optimization is skipped.

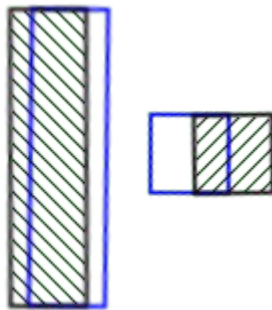


Illustration 48: Shadow optimization tick 6

Tick 6:

- A. There was no collision the previous tick, so the shadow is set to be the polygons previous positions.
- B. Collision detection is performed from the shadows to the current positions. No collision is detected.
- C. Since there is no collision, optimization is skipped.

The code for collision detection is found at [app/modules/sim/_static/physics_engine/phx.js](#)
`PHX.handleCollisionDetection()`.

6.4.9 Collision box optimization

In the broad collision detection phase, the collision detection between segments and corners are limited to the polygons that have overlapping collision boxes. This means that you still have to compare all the polygons collision boxes. Since all boxes must be compared to all others, for each

new collision box added, it must be compared to all previous boxes. The number of comparisons will be $1 + 2 + 3 + \dots + (n - 1) = (n^2 - n) / 2$ where n is the number of boxes. This gives a big O notation of $O(n^2)$. Examples of common solutions that will reduce this problem to $O(n \log(n))$ or better, is the use of a quad tree (12) or other space partitioning algorithms (13). The solution we attempted had a big O notation of $O(n)$. We sorted all the bounding boxes along the various spacial axis using insertion sort, and each time an object was swapped with a neighbor, a collision detection was made. Since sorting an almost sorted list using insertion sort is $O(n)$, this algorithm is also $O(n)$ where n is the number of objects in the simulation. If many polygons swap place, more sorting would have to be done, but as long as the energy density of the system (or energy per number of polygons per area (the polygons are spread across)) is below a threshold, the claim that the insertion sort method is $O(n)$ holds true. After the algorithm was implemented however, the constant relating performance to the number of polygons in the simulation was too high. The number of polygons needed to be simulated before the lower O-notation would save performance, was so high that the over all performance requirement for the simulation went above what we could expect from our users. The approach was scrapped and a naive $O(n^2)$ approach was implemented instead. The collision box detection only takes up about 10% of the over all performance in most cases (measured using google chromes profiler), which is not too bad. A smarter implementation can be implemented later. The code for the scrapped approach, can be found in the folder [app/modules/sim/_static/physics_engine/collision_box_handler](#).

6.5 Collision resolution

6.5.1 *Properties of polygons*

Collision resolution is about how the momentum of polygons change when interacting with each other. The result of the calculations depend on properties of the polygons, for example mass and moment of inertia. These properties depend on the geometry and density of the polygon and must be calculated from this data. The properties of the polygon can be found at [app/modules/sim/_static/physics_engine/polygon/polygon.js](#).

The mass of the polygons is density * area. See 6.7.11 for the calculation of the area. The more density an object has, the more mass it has, and the more resistive it is to change velocity. In the real world, density usually has units of the type mass per volume. Since this simulation is 2 dimensional, the unit is mass, per area. Another way of thinking of it, is that the two dimensional polygons have a depth of one unit length.

The moment of inertia is also dependent on density. The moment of inertia calculation can be found here 6.7.13. The center of mass is the axis the inertia is calculated around.

6.5.2 Normal collisions

The collision detection stage, has already calculated whether the polygons are colliding and how they are colliding. The collision information includes the points where the polygons collide and the normals and tangents of the collision.

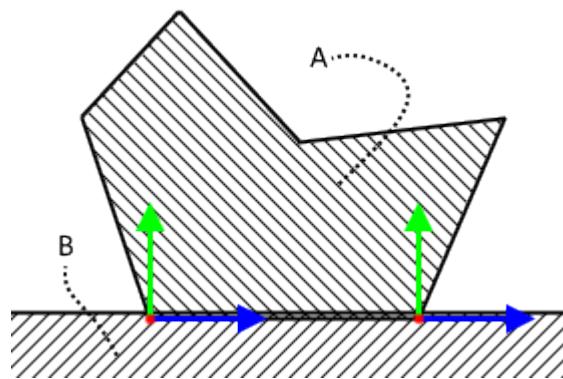


Illustration 49: Tangential and normal

The red dots are the points of collision. The information of how the pair of objects A and B are colliding, is stored within objects of the CollisionGroup class. The Green arrows (vectors) are the normals of the collisions. These are the only directions forces will be applied if there is no friction or if there is no relative horizontal movement between the polygons. The length of the normals are 1 distance unit. The blue arrows are the tangents. These are the directions of the friction forces (positive or negative direction). They also have a length of 1.

In real life, when two physical bodies get close enough to each other, electromagnetic forces between the molecules of the bodies start interacting. This happens all along the touching surfaces. To simplify calculations, the forces are merged into two forces, interacting at each contact point (red dots), see illustration 49. The force calculation is further simplified by calculating each individual force one by one, ignoring any other contact points. The normal calculation and friction calculation is also split into two calculation steps. Any outside forces, like gravity, is also ignored in the initial force calculation. Since the calculation is done in small time steps (ticks), forces are simplified to many small impulses over these small time steps that result in what is perceived as forces over time. The problem is reduced to calculating one such impulse.

Since each contact point is calculated separately and normal force and friction force is also calculated separately, the problem is simply to calculate what impulse must be applied between a pair of rigid bodies to make them not move through each other or bounce off each other if kinetic energy is conserved. Since collisions in reality usually resolve quickly, we will apply this impulse over one tick.

Here is the formula for such an impulse. To see the derivation of the formula and the definition of relative velocity and collision mass, see 6.7.10.

$$i = (1 + e) \cdot \text{relativeVelocity} \cdot \text{collisionMass}$$

Illustration 50: Collision resolution formulae

The variable e is how elastic and inelastic a collision is. e is different between each pair of objects, so its value is a combined property of the properties of the polygons. For example, a bouncy ball against a concrete floor will bounce quite well (e close to 1). A pillow versus concrete will not bounce well. A pillow versus pillow will also have an e close to 0. A way to approximate this behavior is to have each polygon have its own e , and then multiply them together to calculate the shared e .

```
var i = relVel / invCollisionMass;
//...
var bounceFactor = polygonA.bounceFactor * polygonB.bounceFactor;
//...
i *= (1 + bounceFactor);
```

The code can be found at [app/modules/sim/_static/physics_engine/contact_point.js](#)

`ContactPoint.resolveByImpulseNormal()`.

6.5.3 Friction

The friction model used, is the static-dynamic friction model. When the normal impulse over one tick between polygons, multiplied by a static friction constant, is smaller than the impulse required to stop the polygons from sliding across each other, static friction is used. Otherwise, dynamic friction is used.

$$\text{maximumFriction} = \begin{cases} n \cdot s > f : & n \cdot d \\ n \cdot s \leq f : & f \end{cases}$$

Illustration 51: Friction formulae

n is the normal impulse over one tick, s is the static friction constant, f is the friction impulse required to stop the polygons from sliding, d is the dynamic friction constant.

The friction impulse f is calculated the same way as the normal impulse, except that the impulse is in the tangent direction instead of the normal direction, an inelastic coefficient is always used.

The friction constant is a number from 0 to infinity. 0 is no friction, and infinity is total friction. As with bouncing friction, the friction coefficients are multiplied together to calculate the friction between polygons.

```
var i = relVel / invCollisionMass;
// ...
var maxFriction;
if (isStaticFriction) {
    maxFriction = this.collisionGroup.polygonA.staticFrictionConstant *
        this.collisionGroup.polygonB.staticFrictionConstant *
previousSumOfnormalImpulses;
// ...
if(!isStaticFriction){
    maxFriction = this.collisionGroup.polygonA.dynamicFrictionConstant *
        this.collisionGroup.polygonB.dynamicFrictionConstant * normalImpulse /
        this.collisionGroup.phx.collisionResolutionImpulsePrecision;
// ...
if(i < -maxFriction){
    i = -maxFriction;
}else if(i > maxFriction){
    i = maxFriction;
}
```

Positive and negative friction impulses refer to the direction of the friction.

The code for friction can be found at [app/modules/sim/_static/physics_engine/contact_point.js](#)
 ContactPoint.resolveByImpulseFriction().

6.5.4 Compensators

Due to friction, mechanical energy will over time be removed from the mechanical system (become

thermal energy). This will lead to the rigid bodies coming to rest. If a stack of polygons are made, the correct forces must be applied between them so that the stack does not fall down or float upwards.

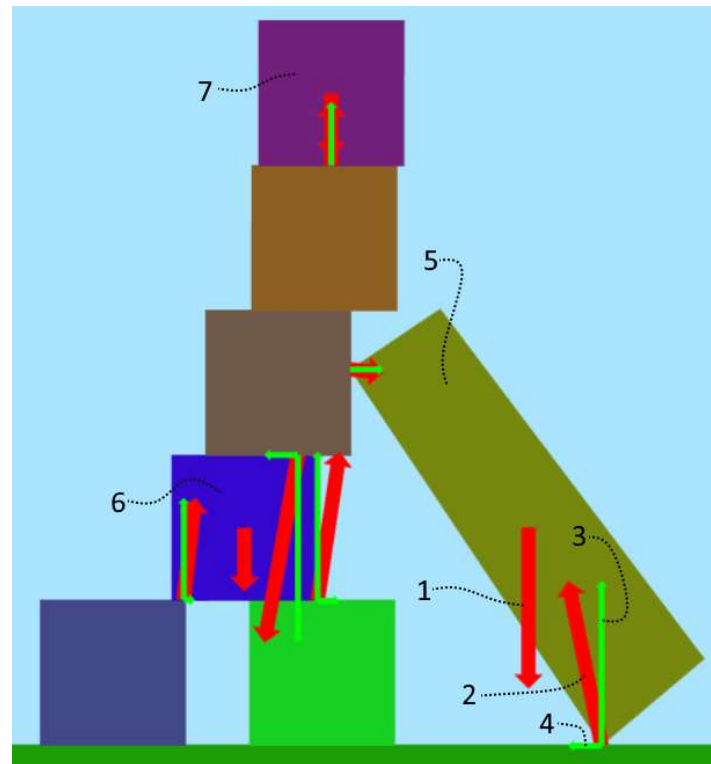


Illustration 52: Compensators

The figure is a screenshot of the simulation. To activate the force visualization tool see page 22.

The red arrows show forces. Not all forces are shown, only the ones affecting polygon 5, 6, and 7. The forces acting on the centers of the polygons are gravity. An example of a gravity force is 1. The rest of the force arrows are electromagnetic forces. These forces are decomposed into normal and friction forces. An example is the force 2 that is decomposed into the friction component 4 and the normal component 3. As we can see, the further down in the stack a polygon is, the larger the forces are, see 6. The higher up in the stack we are, the smaller the forces are, see 7.

The forces applied must take into account how much mass is above each object and if objects are leaning on each other, among other factors. The sum of forces and torques must be 0 for each polygon in the system for the system to be static. Somehow the polygons at the bottom of the stack, must know the magnitude of the normal and friction forces needed to support all polygons above it. This requires some form of communication through the stack.

We made several failed attempts at solving this communication problem. Here is a summary of some of them.

Freezing non moving objects

This failed as it was difficult to calculate when the objects should be unfrozen again. Changes to the forces affecting the frozen system caused a need of calculating the inner forces anyway.

Spring resting forces

The idea behind this approach is that when polygons collide, the electromagnetic forces act similarly to small springs. The more the polygons overlap, the larger the spring forces between the polygons will be, eventually leading to stable resting positions. This attempt failed due to two reasons. The first reason is that for larger forces to be maintained between polygons, there had to be a large overlap. The polygons at the bottom would sink through each other proportionally to the mass they had to support. The second reason is that the larger the forces, the larger the numerical error in integrating the movement of the polygons over the spring forces. This caused large instability of mechanical energy. This breaks axiom 3 (page 33).

Force building up over time

This approach attempts to fix the problem caused by the spring attempt where large forces would require large overlaps. When the overlap between polygons is larger than a predefined threshold, a compensation force is incremented. By each tick, this compensation impulse is added to the force calculated with the impulse formula from illustration 50. The attempt failed since the polygons started to oscillate. When the overlap of the polygons were smaller than the threshold, they would increase their acceleration outwards, and when they were above the threshold, the acceleration would decrease.

The solution

The solution that we ended up with is inspired by the "force building up over time" solution. Instead of increasing the compensator force over time, we increased it over displacement. Let us take a look at the physics pipeline again:

1. Collision resolution ← We are here
 - A. Apply normal and friction + compensators
 - B. Measure errors

2. Kinematics
3. Collision detection
4. Render
5. Wait
6. Repeat

Everything that has to do with collision resolution happens in step 1. At step A, collision is resolved based on how the polygons are moving. At the initial collision, the compensators will be 0. As explained earlier (page 68), collision is resolved between each pair of polygons at one contact point at a time, ignoring the rest of the system including gravity. If step A was repeated multiple times, the collision resolution would get more and more accurate, but as long as there is a chain of contact points longer than 1, the number of iterations of step A required to completely resolve the collision would be a large number or even infinite. The solution is to measure the error at step B, and then apply an adjusted compensator force at step A the next tick. This solution is not perfect, as sometimes the compensator will overshoot, but if it does overshoot, it will reduce the compensators the next iteration. The more polygons are stacked on top of each other, or the heavier the polygons are at the top, the more oscillation will occur, but the oscillation will still dampen over time.

Friction compensators are calculated in a similar way to normal compensators. The difference is that the friction compensators are reset to 0 when the friction is dynamic.

The way the magnitude of the forces is communicated through the stack is similar to how multiple people cooperate when lifting a piece of furniture. If four blind folded people stand around a table and are lifting their end of the table, they will indirectly communicate to each other on how much force to use. If someone is lifting more than someone else, the table might start tilting. The other people will notice this and try to compensate. The stack of polygons work the same way. When a new polygon is added to the top of the stack, the polygons below, will measure that they are moving downwards and need to increase their resting force. This increase in resting force might cause other polygons to have to further increase their resting force and so on. The code for measuring error can be found at [app/modules/sim/_static/physics_engine/contact_point.js](#)

ContactPoint.calculateCompensatorNormal(). The compensator is applied at [app/modules/sim/_static/physics_engine/contact_point.js](#) ContactPoint.js

`resolveByImpulseNormal()`.

6.5.5 Displacement resolution

The method of removing velocity between polygons by applying forces, only prevent overlap from increasing. If an overlap first has occurred, it must be removed by other means. The method used is to displace the polygons by 30% of their overlap each tick. The reason 100% is not used, is because polygons might be displaced in multiple directions at the same time, from multiple polygons. This might cause the sum of displacement to go past 100%. Experiments show that 30% gives the desired results.

When displacing the polygons position, the `Polygon.displaceByImpulse()` is used. See page 106 for the functions inner functionality. The reason this function is used, is it acts similar to the way an impulse is applied, but it causes displacement without momentum being added to the polygon. Since the displacement is only in the normal direction, overlapping polygons will only be displaced in the normal direction without sliding due to the displacement.

The code for the displacement is found here:

[app/modules/sim/_static/physics_engine/contact_point.js](#) `ContactPoint.resolveByDisplacement()`.

6.6 Tools

Tools are used to change the state of the simulation. This includes creation of polygons, moving polygons around, creating constraints, or changing properties of constraints and polygons. The explanation of the tools is more technical here than what is found at page 21.

6.6.1 Default

The code for this tool can be found at

[app/modules/sim/_static/user_interface/user_interface_handler.js](#)

`UIHandler.handleDefault()`.

The default tool was the first tool created. It is used to select polygons to drag them around in the scene. A force propotional to the difference between the grap position and the mouse position is applied to the polygon. This force is also propotional to the mass of the object being dragged. A dampener propotional to the grab positions velocity is added to prevent occolation. The way impulse is applied is described at page 106.

6.6.2 *Move tool*

The code for this tool can be found at

[app/modules/sim/_static/user_interface/user_interface_handler.js](#)

`UserInterfaceHandler.handleMoveTool()`.

Sometimes just applying a force (like what the default tool does) does not give a good enough precision. The move tool accelerate the polygon (both in the translational and angular directions) so that the polygon ends up with no velocity at the mouse position. The function makes sure the acceleration caused is not above a constant (in code the variable is "maxAcceleration" or "speed" depending on whether you are in play mode or in pause mode). The constant acceleration, makes the movement smooth and reduces the chances of polygon bumping into polygons you don't want to affect. In pause mode, the simulation does not move polygons based on their velocities, but will still resolve overlapping. In this mode, the move tool can be used to move polygons even if the polygons are not affected by their momentum. This is another advantage the move tool has to the default tool. The move tool was intended as a replacement to the default tool, but we kept it since it was still useful in some cases.

6.6.3 *Custom shape spawner*

The code for this tool can be found at

[app/modules/sim/_static/user_interface/user_interface_handler.js](#)

`UserInterfaceHandler.handleCustomShapeSpawner()`.

When polygons are spawned into the world, the previous position property of the polygons are set to be the same as the current positions of the shapes. Since collision detection is taken from the previous position to the current position (see 6.4.3), it is important that the spawned polygons, are not overlapping with existing polygons. To test for overlapping, the circumferences of the new polygon and existing polygons are compared, segment by segment to see if any of them overlap. If there is overlapping, the new polygon cannot be spawned. In the case one polygon is completely inside another, one of the corners of one polygon, is tested to see if it is completely inside the other polygon. This requires a function capable of testing if a point (corner) is inside a polygon (14). The implemented point inside polygon algorithm can be found at [app/modules/sim/_static/utilities/tool.js](#) `Tool.positionInsidePolygonInclusive()`.

6.6.4 Polygon drawing

The code for this tool can be found at

[app/modules/sim/_static/user_interface/user_interface_handler.js](#)

`UserInterfaceHandler.handlePolygonDrawing()`.

The drawing tool, is similar to the custom shape spawner in the sense that a polygon cannot be placed if it is overlapping with a currently existing polygon. The "area of polygon" algorithm 6.7.11 is used to determine the orientation of the circumferences. A negative area means clockwise orientation, positive means counter clockwise. This information is then used to determine if the polygon you have created is possible to make or not.

6.6.5 Static polygons

The code for this tool can be found at

[app/modules/sim/_static/user_interface/user_interface_handler.js](#)

`UserInterfaceHandler.handleToggleStatic()`.

Static polygons are polygons with infinite mass. Two polygons that are static cannot collide. The "toggle static" tool, will make polygons static, and set their velocities (angular and translational) to zero.

6.6.6 Show forces

The code for this tool can be found at

[app/modules/sim/_static/user_interface/user_interface_handler.js](#)

`UserInterfaceHandler.handleShowForces()`.

This mode make the forces between polygons visible using arrows to represent them. The force vectors are decomposed into the normal direction and friction direction. The implementation of this tool is still in its beginning stage of implementation. The placement of the force vectors is simplified and might not always be completely accurate. Also, only forces between polygons due to direct contact or gravity is shown. Forces from ropes and hinges are not shown.

6.6.7 Hinges

The implementation of hinges can be found at

[app/modules/sim/_static/physics_engine/hinge_constraint.js](#).

Hinges are constraints that locks a point on one polygon to another point at another polygon. The constraint primarily restricts relative motion of the points connected together, but can also have friction, restricting relative rotation. Since hinges can direct downwards pointing kinetic energy upwards, the friction of hinges had to be applied in a special way. Due to the numerical error in the numerical integration of the friction forces in chains of hinged together polygons, friction could lead to the creation of energy. Since the problem was related to the order in which friction was applied the hinges, the problem was reduced by applying friction first in one order, and then in the reversed order. This two way appliance of friction is repeated 4 times each tick. The solution is not perfect, but reduces the problem till it is invisible. See [app/modules/sim/_static/physics_engine/phx.js](#) `PHX.handleCollisionResolutionByImpulse()` to see the fix for this problem.

The first attempt at implementing hinges was to pretend, the polygons were colliding at the connection point with a normal opposite to the relative velocity at the connection point. Since an impulse to a point on a polygon, does not necessarily changes the velocity of that point in a parallel direction to the impulse, this attempt cause the hinge to shake. **Note:** The translational velocity will change in the direction of the impulse, but not a point at a polygon that is not the center of mass of said polygon. This shaking happens more if the moment of inertia of the polygons were small compared to the lengths from the polygons center of mass to the connection point. A more mathematical approach was made, to ensure the impulses between the polygons at the contact point, truly stopped all relative velocity. See the solution at page 107.

6.6.8 Ropes

The implementation of ropes can be found at [app/modules/sim/_static/physics_engine/rope.js](#).

A rope has one end attached to a point at one polygon and another end attached to a point on another polygon. To make sure axiom 3 is followed (page 33), the implementation of ropes should have taken the same form as with hinges. This would take longer to implement, and a simpler approach was temporarily taken instead. The simpler approach is actually more similar to how real ropes work. The rope is simulated as a spring that starts resisting past a certain length. The spring constant is very strong. This cause instability in conservation of energy and thus break axiom 3. The spring constant is proportional to the collision mass (what is collision mass? See 6.7.10). This makes the rope adjust its force depending on whether the connected polygons are massive or small. Ropes also have friction. The friction of a rope is proportional with how quickly it stretches.

6.6.9 Pulleys

The implementation of pulleys can be found at app/modules/sim/_static/physics_engine/pulley.js.

The pulleys connect multiple ropes together to make them act as one longer rope. Before the rope impulses are applied, the pulley will distribute the length of the total rope (total length) (the sum of all individual ropes) among the individual rope sections. The total length will be distributed in such a way that when the spring forces are applied, each rope section will pull with the same force.

$$[f_0, f_1, f_2, \dots, f_n]$$

$$[M_0, M_1, M_2, \dots, M_n]$$

Illustration 53: Fractions and collision masses

We want to distribute the offset causing the spring forces among each rope segment so that the total offset (length of the total rope minus length the rope spans over) for each rope times its collision mass is the same for each rope since this will result in the same force (offset * collision mass = spring force). Say we have two sets f_i and M_i . i is from 0 to n . f_i is the fraction of the total offset length rope-segment i will bear. M_i is the collision mass of rope i . We set up the equations in maxima (15).

```

(%i32) expand(solve(
[
f_0+f_1=1,
f_0*M_0=f_1*M_1
],
[f_0,f_1]
));
(%o32) [[f_0 =  $\frac{M_1}{M_1+M_0}$ , f_1 =  $\frac{M_0}{M_1+M_0}$ ]]

(%i33) expand(solve(
[
f_0+f_1+f_2=1,
f_0*M_0=f_1*M_1,
f_1*M_1=f_2*M_2
],
[f_0,f_1,f_2]
));
(%o33) [[f_0 =  $\frac{M_1 M_2}{M_1 M_2 + M_0 M_2 + M_0 M_1}$ , f_1 =  $\frac{M_0 M_2}{M_1 M_2 + M_0 M_2 + M_0 M_1}$ , f_2 =  $\frac{M_0 M_1}{M_1 M_2 + M_0 M_2 + M_0 M_1}$ ]]

(%i34) expand(solve(
[
f_0+f_1+f_2+f_3=1,
f_0*M_0=f_1*M_1,
f_1*M_1=f_2*M_2,
f_2*M_2=f_3*M_3
],
[f_0,f_1,f_2,f_3]
));
(%o34) [[f_0 =  $\frac{M_1 M_2 M_3}{M_1 M_2 M_3 + M_0 M_2 M_3 + M_0 M_1 M_3 + M_0 M_1 M_2}$ , f_1 =  $\frac{M_0 M_2 M_3}{M_1 M_2 M_3 + M_0 M_2 M_3 + M_0 M_1 M_3 + M_0 M_1 M_2}$ , f_2 =  $\frac{M_0 M_1 M_3}{M_1 M_2 M_3 + M_0 M_2 M_3 + M_0 M_1 M_3 + M_0 M_1 M_2}$ , f_3 =  $\frac{M_0 M_1 M_2}{M_1 M_2 M_3 + M_0 M_2 M_3 + M_0 M_1 M_3 + M_0 M_1 M_2}$ ]]

```

Illustration 54: Maxima screenshot

We let Maxima solve three cases of the problem when there are two, three and four ropes. The fractions f_0 to f_n must add up to 1 and the fractions times collision mass must be the same for all ropes. When increasing n from 1 to 2 to 3, we can see a pattern. The pattern is abstracted to the following formulae.

$$f_i = \frac{\frac{\prod_{j=0}^n M_j}{M_i}}{\sum_{k=0}^n \frac{\prod_{l=0}^n M_l}{M_k}}$$

Illustration 55: Fraction formulae

If we multiply f_i with the total offset length of the rope, we get the offset of rope segment i (how

much of the total offset rope i will bear).

The dampening of the ropes happens as the ropes stretch. To make the dampening correct when multiple rope segments are put together, the velocity of the distance between the two attachment points for each rope is summarized. This gives the velocity for the total rope and is used when calculating the dampening.

The code code for this distribution of rope suspension and velocity is found at [app/modules/sim/_static/physics_engine/pulley.js](#) `Pulley.adjustRopeLengths()`.

6.6.10 Undo redo

When a tool is used, the state of the simulation, is saved. This state can be loaded at a later point in time. A history of previous states are stored. This allow for undo and redo functionality.

6.7 Definitions and proofs

6.7.1 Corner

In this text, a corner is a point in 2d space, that is used to make up polygons. A corner can be concave or convex or. A convex corner is one that has an inner angle larger or equal to 0 and less or equal to 180 degrees. A corner is concave if the inner angle is larger that 180 degrees and less or equal to 360 degrees. Two corners make up a segment. If we move in the counter clockwise direction around the outer boundary of a polygon, the inside of the polygon is to the left.

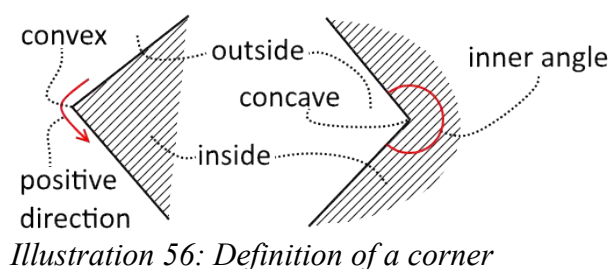
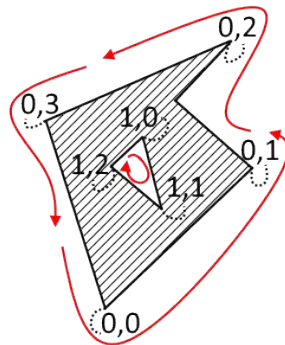


Illustration 56: Definition of a corner

6.7.2 Polygon

A polygon consists of a 2 dimensional list of corners. Each subsequent pair of corners form a segment, in addition to a segment starting at the last corner in the list and ending at the first corner in the list. The first corner is referred to as corner 0, then next one 1, then 2 and so on. Each list of corners creates a circumference of segments. Each segment does not intersect with any other

segment, except for at the end positions. At each end position, each segment intersects exactly one other segment that is either the next segment, or the previous one with respect to the list of corners making up the segments. A circumference's positive direction is the same direction as the order of the corners that makes up the circumference. If you move in the positive direction of the circumference, the inside of the polygon is to the left. No corner is allowed to be on the outside of any circumferences. There must always be exactly one outer circumference (a circumference oriented counter clockwise).



*Illustration 57:
Polygon list of
corners*

The red arrows show the positive direction. The numbers are the placement of the corners in the 2d list. The first index is the index of the outer list, and the second index is the index of the inner lists.

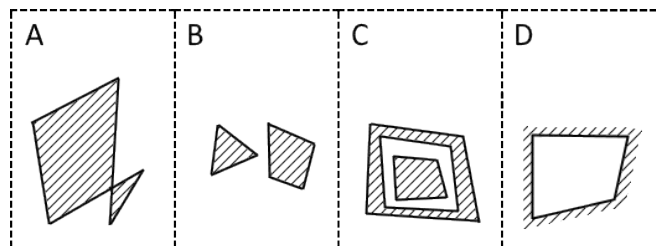


Illustration 58: Illegal polygons

The polygons in illustration 58 show illegal polygons. A has two segments overlapping at a point that is not the endpoints. B show a polygon with two outer circumferences. This breaks the rule that all corners must be inside all other circumferences. C breaks the same rule. D breaks the rule of having exactly one outer circumference.

6.7.3 Mesh

A mesh is a set of triangles forming a surface used for rendering.

6.7.4 Theorem 1

For two polygons' surfaces to intersect, at some point, a corner of one of the polygons will at some point pass through a segment of the other polygon, or the other way around.

The proof is as follows:

At the moment of intersection, the two polygons' circumferences will touch, but not completely penetrate. See figure below:

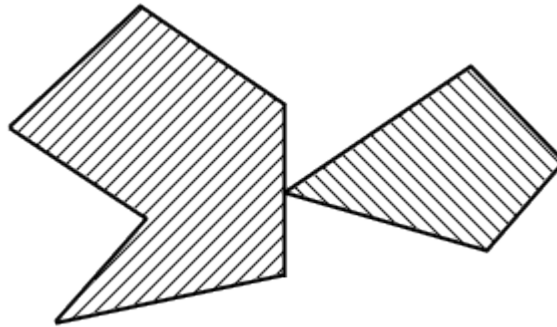


Illustration 59: Polygons intersecting

Since the circumferences are made up of segments, when the circumferences intersect, at least one pair of segments will intersect. Since the circumferences only intersect, but does not penetrate, the same will be true for the segments. An intersection between the segments will take one of these forms:

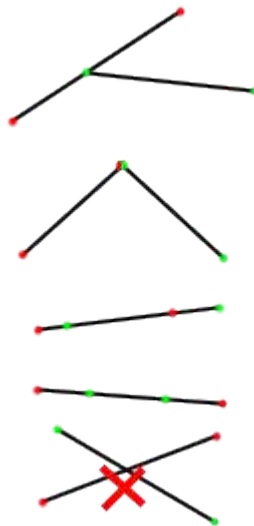


Illustration 60: Cases of intersection

In the four top cases, there is a corner intersecting with a segment. Q. E. D. The bottom case will not occur at the moment of initial intersection as there is a penetration.

6.7.5 Corner segment are colliding

This is the explanation of the math and logic behind the function `cornerSegmentAreColliding()`. The function is found in `app/modules/sim/_static/physics_engine/contact_point.js`. The signature of the function is as follows.

```
ContactPoint.cornerSegmentAreColliding = function (pointPrevious, pointNow,
segStartPrevious, segStartNow, segEndPrevious, segEndNow, margin = 0)
```

The purpose of the function is to check if a point moving in a linear path from `pointPrevious` to `pointNow` is at any point intersecting a line segment. The line segment is at the initial position spanning from `segStartPrevious` to `segEndPrevious`, and is at the end position spanning from `segStartNow` to `segEndNow`. The margin is a number used to increase the length of the segment by a tiny amount to counter floating point rounding errors. The margin is 0 by default. The time unit used within the function is referred to as "t". The return value of the function is t. t is the time when the point first intersects the segment. An early intersection gives a t close to 0. A late intersection gives a t close to 1. This means that if the point is always intersecting the segment, 0 will be returned. If no intersection occurs in between $t = 0$ and $t = 1$, -1 will be returned.

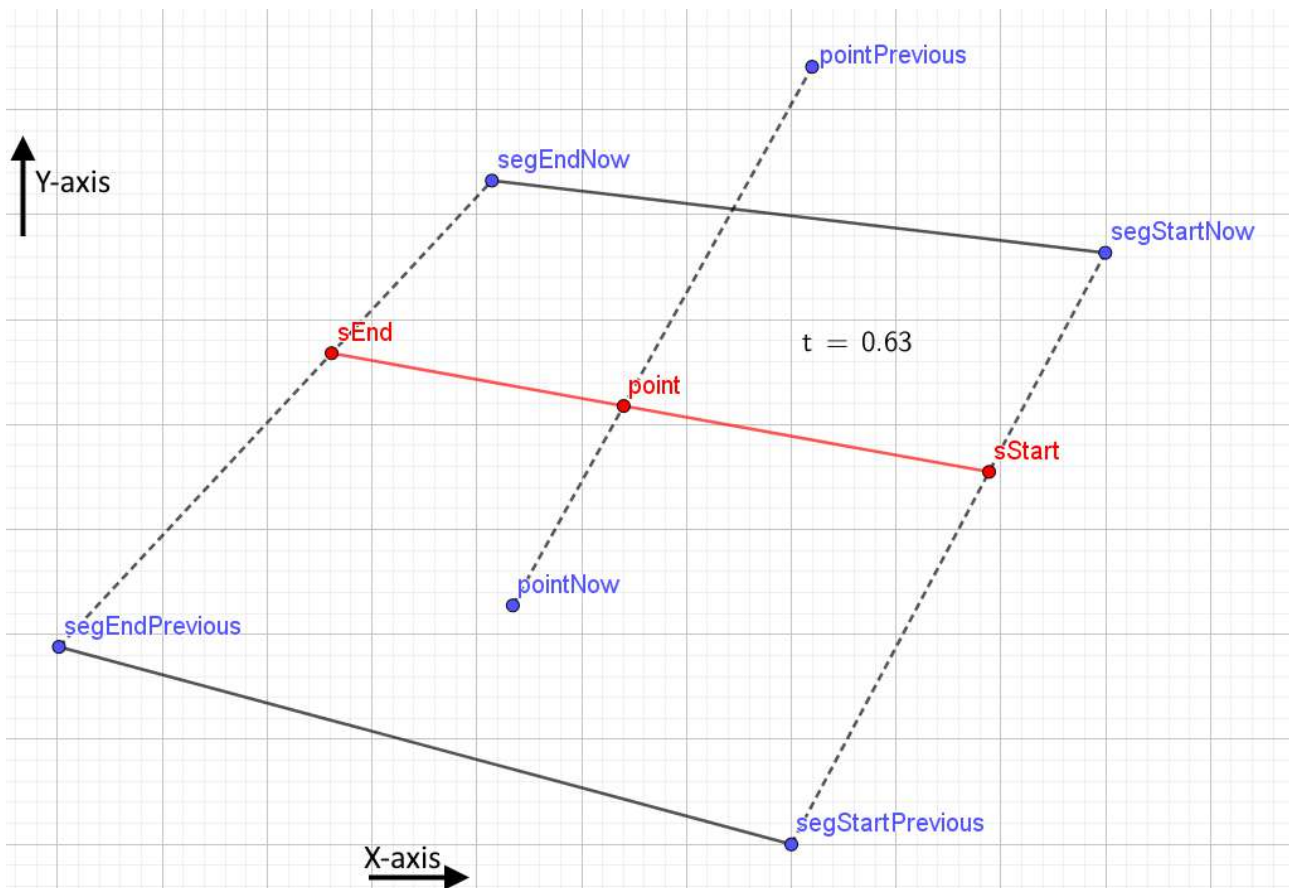


Illustration 61: Point in time

t is here calculated to be 0.63. This means that 0.63 of the time between the previous tick and the current one, the point intersects the segment.

To calculate t at the point of intersection, the first step is to transform the coordinate system to simplify the problem.

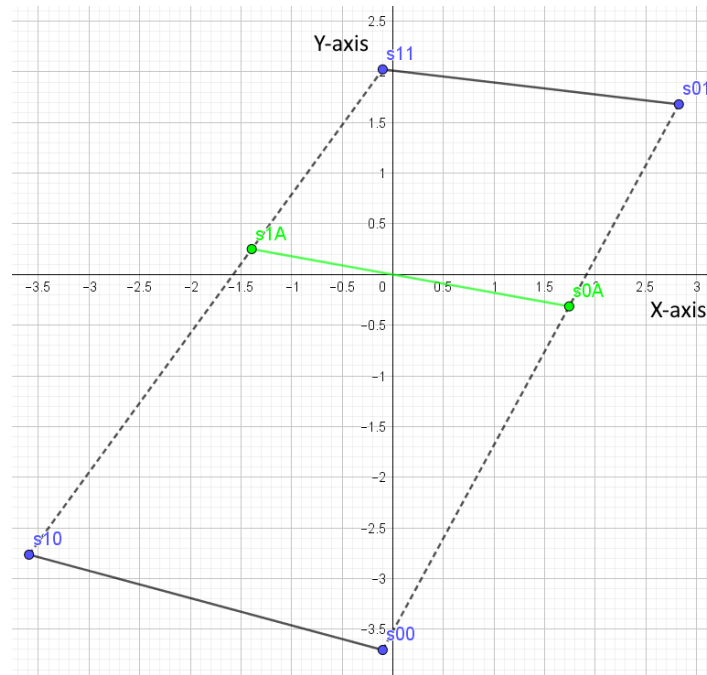


Illustration 62: Point of intersection

```
var s00 = Vec2.sub(segStartPrevious, pointPrevious);
var s01 = Vec2.sub(segStartNow, pointNow);

var s10 = Vec2.sub(segEndPrevious, pointPrevious);
var s11 = Vec2.sub(segEndNow, pointNow);

var s0v = Vec2.sub(s01, s00);
var s1v = Vec2.sub(s11, s10);
```

`Vec2.sub(a, b)` return the vector $a - b$.

The coordinate system is changed so that the corner has a velocity of 0 and is inside the origin. $s00$ to $s10$ is the initial position of the segment and $s01$ to $s11$ is the end position of the segment. If at any point during the motion of the segment, the origin is inside the segment, there is an intersection as the point is inside the origin. $s0A$ to $s1A$ is the segment when there is an intersection. The moment the segment intersects the origin, the vectors $s0A$ and $s1A$ are parallel. If we can find the time when the vectors are parallel, we can use the time to find $s0A$ and $s1A$ (and other solutions as the equation we end up with is quadratic) and then see if the origin is intersecting the segment, or if the segment is simply being inside an axis through the origin. When the determinant of two vectors is 0, the vectors are linearly dependent, and thereby parallel. Before doing the calculations, let's begin with some notations.

$$\begin{vmatrix} \vec{a} \\ \vec{b} \end{vmatrix} = \vec{a}_x \vec{b}_y - \vec{a}_y \vec{b}_x$$

Illustration 63: Vector notation

Small arrows above variables mean they are vectors. The vectors are two dimensional unless otherwise stated. The determinant of two vectors, $\vec{a}$ and $\vec{b}$ in the figure, is noted as above. The right side is how the determinant is defined. Determinants between 2d vectors can also be thought of as the z-component of the cross product of two 2d vectors. The cross product between 2d vectors can be defined as a cross product between 3d vectors with equal x and y components as the 2d vectors, but with 0 as z components. If the angle between two vectors are 0 or 180 degrees, the cross product gives a 0-vector. This means that the determinant of two 2d vectors is 0 if they are parallel. We take use of this to check for parallelity between the vectors describing the movement of the segment. After we have transformed the coordinate system, as seen from the code example, we now have the four vectors s_{00} , s_{10} , s_{01} and s_{11} , describing the motion of the segment relative to the point which is inside the origin with zero velocity. To describe the movement of the segment, we define the movement vectors s_{0v} and s_{1v} . The position of the segment at time t can be described as starting at point $s_{00} + s_{0v} * t$ and ending at $s_{10} + s_{1v} * t$. When we check for parallelity we set up the following equation.

$$\begin{vmatrix} s_{00} + s_{0v}t \\ s_{10} + s_{1v}t \end{vmatrix} = 0$$

Illustration 64: Parallel equation

The definition from illustration 63 leads to the following:

$I :$

$$\begin{vmatrix} \vec{a} + \vec{b} \\ \vec{c} + \vec{d} \end{vmatrix} = \begin{vmatrix} \vec{a} \\ \vec{c} \end{vmatrix} + \begin{vmatrix} \vec{a} \\ \vec{d} \end{vmatrix} + \begin{vmatrix} \vec{b} \\ \vec{c} \end{vmatrix} + \begin{vmatrix} \vec{b} \\ \vec{d} \end{vmatrix}$$

$II :$

$$t \begin{vmatrix} \vec{a} \\ \vec{b} \end{vmatrix} = \begin{vmatrix} t\vec{a} \\ \vec{b} \end{vmatrix} = \begin{vmatrix} \vec{a} \\ t\vec{b} \end{vmatrix}$$

Illustration 65: Rules of determinants

"t" is a scalar. The equalities can easily be proven by using the definition from illustration 63. Now we solve the equation from illustration 64.

$$\begin{vmatrix} s_{00} + s_{0v}t \\ s_{10} + s_{1v}t \end{vmatrix} = 0$$

$$\begin{vmatrix} s_{00} \\ s_{10} \end{vmatrix} + t \begin{vmatrix} s_{00} \\ s_{1v} \end{vmatrix} + t \begin{vmatrix} s_{0v} \\ s_{10} \end{vmatrix} + t^2 \begin{vmatrix} s_{0v} \\ s_{1v} \end{vmatrix}$$

$$t^2 \begin{vmatrix} s_{0v} \\ s_{1v} \end{vmatrix} + t \left(\begin{vmatrix} s_{00} \\ s_{1v} \end{vmatrix} + \begin{vmatrix} s_{0v} \\ s_{10} \end{vmatrix} \right) + \begin{vmatrix} s_{00} \\ s_{10} \end{vmatrix} = 0$$

Illustration 66: Quadratic equality

$$a := \begin{vmatrix} s_{0v} \\ s_{1v} \end{vmatrix} \quad b := \begin{vmatrix} s_{00} \\ s_{1v} \end{vmatrix} + \begin{vmatrix} s_{0v} \\ s_{10} \end{vmatrix} \quad c := \begin{vmatrix} s_{00} \\ s_{10} \end{vmatrix}$$

$$t = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

$$t = \frac{2c}{-b \pm \sqrt{b^2 - 4ac}}$$

Illustration 67: Time solution

The top variant of the solution of t, is the one usually used for solving quadratic equations. A problem with this variant, is that if a is really small, the solution could get inaccurate. Since a is the result of a determinant, which include subtraction, the result could be wrong due to floating point

errors by a margin much larger than the magnitude of a itself. When a is then used as a divisor, this leads to a result that could be many orders of magnitude away from the correct result. To solve this problem, the approach explained here (16) is used. Once the time is found, each solution is tested. If the time is less than 0 or larger than 1, there is no collision according to that solution. If there is a solution within the time range, the segment at that point in time is calculated. Such a segment is described by 2 vectors. The green segment from illustration 62 is an example of such a segment. If the dot product between the vectors describing the beginning and end of the segment is less than 0, there is no collision according to that solution since the origin is not inside the segment. If there is a collision according to one or both solutions, the point collides with the segment.

6.7.6 The separating axis theorem

Usually the base shape used in 2d simulations is a geometrical shape bounded by straight line segments. Each line segment is between two vertices that define the corners of the shape. Each corner must be convex or the angle of the corner must be 180 degrees (the vertex is between two segments that form a longer segment that is also straight). There can not be holes in the shape.

The standard collision detection is based on overlapping tests. The idea is that if two surfaces overlap, but no impulses are preventing the surfaces to penetrate each other, shortly after, the shapes will overlap with each other.

The idea of the separating axis theorem is that if two such geometrical shapes does not overlap, an axis exists that separate the two shapes.

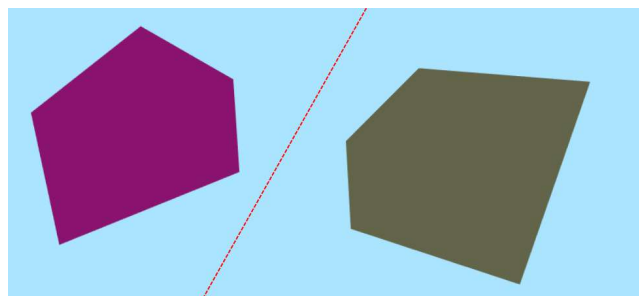


Illustration 68: Separating axis

If no such axis exists, the shapes are not overlapping. If such an axis exists, an axis through one of the segments of one of the shapes, will be such an axis. So the algorithm for collision detection between two shapes, will be about checking if there exists a separating axis along all of the segments of both of the shapes.

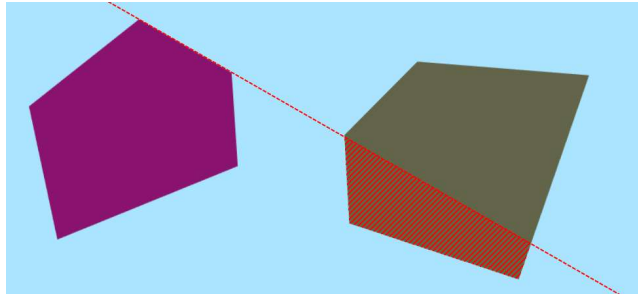


Illustration 69: Not a separating axis

This is not a separating axis.

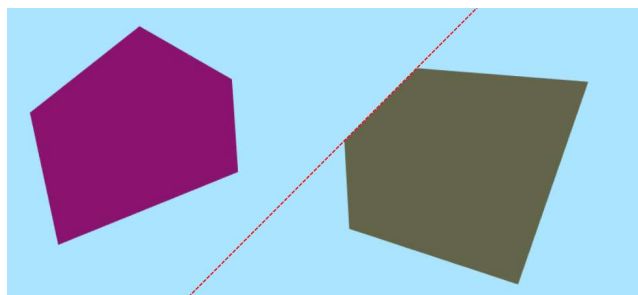


Illustration 70: Valid separating axis

This is a separating axis.

The advantages of this algorithm is that it is easy to implement and requires few computations. However, there is also several disadvantages to this approach. These problems and attempts at solving them will explained in the two following chapters.

6.7.7 The tunneling problem

The tunneling problem is a problem related to the overlap test approach. If the shapes move too fast, or are too small, bodies may completely or partly pass by each other during a time step.

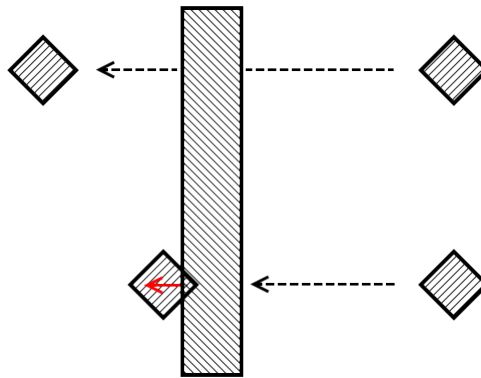


Illustration 71: Example of tunneling during timestep

In the picture, there are three shapes. One rectangle and two smaller squares that move through the rectangle during one time step. The top square moves so fast that even though the surfaces are supposed to touch, the square does not overlap with the rectangle at the beginning or at the end of the time step. This causes the collision to be missed. The second square is moving slow enough for collision to be detected, but the penetration depth is so large that the collision engine interprets that the collision normal is pointing to the left. Just based on the overlapping, it is impossible to see what is the direction the square came from and in what direction it should end up at after the collision resolution step.

6.7.8 Concave corners and holes

Another problem with the separating axis theorem is, it does not support concave polygons. The definition of convex and concave corners is found at 6.7.1. The separating axis theorem solely supports convex-only polygons. A workaround for this is to split complex shapes into convex-only polygons. Here is an example of this:

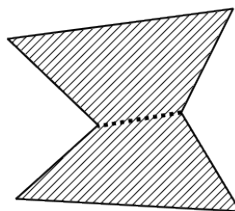


Illustration 72: Complex shape to convex only

The tunneling problem combined with this way of creating concave polygons often cause various

glitches. Fast colliding polygons might get entangled inside each other. The overlap is so large that the calculation of the normal direction fail. Inconsistency cause the polygons to be stuck.

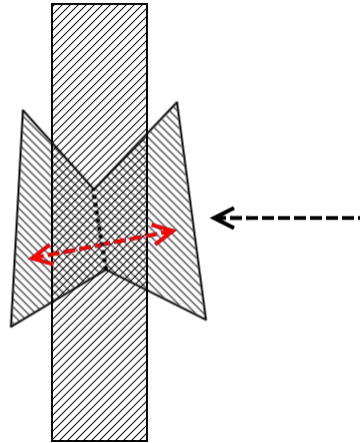


Illustration 73: Complex shape stuck problem

Here the combined polygon moves into the vertical rectangle in one times step. The two convex-only components disagree on the collision direction. In simple cases like this, workarounds are easy to find. As the complexity of both colliding shapes increases, finding a method that works consistently gets difficult. This is where our method comes into play. Instead of making new layers of workarounds on top of the existing methods (overlap based testing and separating axis theorem), we take a step back and develop something that solve these problems from the bottom up. See 6.4.3.

6.7.9 You can always split a polygon

A polygon will always have a convex corner. If it did not, the outer circumference would never loop back to where it started. Lets look at one of these convex corners and consider two possible situations.

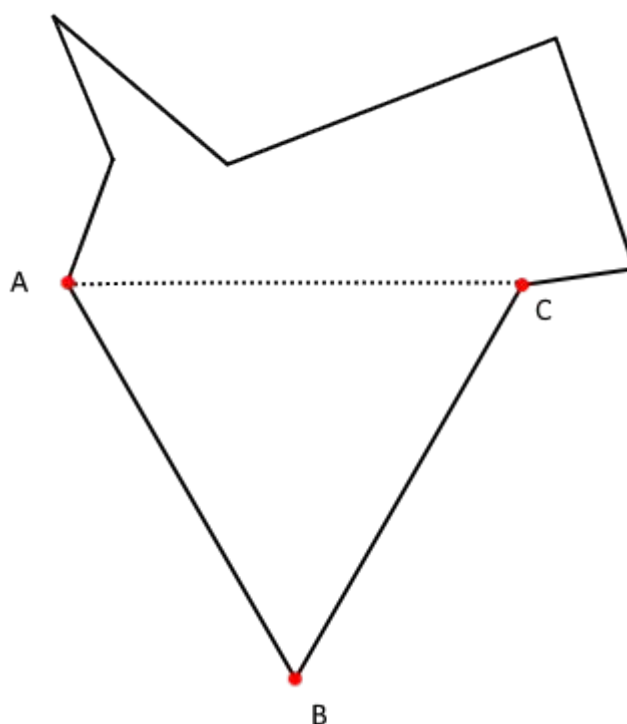


Illustration 74: Simple polygon split

The convex corner is the corner between the vertices A, B, and C. The illustration above describes the first possible situation. Here the geometry is all outside of the ABC triangle. Now a segment can be drawn between A and C.

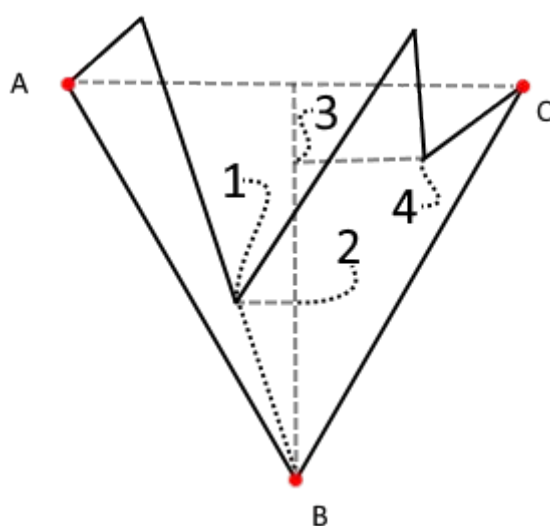


Illustration 75: Complex polygon split

The second possible situation is when there is geometry inside the ABC triangle. Now let's define a direction called "v". $v = B - (A+C)/2$. Here we treat A, B, and C as vector positions. The geometry

(corners and segments) inside the triangle that is furthest in the v direction, will be one or multiple corners and potentially some segments. In the example, the corner that is closest in the v direction, is corner 1. If corner 1 and corner 4 is projected onto v , we see that 1 is furthest in the v direction (2 is further down than 3). If we draw a segment from 1 towards B, it will not intersect any other geometry since all points on the segment except from the start position, will be closer in the v direction then the rest of the geometry that is inside the ABC triangle.

Since the two situations are the only possible situations, this proves a polygon can always be split from one corner to another. **Note:** this also works if the polygon has holes.

6.7.10 Collision formulae

The calculation is based upon this (17).

The goal is to find the magnitude of an impulse that is to be applied at a known point, in a known direction, between two polygons. The impulse conserves momentum and energy.

Let us call the impulse we are looking for " i ". We will start with calculating the collision resolution in an energy conserving way, that is, a perfectly elastic collision. Afterwards we will make the formula support inelastic collisions. The impulse we are looking for will be in the normal direction. It will be both conserving momentum, translational and angular, and also energy.

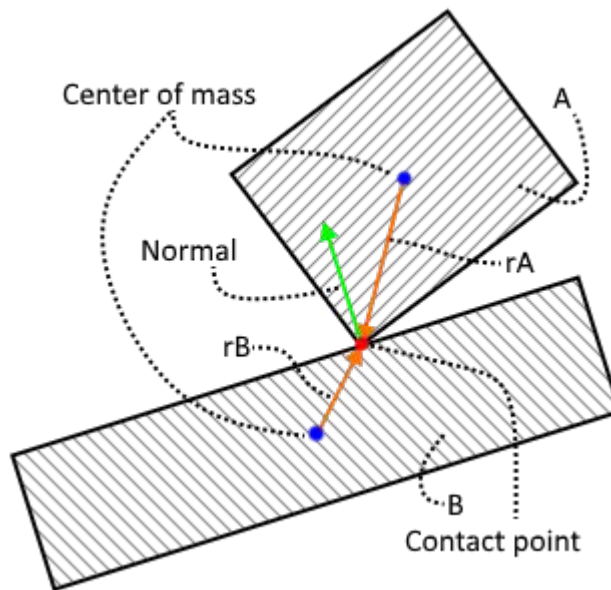


Illustration 76: Collision variables

"rA" and "rB" is vectors from polygon A's center of mass to the contact point, and from polygon B's center of mass to the contact point. The normal is a unit vector that is pointing in the direction A would experience an impulse. If the contact point on polygon A is moving trough the surface of B (in the negative direction of the normal) relative to the movement of the contact point on polygon B, some impulse must be applied to resolve the collision. This impulse would be applied in the normal direction at polygon A, and an equal and opposite force would be applied to B at the same position. This ensures conservation of translational and angular momentum. Since the collision is elastic, the energy will be conserved.

$I :$

$$\begin{aligned} \frac{1}{2}m_A \cdot v_{A0}^2 + \frac{1}{2}m_B \cdot v_{B0}^2 + \frac{1}{2}\omega_{A0}^2 \cdot I_A + \frac{1}{2}\omega_{B0}^2 \cdot I_B &= \frac{1}{2}m_A \cdot v_{A1}^2 + \frac{1}{2}m_B \cdot v_{B1}^2 + \frac{1}{2}\omega_{A1}^2 \cdot I_A + \frac{1}{2}\omega_{B1}^2 \cdot I_B \\ m_A (v_{A0} - v_{A1}) (v_{A0} + v_{A1}) + m_B (v_{B0} - v_{B1}) (v_{B0} + v_{B1}) + \\ I_A (\omega_{A0} - \omega_{A1}) (\omega_{A0} + \omega_{A1}) + I_B (\omega_{B0} - \omega_{B1}) (\omega_{B0} + \omega_{B1}) &= 0 \end{aligned}$$

Illustration 77: Conservation of kinetic energy

m_A and m_B are the masses of the polygons. I_A and I_B are the moments of inertia. Omega is the angular velocity and v is the translational velocity. Variables marked with A and B that show what polygon they belong to, and 0 and 1 to show if they refer to the moment before or after the impulse is applied.

$II :$

$$i_A = -m_A (v_{A0} - v_{A1})$$

$III :$

$$i_B = -m_B (v_{B0} - v_{B1})$$

$IV :$

$$L_A = I (\omega_{A0} - \omega_{A1})$$

$V :$

$$L_B = I (\omega_{B0} - \omega_{B1})$$

$II, III, IV, V \rightarrow I :$

$$i_A (v_{A0} + v_{A1}) + i_B (v_{B0} + v_{B1}) + L_A (\omega_{A0} + \omega_{A1}) + L_B (\omega_{B0} + \omega_{B1})$$

Illustration 78: Impulse definitions

L_A and L_B are the angular impulses applied to the polygons.

VI :

$$i\vec{A} = i\vec{n} = -i\vec{B}$$

VII :

$$L_A = \left| \frac{\vec{r}_A}{i\vec{A}} \right| = i \left| \frac{\vec{r}_A}{\vec{n}} \right|$$

VIII :

$$L_B = -i \left| \frac{\vec{r}_B}{\vec{n}} \right|$$

VI, VII, VIII $\rightarrow$ I :

$$i\vec{n} (v_{A0} + v_{A1}) - i\vec{n} (v_{B0} + v_{B1}) + i \left| \frac{\vec{r}_A}{\vec{n}} \right| (\omega_{A0} + \omega_{A1}) - i \left| \frac{\vec{r}_B}{\vec{n}} \right| (\omega_{B0} + \omega_{B1}) = 0$$

the first solution: $i = 0$

$$\vec{n} (v_{A0} + v_{A1}) - \vec{n} (v_{B0} + v_{B1}) + \left| \frac{\vec{r}_A}{\vec{n}} \right| (\omega_{A0} + \omega_{A1}) - \left| \frac{\vec{r}_B}{\vec{n}} \right| (\omega_{B0} + \omega_{B1}) = 0$$

Illustration 79: The first solution

r_A and r_B and the normal n , can be seen in the illustration 76. We now has the first solution for i . If i is zero, the impulse vector is also zero and energy and momentum is obviously conserved. This is not the solution we are looking for.

II :

$$v_{A1} = v_{A0} + \frac{i\vec{n}}{m_A}$$

III :

$$v_{B1} = v_{B0} - \frac{i\vec{n}}{m_B}$$

VII $\rightarrow$ IV :

$$I_A (\omega_{A1} - \omega_{A0}) = i \left| \frac{\vec{r}_A}{\vec{n}} \right|$$

$$\omega_{A1} = \omega_{A0} + \frac{i \left| \frac{\vec{r}_A}{\vec{n}} \right|}{I_A}$$

VIII $\rightarrow$ V :

$$I_B (\omega_{B1} - \omega_{B0}) = -i \left| \frac{\vec{r}_B}{\vec{n}} \right|$$

$$\omega_{B1} = \omega_{B0} - \frac{i \left| \frac{\vec{r}_B}{\vec{n}} \right|}{I_B}$$

II, III, IV, V $\rightarrow$ I :

$$i = 2 \frac{\left(\vec{n} (v_{A0} - v_{B0}) + \left| \frac{\vec{r}_A}{\vec{n}} \right| \omega_{A0} - \left| \frac{\vec{r}_B}{\vec{n}} \right| \omega_{B0} \right)}{\frac{1}{m_A} + \frac{1}{m_B} + \frac{\left| \frac{\vec{r}_A}{\vec{n}} \right|^2}{I_A} + \frac{\left| \frac{\vec{r}_B}{\vec{n}} \right|^2}{I_B}}$$

$$\text{relativeVelocity} = \vec{n} (v_{A0} - v_{B0}) + \left| \frac{\vec{r}_A}{\vec{n}} \right| \omega_{A0} - \left| \frac{\vec{r}_B}{\vec{n}} \right| \omega_{B0}$$

$$\text{collisionMass} = \frac{1}{\frac{1}{m_A} + \frac{1}{m_B} + \frac{\left| \frac{\vec{r}_A}{\vec{n}} \right|^2}{I_A} + \frac{\left| \frac{\vec{r}_B}{\vec{n}} \right|^2}{I_B}}$$

Illustration 80: Collision formulae

We have now solved the problem. To organize the solution the numerator will be called `relativeVelocity` and one over the denominator, `collisionMass`. The relative velocity is A's velocity relative to B's velocity at the contact point projected in the normal direction. If this number is negative the objects are moving apart and the impulse should not be applied. The collision mass is the mass you would perceive polygon B to have if you are standing on the contact point on polygon

A and are pushing polygon B in the normal direction assuming polygon A has an infinite mass. The collision mass is the perceived resistance to change in velocity between bodies.

To apply this impulse to the polygon, the `Polygon.applyImpulse()` function is used. See page 106.

To be able to calculate inelastic collision and semi inelastic collisions, the calculated impulse must remove energy. This is a graph of the energy of the system after the impulse has been added.

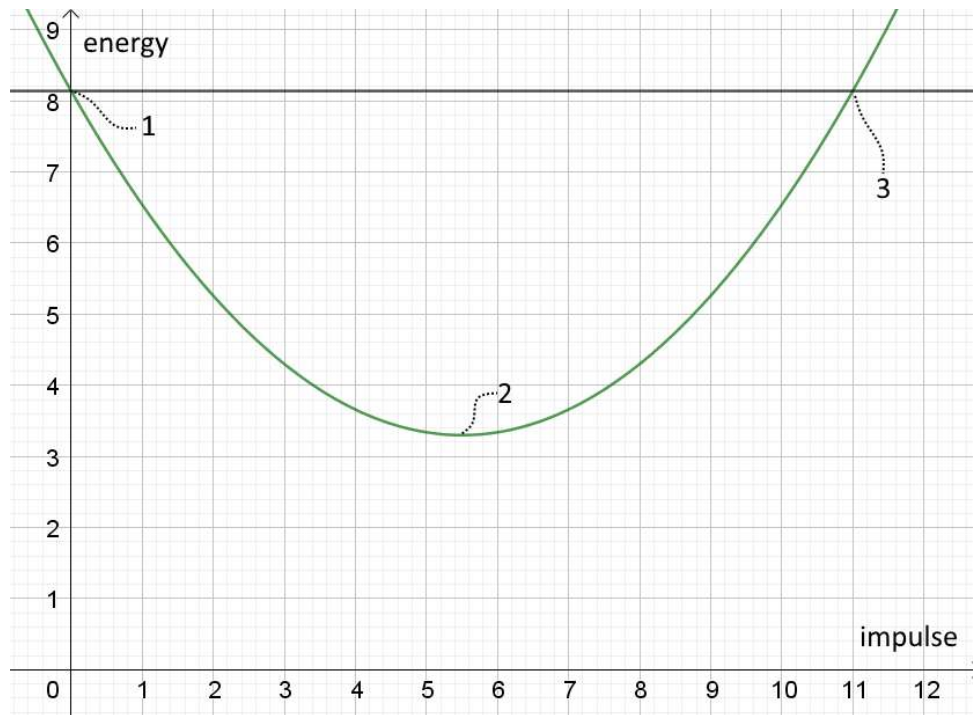


Illustration 81: Energy curve

This is a general example of how such a curve might look like, if the collision is perfectly central with no sliding, point 2 would intersect with the impulse axis at one point.

At 1, the impulse applied is 0, so energy is not changed. The black line going from 1 to 3 show the original energy. At 3 is the second solution that conserves energy. The green energy curve is a quadratic curve since kinetic energy has a quadratic relation to its translational and angular velocities which in turn has a linear relation to the impulse as we can see from the final versions of equation II, III, IV and V. Since a quadratic equation is symmetric, halving the inelastic impulse (point 3 in illustration 81) will result in the minimum final energy between the polygons as a result of the collision. To achieve this, a new variable "e" (for elasticity) is introduced. The number 2 is replaced with $(1+e)$.

$$i = (1 + e) \cdot \text{relativeVelocity} \cdot \text{collisionMass}$$

Illustration 82: Collision formulae with friction

If e is 1, the collision is perfectly elastic and if e is 0, it is perfectly inelastic.

6.7.11 Area of a polygon

To calculate the area, Green's Theorem is used ((18) or (19)). As we can see in Calculus page 922, we can derive the following formulae from Green's Theorem.

$$\oint_C x \, dy = \text{area of } R$$

Illustration 83: Area formulae

C is the outer boundary of the polygon and R is the region bounded by C . The area of R is the area of the polygon. To calculate the closed line integral, a line integral will be taken over one line segment at a time, and then the results will be summarized together.

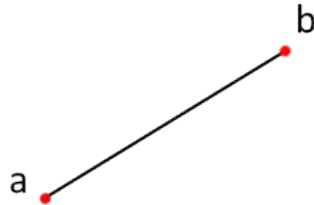


Illustration 84: Line segment

Here is an example of such a line segment. We want to do the integration over this segment. a and b is the beginning position and end position of the segment.

$$\begin{aligned}
\int_S x \, dy &= \lim_{n \rightarrow \infty} \sum_{i=1}^n x \cdot \frac{b_y - a_y}{n} = \\
&\lim_{n \rightarrow \infty} (b_y - a_y) \sum_{i=1}^n x \frac{1}{n} = \\
&(b_y - a_y) (\text{average } x) = \\
&\frac{1}{2} (b_y - b_a) (a_x + b_x)
\end{aligned}$$

Illustration 85: Integrated line segment

The segment is called S. b_x, a_x , and b_y, a_y are the x and y coordinates of the points.

If we sum up the solution for each line segment in the positive direction, we will get the area of the polygon. The outer circumference has positive direction in the counter clockwise direction, so that will give us a positive area. The holes has positive direction in the clockwise direction, so that gives us a negative areas. If we sum together the line integrals for all circumferences, we get the total area of the polygon.

6.7.12 Center of mass of polygon

To calculate the center of mass of a polygon the polygon is split into triangles. The mass and center of mass of each triangle is calculated. This information is then combined to find the mass and center of mass of the whole polygon.

To find the mass of a triangle, the determinant is used.

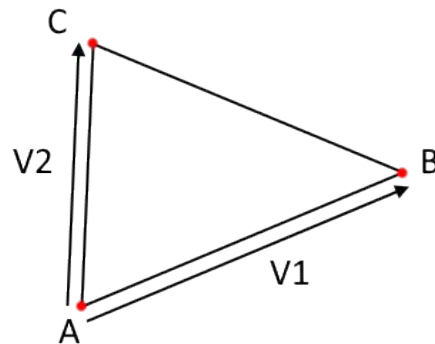
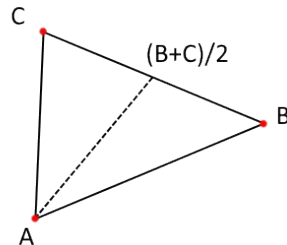


Illustration 86: ABC triangle

V1 is the difference of B and A, and V2 is the difference of C and A. The determinant of V1 and V2

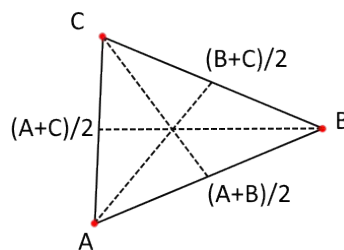
gives the area of the parallelogram A , B , $B+V_2$, and C . If we half this, we get the area of the triangle.

The center of mass of a triangle, is the average positions of its corners. Here is the proof:



*Illustration 87: Axis
through center of mass*

If you draw a line through a triangle from A through the middle of the opposite line segment, you split the triangle in two areas that are equal in size. The distance from the areas to the line is also the same on each side since C and B are equally far from the line. Since the areas are the same in size and distance to the line, the center of mass must be somewhere on this line. You can do the same with all corners.



If we write down the equations for these lines we get:

$$\begin{aligned}
I : P &= \frac{A+B}{2} + \left(C - \frac{A+B}{2}t \right) \\
II : P &= \frac{B+C}{2} + \left(A - \frac{B+C}{2}t \right) \\
III : P &= \frac{A+C}{2} + \left(B - \frac{A+C}{2}t \right) \\
I + II + III : \\
3P &= A + B + C \\
P &= \frac{A + B + C}{3}
\end{aligned}$$

Illustration 88: Center of mass formulae

P is the center of mass. t is a variable giving each line its degree of freedom.

Now as we know the mass of the triangle and the center of mass we can calculate the center of mass of the entire polygon. If we follow the definition of center of mass, we get:

$$\text{total center of mass} = \frac{\sum_{i=1}^n \text{triangleMass}_i \cdot \text{triangleCenterOfMass}_i}{\sum_{i=1}^n \text{triangleMass}_i}$$

Illustration 89: Total center of mass formulae

n is the number of triangles.

The process of turning triangles into triangles can be simplified. One can simply remove corner by corner from the polygon at calculate the center of mass and mass of the triangles emerging from the difference of the polygon as a corner is removed. This means that sometimes, you would get a negative area and thereby a negative mass. This would happen if the corner removed is concave. Since removing a concave corner, increases the area of the polygon, the negative mass accumulated, will be removed at a later stage.

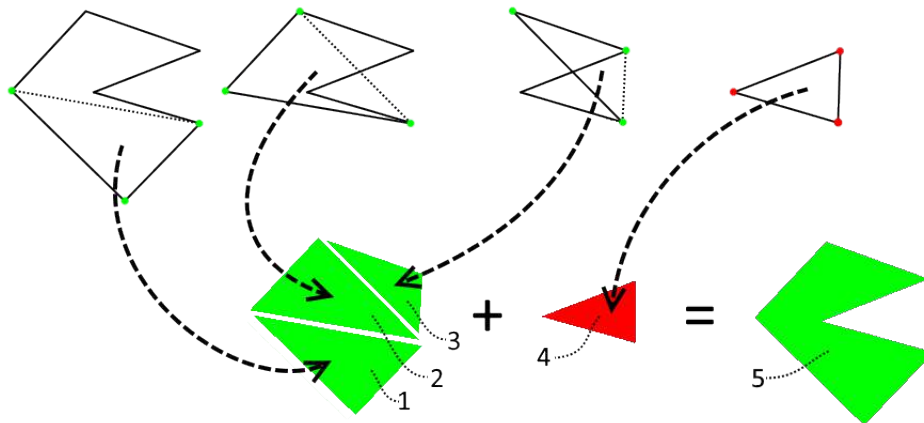


Illustration 90: Area cancellation

Triangle 1, 2, and 3 have positive areas since they are counter clockwise. Triangle 4 has a negative area since it is clockwise. The positive and negative give a sum equal to the area of the whole polygon.

Code example:

```
// calculate mass.
var totalArea = 0;
var totalCenterOfMass = new Vec2();
for (var i = 0; i < this.geometry.length; i++) {
  var c = this.geometry[i];
  for (var j = 2; j < c.length; j++) {
    // Define triangle. Counter clockwise orientation.
    var p2 = c[j].position;
    var p1 = c[j-1].position;
    var p0 = c[0].position;
    // Calculate mass of this triangle.
    var v1 = Vec2.sub(p2, p0);
    var v2 = Vec2.sub(p1, p0);
    var mass = Vec2.det(v2, v1) * 0.5;
    var centerOfMass = Vec2.addMany(p0, p1, p2).div(3);
    totalArea += mass;
    totalCenterOfMass.add(centerOfMass.mul(mass));
  }
}
totalCenterOfMass.div(totalArea);

this.area = totalArea;
this.mass = totalArea * this.density;
```

"this" refers to the polygon itself. This code can be found in app/modules/sim/_static/physics_engine/polygon/polygon.js Polygon.calculateInertia().

6.7.13 Moment of inertia of polygon

The approach is to first split the polygon into triangles, then calculate the moment of inertia of each triangle. For each triangle, the moment of inertia will be calculated with respect to the center of mass of the entire polygon and then added together to get the moment of inertia of the whole polygon.

We will start with calculating the moment of inertia of a triangle.

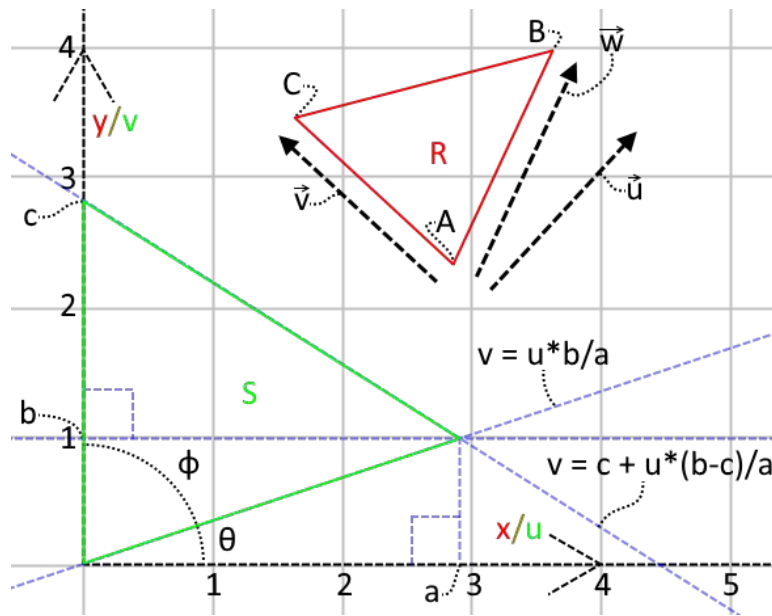


Illustration 91: Transformation of coordinate system

The triangle we want to calculate the moment of inertia of is A, B, C. This is the red triangle. It exists within the x, y coordinate system. We will begin with calculating the moment of inertia round corner A. To simplify the calculation we will change coordinate system from the x, y coordinate system to the u, v coordinate system (the green triangle). When we do this transformation, we change the triangle in 3 ways. We translate, rotate and scale the triangle.

To compensate for the translation, we will find the moment of inertia around the origin, instead of A. This would be the same as finding the moment of inertia around A as A maps to the origin after the transformation.

To compensate for the rotation, we don't need to do anything as rotation around the axis we calculate the moment of inertia around, does not change the moment of inertia.

We also need to compensate for the scaling. When the triangle is scaled, the moment of inertia will change. We will wait with this problem for later, and begin with calculating the moment of inertia

of the transformed (green) triangle, S. We will assume S has a density of 1, so density factor will be omitted.

$$\begin{aligned}\vec{v} &= C - A \\ \vec{u} &= (v_y, -v_x) \\ \vec{w} &= B - A \\ a &= w \cdot u = \left| \frac{\vec{w}}{\vec{v}} \right| = |\vec{w}| \sin \phi |\vec{v}| \\ b &= \vec{w} \cdot \vec{v} = |\vec{w}| \cos \theta |\vec{v}| \\ c &= \vec{v} \cdot \vec{v} = |\vec{v}|^2\end{aligned}$$

$$\int_0^a du \int_{\frac{b}{a}u}^{c+\frac{b-c}{a}u} dv (u^2 + v^2) = ac \frac{a^2 + b^2 + c^2 + bc}{12} = I_S$$

Illustration 92: Moment of inertia of S

The integration was done using the computer algebra system Maxima (15). I_S is the moment of inertia of S.

What is now left to do, is to compensate for the scaling. As mentioned before, when the triangle in the x, y coordinate system (the red one, R), is transformed into the u, v coordinate system (the green one, S), translation, rotation, and scaling is done. Since the former two is already dealt with, we will only focus on the scaling. To simplify the calculation we will assume we do not move from the x, y coordinate system to the u, v coordinate system, but instead to the u', v' coordinate system. When transforming the triangle from the x, y coordinate system to the u', v' system, only scaling is done. The change to the rotation mass during this transformation should be the same as when transforming to the u, v system.

$$\begin{aligned}(x, y) &\rightarrow (u', v') \\ \sqrt{c} &= |\vec{v}| \\ u' &= x\sqrt{c} \quad v' = y\sqrt{c}\end{aligned}$$

Illustration 93: Alternative transformation of coordinate system

When transforming R into S, the only difference in moment of inertia (calculated around corner A of R which is the equivalent of the corner in the origin of S) is due to the scaling. The triangle is scaled by the length of vector v which is the same as the square root of c . We define the transformation from R to S' as shown in illustration 93. S' is scaled by the same factor as S.

$$\begin{aligned}
 x &= \frac{u'}{\sqrt{c}} & y &= \frac{v'}{\sqrt{c}} \\
 \frac{du'}{dx} &= \sqrt{c} & \frac{dv'}{dy} &= \sqrt{c} \\
 dx &= \frac{du'}{\sqrt{c}} & dy &= \frac{dv'}{\sqrt{c}} \\
 \iint_R r dM &= \iint_R (x^2 + y^2) \rho dA = \\
 &\rho \iint_R (x^2 + y^2) dx dy = \\
 &\rho \iint_{S'} \left(\left(\frac{u'}{\sqrt{c}} \right)^2 + \left(\frac{v'}{\sqrt{c}} \right)^2 \right) \frac{du'}{\sqrt{c}} \frac{dv'}{\sqrt{c}} = \\
 &\frac{\rho}{c^2} \iint_{S'} (u'^2 + v'^2) du' dv' = \\
 &\frac{\rho}{c^2} I_{S'} = \frac{\rho}{c^2} I_S = \\
 &\rho a \frac{a^2 + b^2 + c^2 + bc}{12c} = I_R
 \end{aligned}$$

Illustration 94: Moment of inertia of R

Rho is the density of the polygon. When calculating I_S , we assumed the density was 1, so we ignored it. The moment of inertia of the triangle R is I_R .

Now that we can have I_R , we can use the parallel axis theorem (20) to find the moment of inertia around the triangles center of mass, and then the polygons center of mass. We can then sum together all the moment of inertia's of all the triangles. Just as with the calculation of the mass of a polygon, it is not necessary to split the polygon into non-negative triangles. Any three subsequent triangles can be chosen. After the moment of inertia is calculated for one triangle, it is removed before the next moment of inertia is calculated. If the chosen corner gives a negative moment of inertia (which will happen automatically if this formula is applied to a clockwise triangle), the moment of

inertia will be added back when another positive triangle is calculated. The idea is similar to what is shown in illustration 90.

The implementation of the formula can be found at

[app/modules/sim/_static/physics_engine/polygon/polygon.js](#) Polygon.calculateInertia().

6.7.14 Random seed generator

The generator has a seed which is an integer. For each random number returned, the seed (or iterator) will increase. The random number returned is larger or equal to 0 and less than 1. The number is not completely random, but is sufficiently random for its uses in the simulator. To generate a random number, 17 digits are selected from different lists of different prime lengths filled with randomized digits 0 to 9. The lengths of the lists are the prime numbers 307 to 401. Since JavaScript uses 64 bit floating points, 17 significant digits (10^{th} decimal system) is enough to achieve maximum precision. The lists will each be iterated over. When one list reaches the last digit, it will start from the top. Since the lengths of the lists are prime, they all have a greatest common divisors of 1. This ensures that all combinations of digits from the lists are returned before the random pattern repeats itself. This means that the generator can generate $307 \cdot 311 \cdot \dots \cdot 397 \cdot 401$ which is approximately $2.0 \cdot 10^{43}$ random numbers.

6.7.15 Variants of applying impulses and displacements

There are many different ways of applying impulses and displacements to polygons. It is important that when changes are added to the system, that momentum is conserved according to the second axiom at page 32.

Apply impulse

The code is found at [app/modules/sim/_static/physics_engine/polygon/polygon.js](#). The signature is:

```
Polygon.prototype.applyImpulse = function (position, impulse)
```

The position is a 2d vector describing the global position where the impulse is applied to the polygon. The impulse is a 2d vector.

Displace by impulse

The code is found at [app/modules/sim/_static/physics_engine/polygon/polygon.js](#) The signature is:

```
Polygon.prototype.displaceByImpulse = function (position, impulse)
```

The function is similar to `Polygon.applyImpulse()`. Instead of actually applying an impulse, only displacement is applied. The function acts as if the polygon starts of with zero velocities (angular and translational) and then has this impulse applied to it. Then one tick passes, and the polygon changes position. This displacement in position is the displacement added to the polygon.

Set relative velocity

The method described below is used in several functions. The cleanest implementation can be found at [app/modules/sim/_static/physics_engine/hinge_constraint.js](#)

`HingeConstraint.resolveByImpulse()`.

The goal is to find an impulse vector, if applied between two polygons at a point, the polygons will end up having a relative velocity of zero at that point. This is useful for hinges, as you want to stop the polygons relative movement at a point to make it seem they are hinged together.

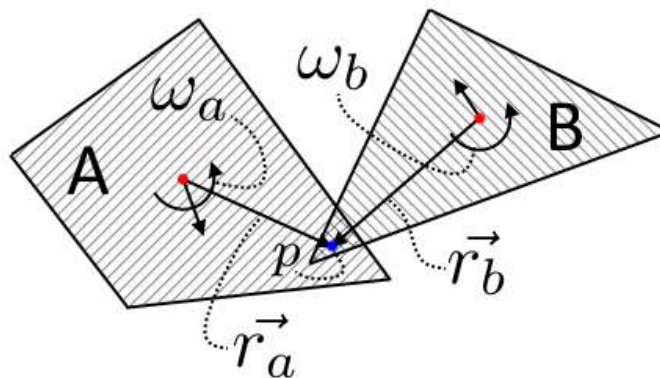


Illustration 95: Hinged polygons

ω_a and ω_b is the angular velocities of the polygons. r_a and r_b is vectors from the centers of mass to the point p . p is the point where we want the relative velocity to be 0.

$$\begin{aligned}
& \vec{r}_a \quad \vec{r}_b \\
& \vec{r}'_a = (-r_{ay}, r_{ax}) \\
& \vec{r}'_b = (-r_{by}, r_{bx}) \\
& \vec{v}_p = \vec{v}_b - \vec{v}_a + \vec{r}'_b \omega_b - \vec{r}'_a \omega_a \\
& \Delta v_p = \frac{\vec{i}}{m_b} + \frac{\vec{i}}{m_a} + \frac{\left| \vec{r}'_b \right|}{I_b} r'_b + \frac{\left| \vec{r}'_a \right|}{I_a} r'_a
\end{aligned}$$

Illustration 96: Delta relative velocity at attachment point

r'_a and r'_b is r_a and r_b rotated 90 degrees counter clockwise. v_p is the relative velocity of the polygons at p as seen from a 's perspective. Delta v_p is the change in v_p when an impulse i is applied at p . We want to solve these equations for i . With help from maxima (15), we were able to solve the equations:

$$\begin{aligned}
\alpha &= -r_{by}\omega_b + v_{bx} + r_{ay} - v_{ax} \\
\beta &= r_{bx}\omega_b + v_{by} - r_{ax}\omega_a - v_{ay} \\
\gamma &= \frac{r_{by}^2}{I_b} + \frac{1}{m_b} + \frac{r_{ay}^2}{I_a} + \frac{1}{m_a} \\
\delta &= \frac{r_{bx}^2}{I_b} + \frac{1}{m_b} + \frac{r_{ax}^2}{I_a} + \frac{1}{m_a} \\
\epsilon &= -\frac{r_{bx}r_{by}}{I_b} - \frac{r_{ax}r_{ay}}{I_a} \\
\zeta &= \epsilon^2 - \delta\gamma \\
\vec{i} &= \frac{(\beta\epsilon - \alpha\delta, \alpha\epsilon - \beta\gamma)}{\zeta}
\end{aligned}$$

Illustration 97: Stopping impulse

Alpha and *beta* is the x and y component of the relative velocity of p . This fact is used in various implementations of this formula. For example in Polygon.applyDisplacement(). i is the impulse

required to make polygon A and polygon B have zero relative velocity at point p .

7 Process

Collision detection and resolution require high precision in order to function. A tiny error in collision detection could easily cause objects to fall through each other, or get stuck inside each other. A bug in the collision resolution could cause angular momentum to not be conserved, which again could lead to the entire system quickly spinning out of control. To detect bugs, thorough testing is essential.

To test the simulation, debug tools were developed and used from the beginning of the development. We made debug modes that allowed the camera to move freely without being affected by the simulation, and rendered debug information to the screen. If a bug occurred, we could freeze the time, move the camera to the problematic area, and zoom in on the position where the problem occurred. The formulas we used in the simulation was tested using the math tool Geogebra (21). When things were not acting as they should, we had functions that would export the data from the simulation to a format compatible with Geogebra, so we could analyze the problem further.

To detect bugs early, we wrote code that would test the rest of the code. If polygons ended up in an invalid overlapping state, the program would pause, and information about the error be displayed. If momentum was suddenly not conserved, or energy started building up, this would be detected at an early stage. The earlier the problems were detected, the easier the isolation step of debugging became, as new problems probably is related to recent changes in code. To isolate bugs we also needed to reproduce them. Since parts of our code take uses of random values, the program would not be deterministic if we used JavaScripts `Math.random()` function. Since JavaScripts random function cannot be made deterministic, we made our own random generator that is deterministic. The code can be found at [app/modules/sim/_static/physics_engine/utilities/random_seed.js](#). The math behind the deterministic generator can be found at 6.7.14. Since the number sequence of pseudo random numbers is the same for each time the simulation is run, the simulation is deterministic. We ran automatic tests that would generate a random seed using JavaScripts random function that would be feed to our deterministic random generator. After the testing code found a problem, the testing would stop and we would be given the seed needed to reproduce the problem.

One of the tests ran, was one where 100 polygons would be placed pseudo randomly around the scene and then a force would attract them towards each other. They would crash violently against

each other, which lead to bugs. The testing code detects the problem and gives us the data needed to reproduce the problem. After the bug causing the problem is fixed, the process is repeated. In the beginning the test could run for 1 second before a problem would occur, then 10 seconds, then 100 seconds and so on. At the end, it could take hours before a bug would occur, but that was not a problem for us, as the computer could do the testing without our presence. This led to a very stable product.

To test the static and dynamic friction, we had various boxes slide across each other and have the sliding lengths measured. Here is some data from dynamic and static friction testing.

Static:

```

tan(angle) = calculated frictionconstant ~ actual friciton constant
Method 1:
tan(0.15770262849214073) = 0.15902312748 ~ 0.16
tan(0.34606289027436427) = 0.36057316175 ~ 0.36
Method 2:
tan(0.009983290408217679) = 0.00998362208 ~ 0.01
tan(0.03998549428279891) = 0.04000681805 ~ 0.04
tan(0.15868535251323931) = 0.16003086078 ~ 0.16
tan(0.3456101169970297) = 0.36006160554 ~ 0.36
tan(0.5693613534737539) = 0.64006789177 ~ 0.64
tan(0.7856425844858911) = 1.0004889617 ~ 1.
tan(0.8782496874867649) = 1.20536170368 ~ 1.21
tan(0.9630725388827588) = 1.43773984508 ~ 1.44
tan(1.099143613352996) = 1.96060436857 ~ 1.96
tan(1.198481603922418) = 2.56063255171 ~ 2.56

Average absolute error: 0.0008807887279999782.
Average percentage error: 0.08744991861461869%.

```

Dynamic:

```

Distance units
Measured:          Predicted:
32.95 ~ 33.109453125

Error:
Absolute: 0.159453125
Percentage: 0.483924506%

```

The static tests were performed by slightly changing the tilt angle of a static polygon with a box on top. The angle data shown is when the friction changed from static to dynamic (the box started sliding). We used this data to calculate what the static friction should be. After fixing all the bugs, we saw this friction constant become increasingly closer to the desired result. The dynamic data was obtained by sliding a polygon along a surface and measuring the sliding length. This length was compared to the calculated length based on the dynamic friction constant.

The architecture of the code is a hierarchical one. Modules further down the hierarchy is unaware of

the specific code of the modules higher up the hierarchy. The parent modules is the ones to establish contact with the child modules by setting listeners and calling functions, following the interfaces defined by the child modules. The main architecture of this application is the web part of the application at the top of the hierarchy, then the physics simulation as a child of that. The physics simulation has two child modules. One is the physics engine, and the other one is the graphics engine. There are also various helping libraries, like the vector library, that act as a child library, or a support library, for all modules.

For version control we used GitHub. We implemented functionality in branches. These were merged into a development branch. The development branch should be runnable. The most significant and stable versions of the dev branch was merged into a master branch.

8 Conclusion

The specification of our application was to address the problem students of FYS100 faced when studying physics. We wanted to provide a supplement to the curriculum, an alternative and more intuition oriented approach to understand the content of the course. We have achieved a proof of concept approaching this problem with our platform. The platform is easily accessible from the web and provides the functionality to connect, cooperate and share learning experiences. We look forward to further develop the platform.

9 References

- (1) https://phet.colorado.edu/sims/html/pendulum-lab/latest/pendulum-lab_nb.html Pendulum example.
- (2) <https://github.com/krasimir/navigo> Navigo by Krasimir Tsonev.
- (3) <http://bradfrost.com/blog/post/atomic-web-design/> Atomic Design pattern.
- (4) <http://sam.js.org/> SAM Pattern.
- (5) <https://github.com/dimaba/sendmail> Python Email Sender
- (6) <https://www.tutorialspoint.com/webgl/index.htm> Learn WebGL.
- (7) <https://webglfundamentals.org/webgl/lessons/webgl-3d-perspective.html> Matrices.

-
- (8) https://en.wikipedia.org/wiki/OpenGL_Shading_Language GLSL.
 - (9) <https://learnopengl.com/Getting-started/Coordinate-Systems> Coordinate systems.
 - (10) <https://nb.khanacademy.org/science/physics/one-dimensional-motion/kinematic-formulas/a/what-are-the-kinematic-formulas> Kinematic formulas.
 - (11) <https://learnopengl.com/In-Practice/2D-Game/Collisions/Collision-detection> Axis aligned bounding boxes. The first sub heading talk about axis aligned collision boxes.
 - (12) <https://gamedevelopment.tutsplus.com/tutorials/quick-tip-use-quadtrees-to-detect-likely-collisions-in-2d-space--gamedev-374> Quad tree.
 - (13) https://en.wikipedia.org/wiki/Space_partitioning Space partitioning.
 - (14) Joseph O'Rourke (1998). Computational Geometry in C. (2nd ed.). Cambridge University Press. ISBN 0-521-64976-5. Chapter 7.4 Page 239.
 - (15) <http://maxima.sourceforge.net/> Computer algebra system. Maxima.
 - (16) <https://math.stackexchange.com/questions/311382/solving-a-quadratic-equation-with-precision-when-using-floating-point-variables> Quadratic equation with precision.
 - (17) <https://www.myphysicslab.com/engine2D/collision-en.html> Collision resolution formula.
 - (18) Robert A. Adams and Christopher Essex (2013). Calculus: A Complete Course. Eighth Edition. ISBN-13: 978-0321781079. Page 922.
 - (19) <http://tutorial.math.lamar.edu/Classes/CalcIII/GreensTheorem.aspx> Green's Theorem.
 - (20) https://en.wikipedia.org/wiki/Parallel_axis_theorem Parallel axis theorem.
 - (21) <https://www.geogebra.org/> Geogebra.
 - (22) <http://flask.pocoo.org/> Flask.

10 Word list

- i. WSGI Web Server Gateway Interface
- ii. SPA Single Page Application
- iii. JS JavaScript
- iv. MVC Model View Controller
- v. jML javascript Markup Language
- vi. GUI Graphical User Interface
- vii. SAM State-Action-Model